

Advanced Machine learning Mastering Course

Deep Learning

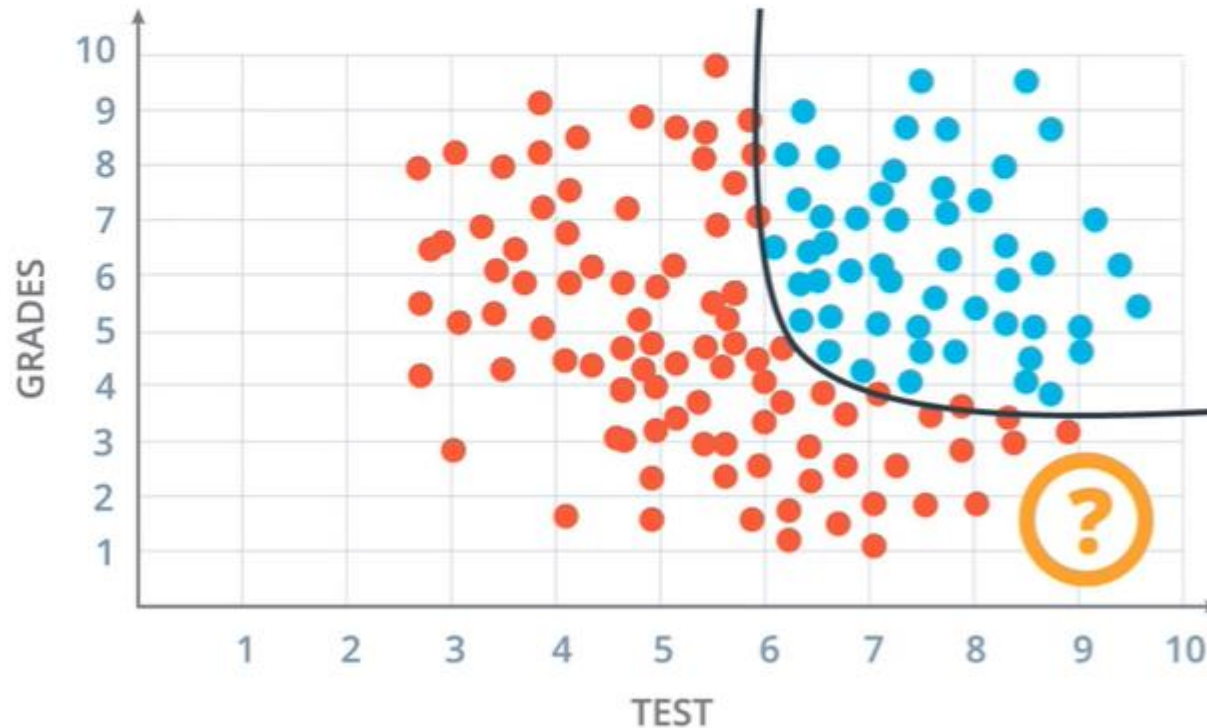
Feed forward- Back propagation

Agenda

- Artificial Neural Networks
 - Feed Forward
 - Backpropagation
 - Activation Functions
 - Weights Initialization
- Deep Neural Networks
 - Stochastic ,Mini-batch and Batch Gradient Descent
 - Deep Learning Frameworks
 - DNN Different Architectures

Acceptance at a University

That would also work. So let's go with that.
Let's go with the curve.



"If your grades were terrible, who matters in the test, this no matter what you got on student's book and grades. The only line on the grades. This model for accepting it stipulated over here in the positive region of this line. So our data should look more like this instead. But let's say we don't want this model, is much more realistic but now we have a problem which is the data can no longer be separated by just a line. So what is the next thing after a line?"

STUDENT 4

Test: 9/10

Grades: 1/10

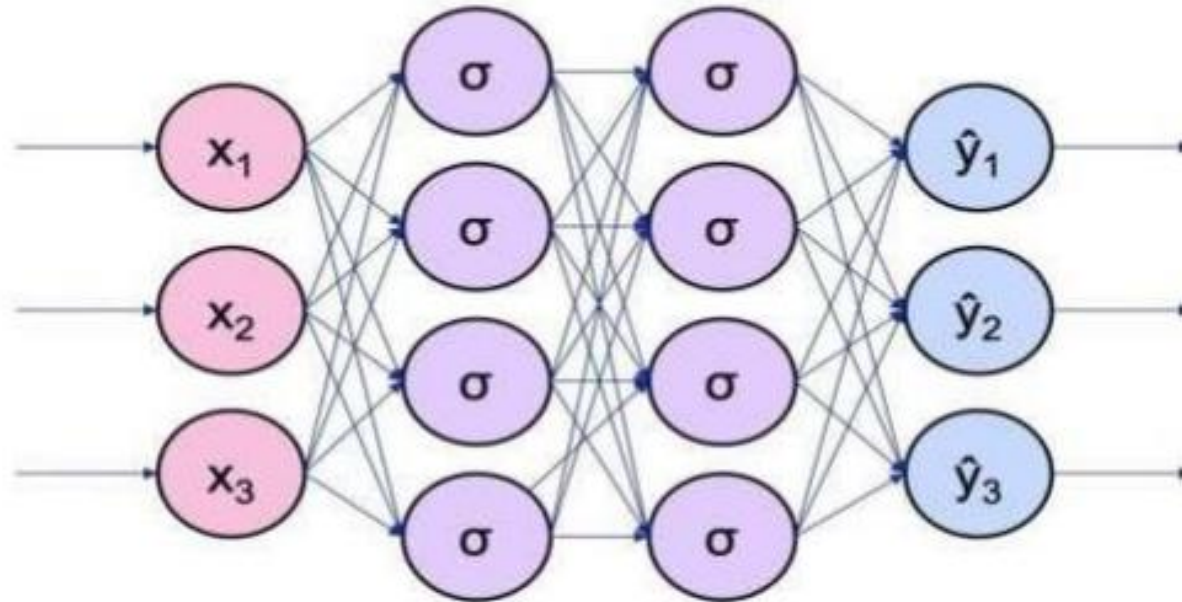
Maybe a circle. A circle would work.

Maybe two lines. That could work, too.

Or maybe a curve like this.

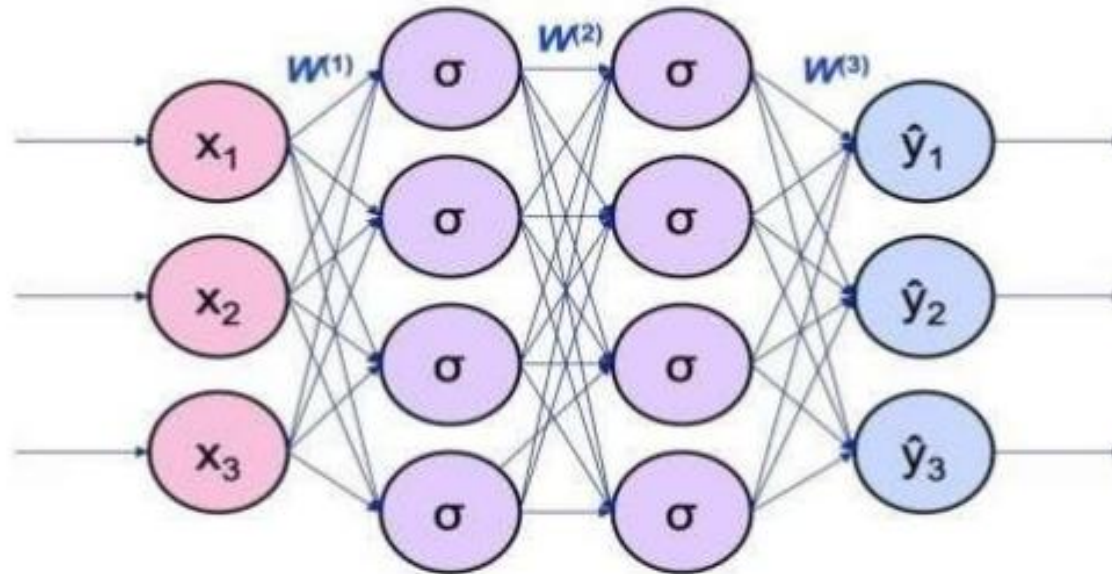
Artificial Neural Network

Multilayer Perceptron: Weights



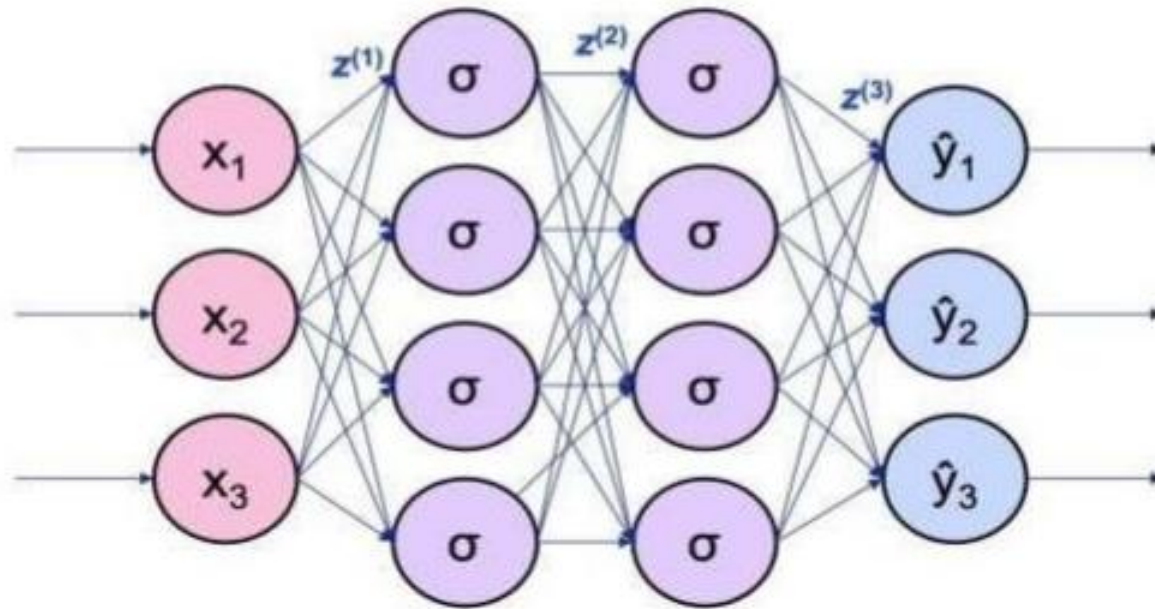
Artificial Neural Network

Weights: Represented By Matrices



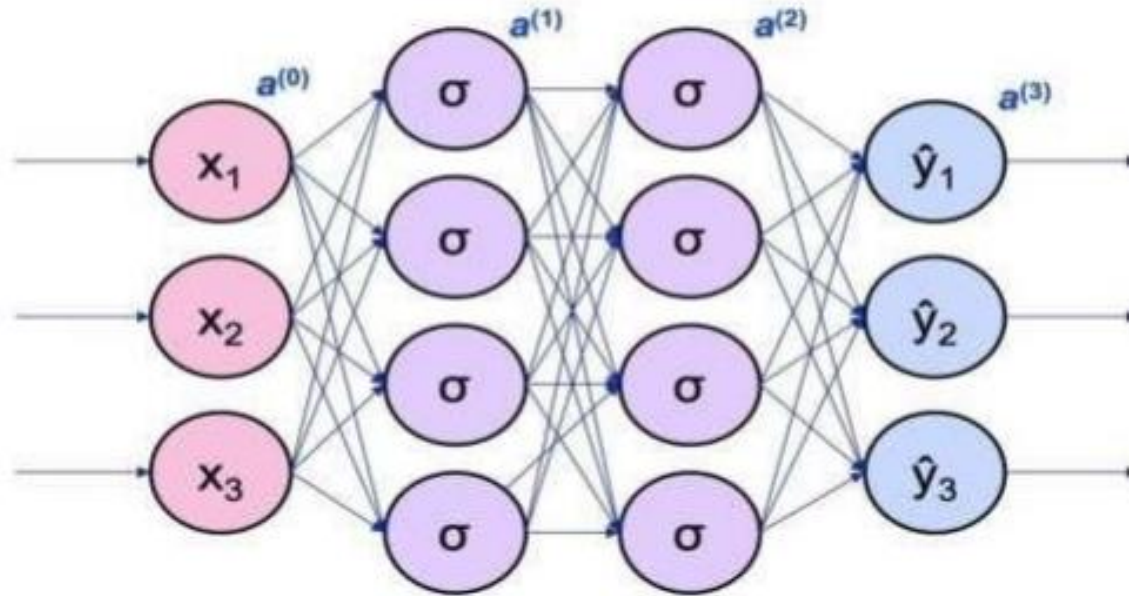
Artificial Neural Network

Net Input: Sum of Weighted Inputs



Artificial Neural Network

Activations: Output To Next Layer



Artificial Neural Network

Matrix Representation of Computation

For a single data point (instance):

$W^{(1)}$ is a 3x4 matrix

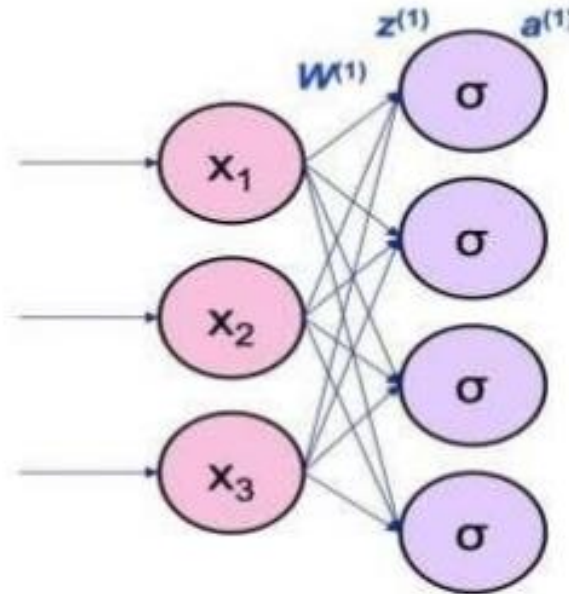
$z^{(1)}$ is a 4-vector

$a^{(1)}$ is a 4-vector

x ($x = a^{(0)}$)

$z^{(1)} = xW^{(1)}$

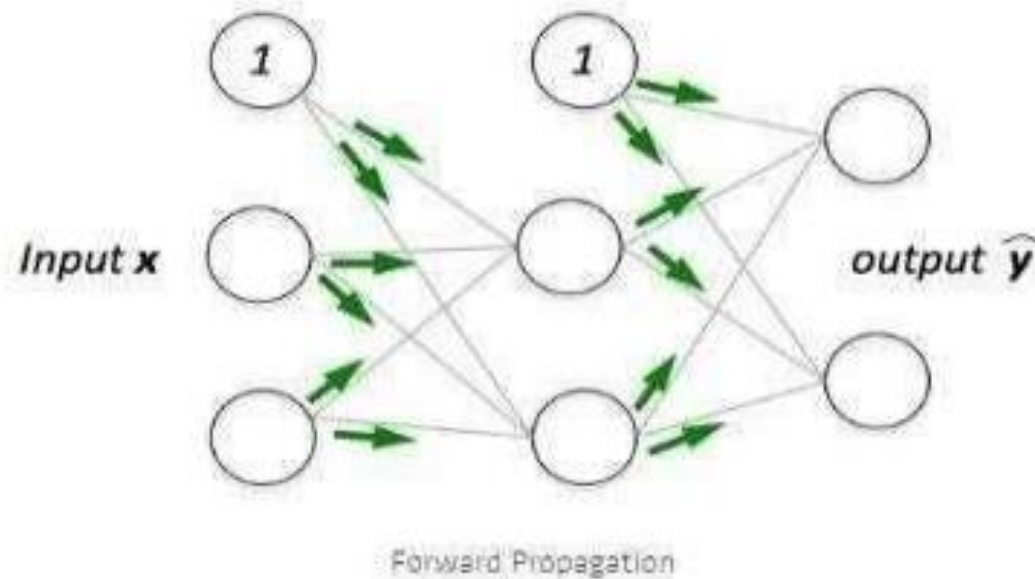
$a^{(1)} = \sigma(z^{(1)})$



Forward Propagation

- It is a process of feeding input values to the neural network and getting an output which we call predicted value. Sometimes we refer forward propagation as inference. When we feed the input values to the neural network's first layer, it goes without any operations.
- Second layer takes values from first layer and applies multiplication, addition and activation operations and passes this value to the next layer. Same process repeats for subsequent layers and finally we get an output value from the last layer. We can see that it is called forward propagation as we are moving forward from the input layer to the output layer. Forward propagation is not sufficient for learning as after reaching output layer for the first time there will be no chance to start again with a more optimized parameters and here comes the idea of backpropagation to move backwards from the output layer to the input layer and so on till we reach a well trained model with suitable learning parameters (weights or θ 's).

Forward Propagation



Forward Propagation

Equation:

$$a_j^{(n+1)} = g(b^{(n)} + \sum_{i=1}^{N^{(n)}} \theta_i^{(n)} \cdot a_i^{(n)})$$

; $n \equiv$ layer index

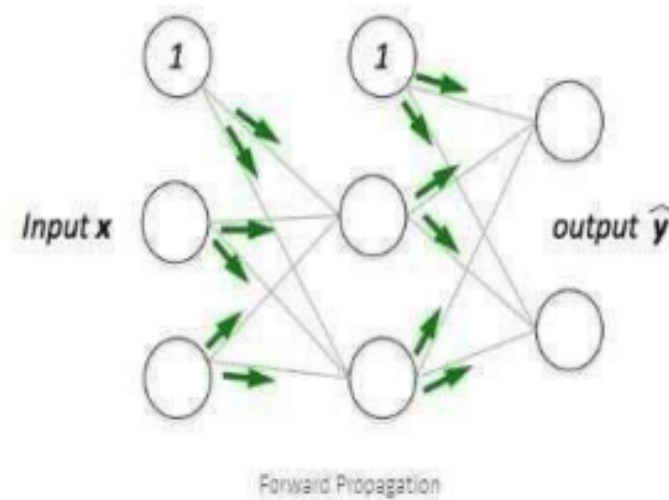
; $N^{(n)} \equiv$ Number of nodes in layer (n)

; $i \equiv$ node index in layer (n)

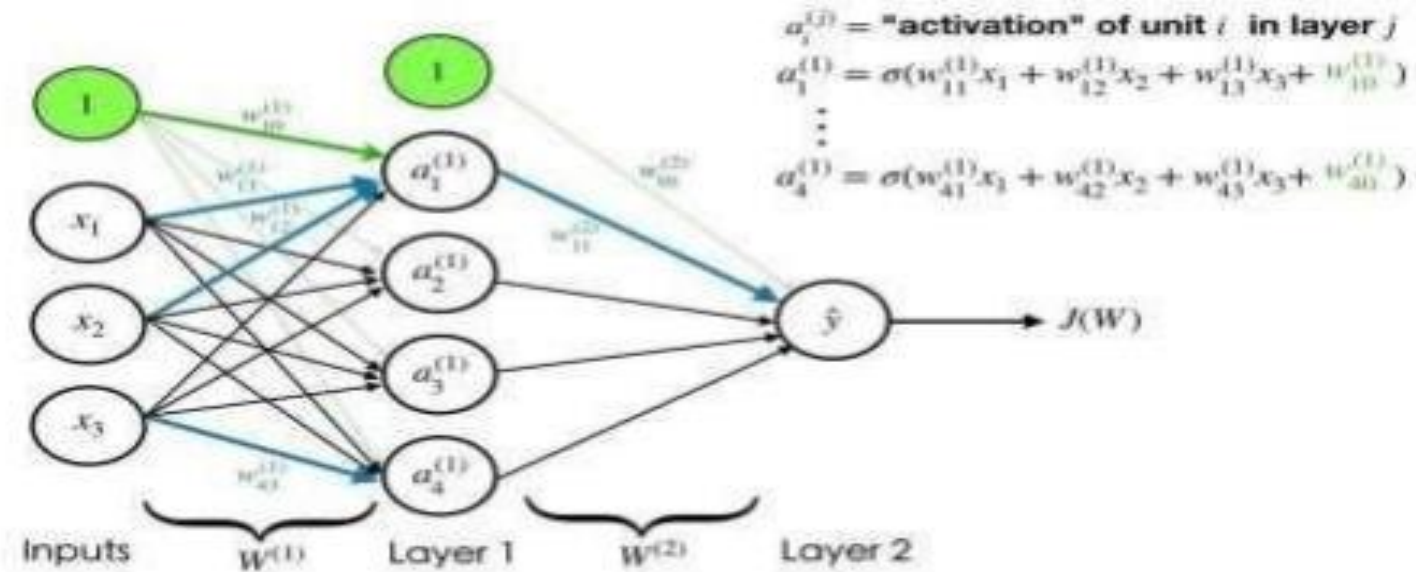
; $j \equiv$ node index in layer $(n + 1)$

; $g \equiv$ activation function

Note: w and θ are the same, they're both defining our trainable parameter

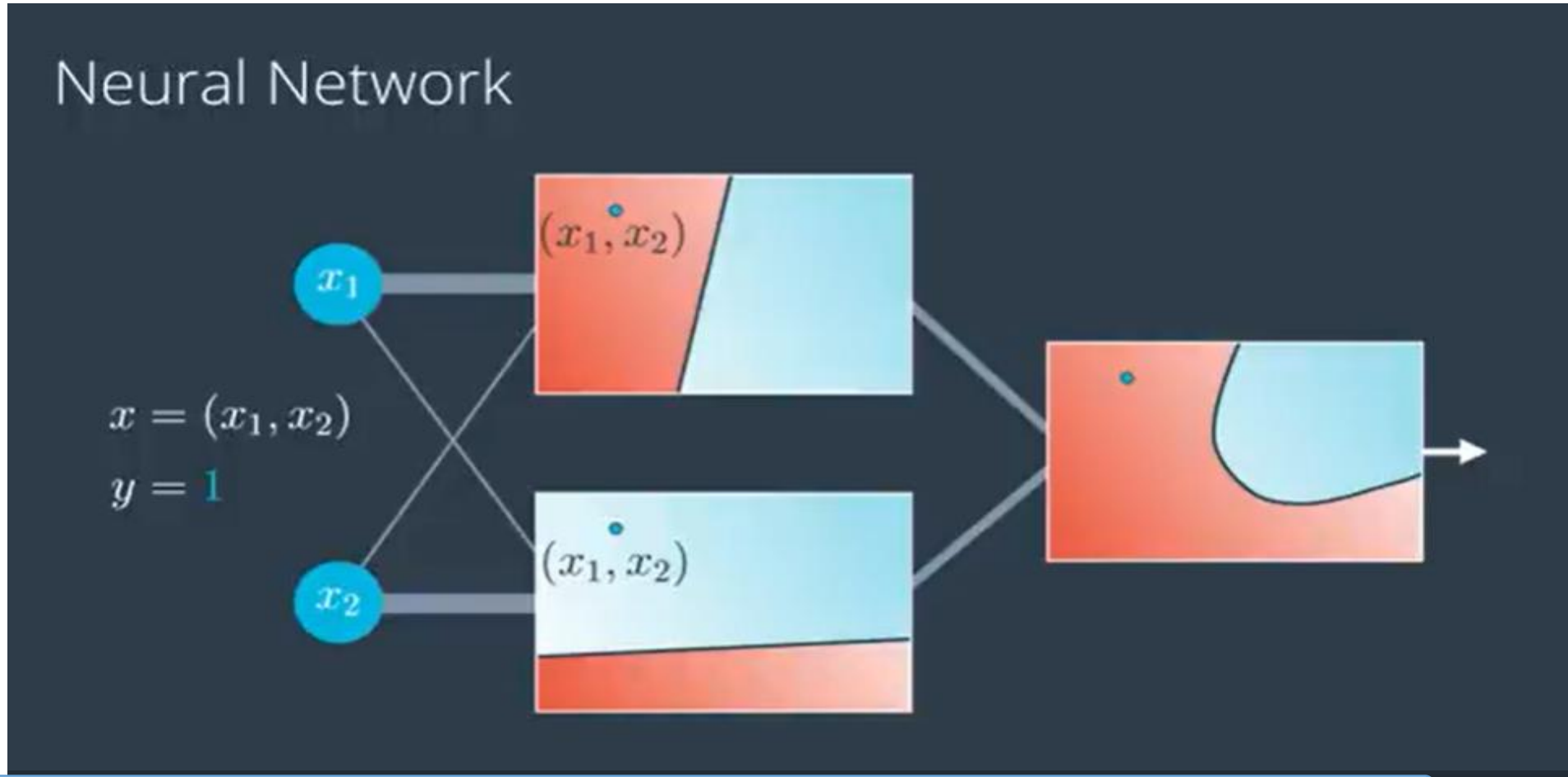


Forward Propagation

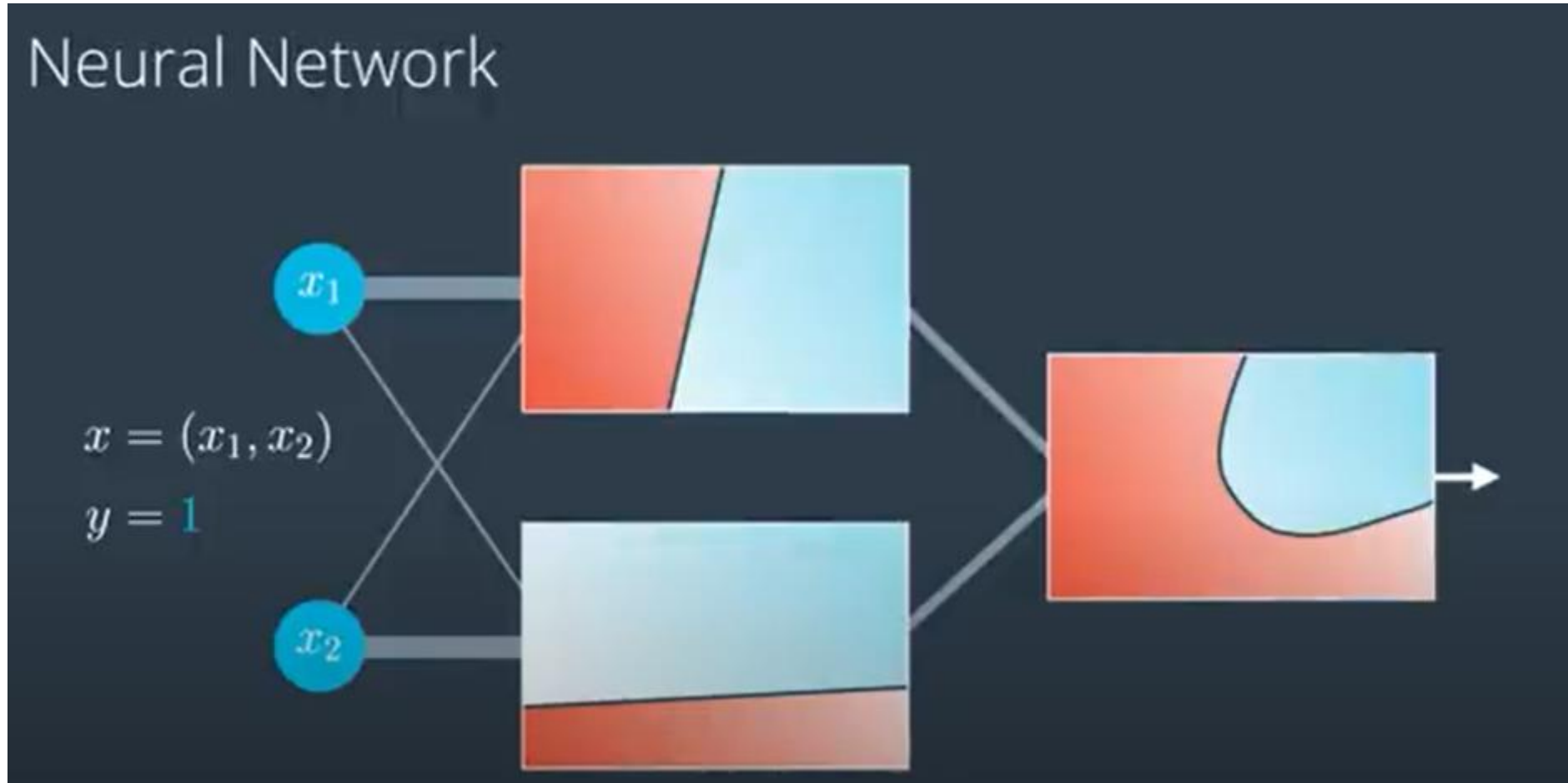


$$\hat{y} = a_1^{(2)} = \sigma(w_{11}^{(2)}a_1^{(1)} + w_{12}^{(2)}a_2^{(1)} + w_{13}^{(2)}a_3^{(1)} + w_{14}^{(2)}a_4^{(1)} + w_{10}^{(2)})$$

Feed forward

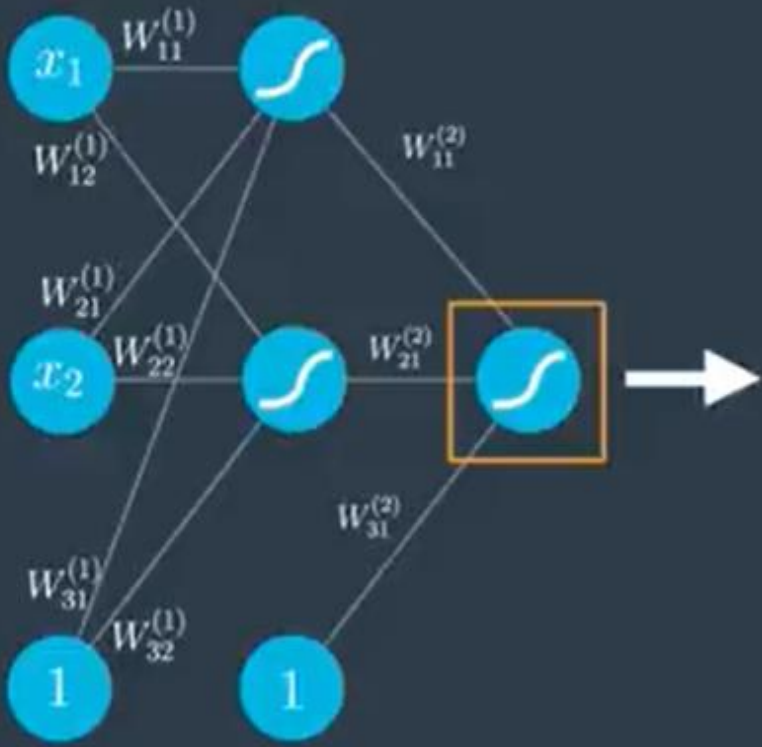


Deep Learning 1 : Artificial Neural Network



Deep Learning 1 : Artificial Neural Network

Feedforward

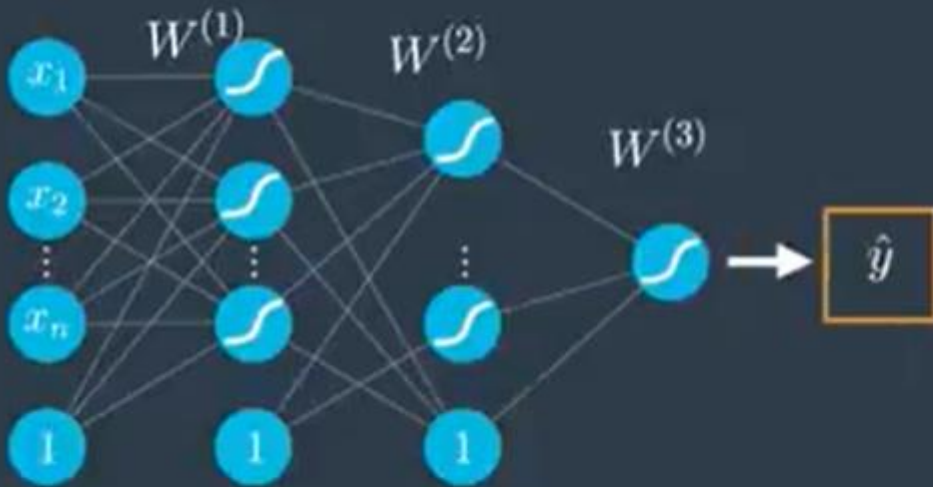


$$\hat{y} = \sigma \begin{pmatrix} W_{11}^{(2)} \\ W_{21}^{(2)} \\ W_{31}^{(2)} \end{pmatrix} \sigma \begin{pmatrix} W_{11}^{(1)} & W_{12}^{(1)} \\ W_{21}^{(1)} & W_{22}^{(1)} \\ W_{31}^{(1)} & W_{32}^{(1)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix}$$

$$\hat{y} = \sigma \circ W^{(2)} \circ \sigma \circ W^{(1)}(x)$$

Deep Learning 1 : Artificial Neural Network

Multi-layer Perceptron

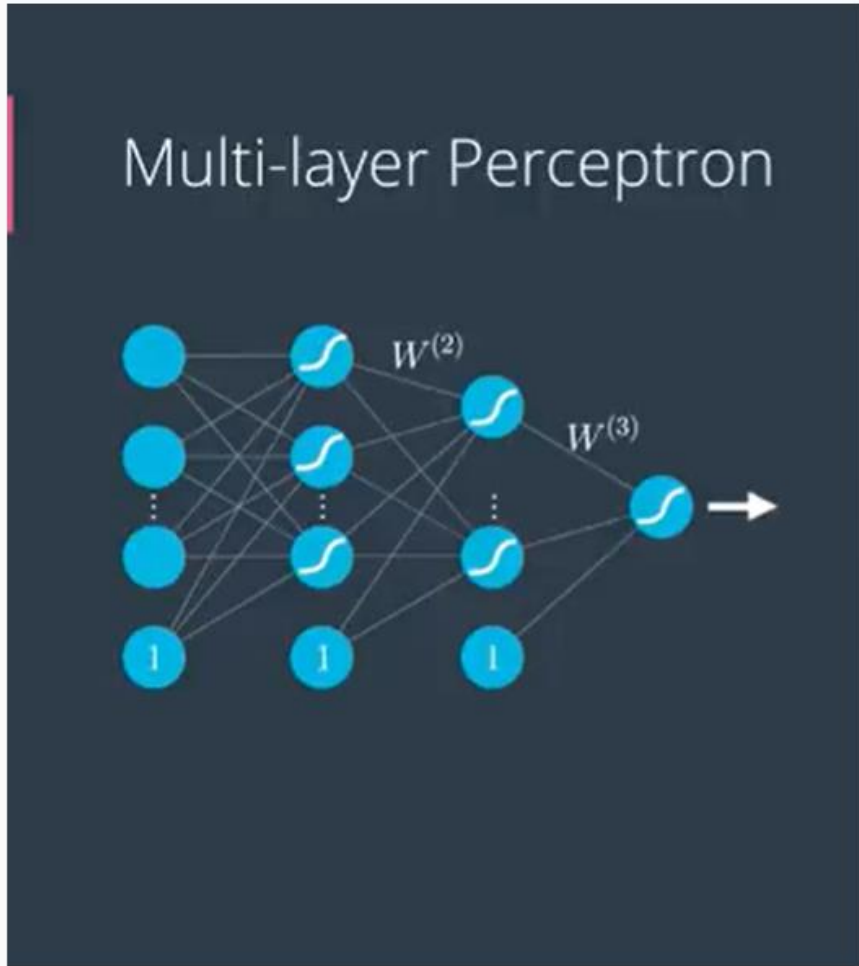


PREDICTION

$$\boxed{\hat{y}} = \sigma \circ W^{(3)} \circ \sigma \circ W^{(2)} \circ \sigma \circ W^{(1)}(x)$$

Deep Learning 1 : Artificial Neural Network

Error Function



PREDICTION

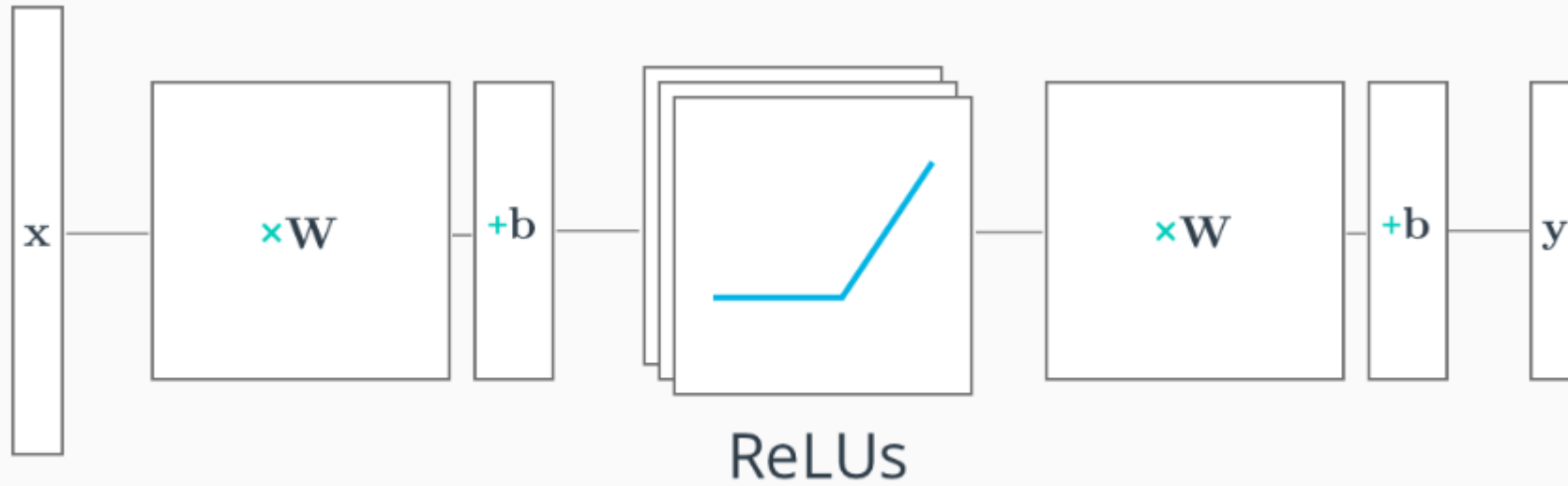
$$\hat{y} = \sigma \circ W^{(3)} \circ \sigma \circ W^{(2)} \circ \sigma \circ W^{(1)}(x)$$

ERROR FUNCTION

$$E(W) = -\frac{1}{m} \sum_{i=1}^m y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)$$



Deep Learning 1 : Artificial Neural Network



Backpropagation

Understanding How to Update ANNs Weights Step- by-Step

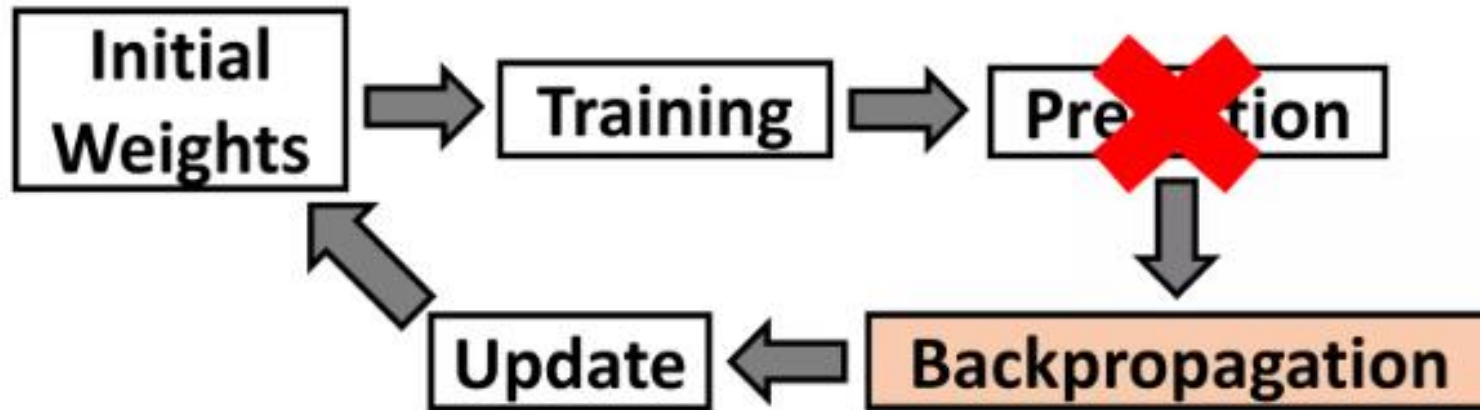
Train then Update

- The backpropagation algorithm is used to update the NN weights when they are not able to make the correct predictions. Hence, we should train the NN before applying backpropagation.



Train then Update

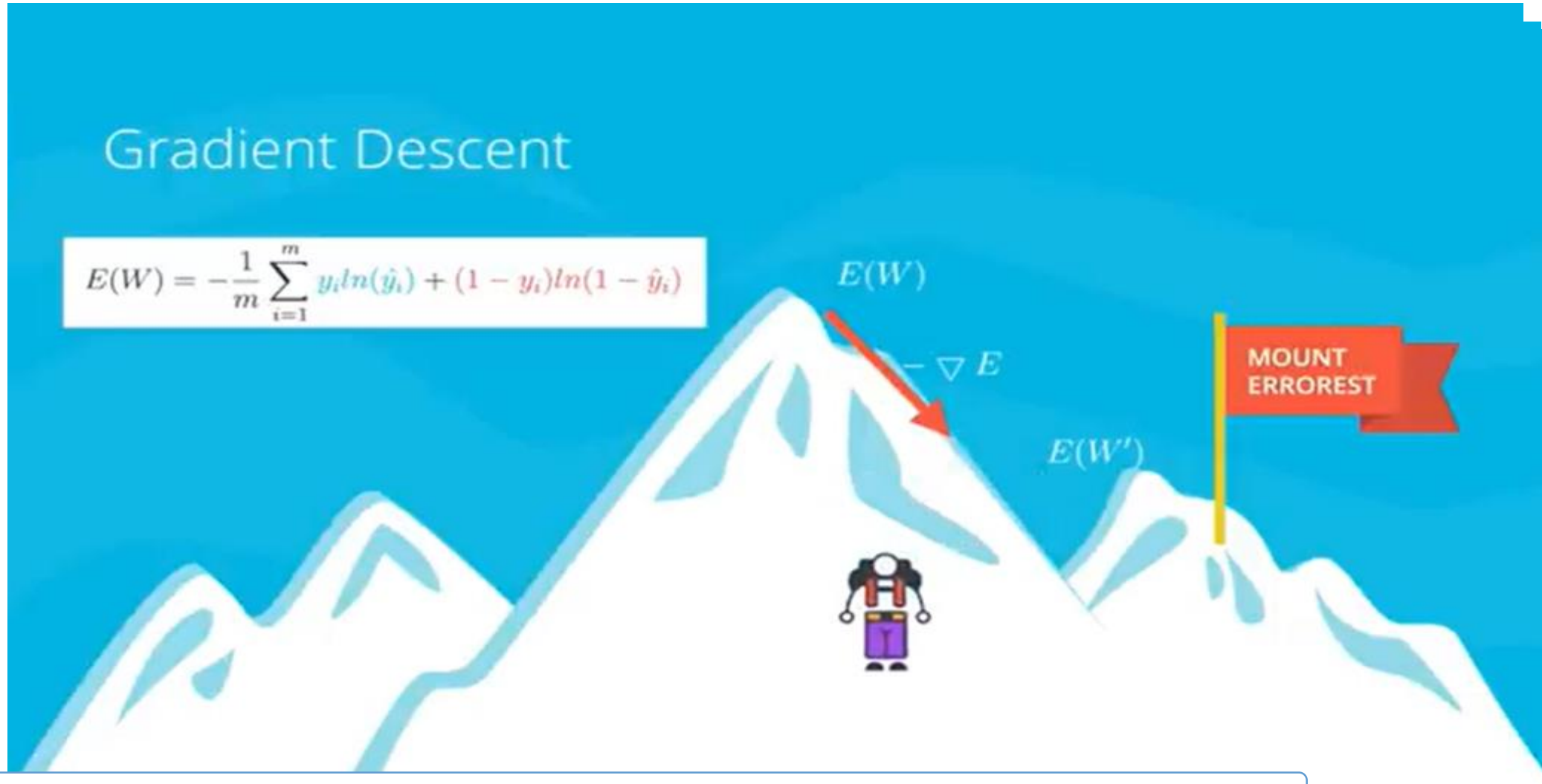
- The backpropagation algorithm is used to update the NN weights when they are not able to make the correct predictions. Hence, we should train the NN before applying backpropagation.



Deep Learning 1 : Artificial Neural Network

Gradient Descent

$$E(W) = -\frac{1}{m} \sum_{i=1}^m y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)$$



Deep Learning 1 : Artificial Neural Network

Gradient Descent: The Math

- CHAIN RULE REFRESHER

$$\frac{\partial}{\partial z} p(q(z)) = \frac{\partial p}{\partial q} \frac{\partial q}{\partial z}$$

$$q = (y - \hat{y}(w_i)) \quad p = \frac{1}{2} q(w_i)^2$$

$$\frac{\partial E}{\partial w_i} = (y - \hat{y}) \frac{\partial}{\partial w_i} (y - \hat{y})$$

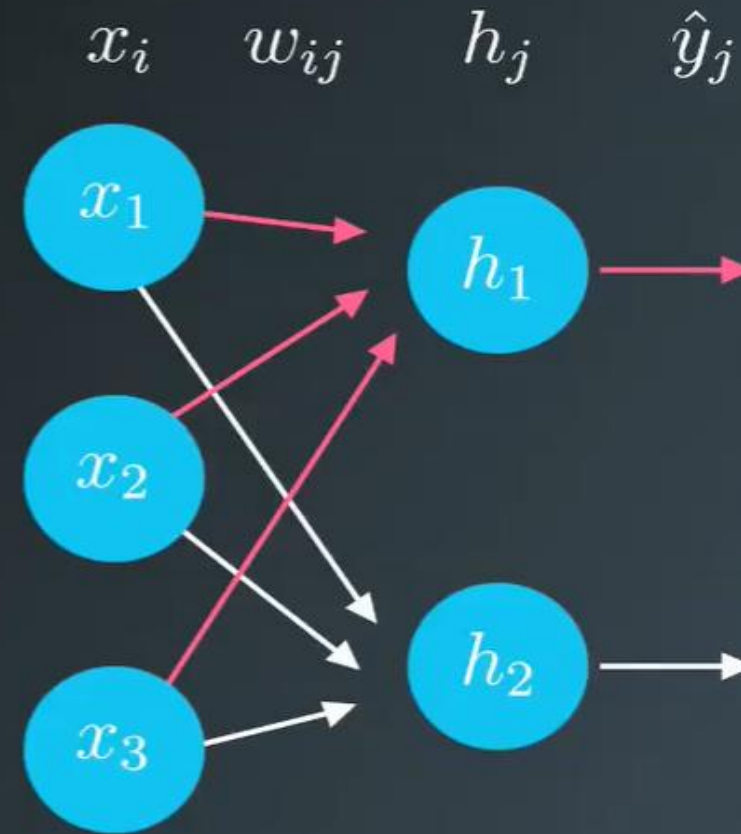
Deep Learning 1 : Artificial Neural Network

Gradient Descent: The Math

IF MULTIPLE OUTPUT UNITS:

$$\delta_j = (y_j - \hat{y}_j) f'(h_j)$$

$$\Delta w_{ij} = \eta \delta_j x_i$$

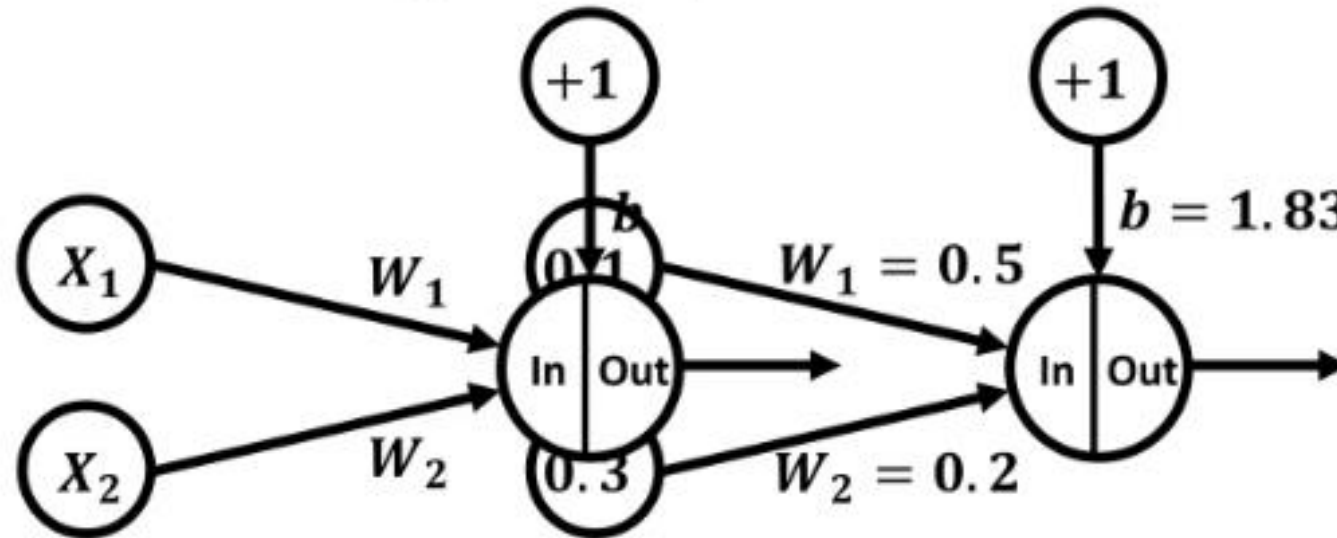


Back propagation

back propagation will consist of:

- Doing a feed forward operation.
- Comparing the output of the model with the desired output.
- Calculating the error.
- Running the feed forward operation backwards (back propagation) to spread the error to each of the weights.
- Use this to update the weights, and get a better model.
- Continue this until we have a model that is good.

Neural Network Training Example



Training Data

X_1	X_2	Output
0.1	0.3	0.03

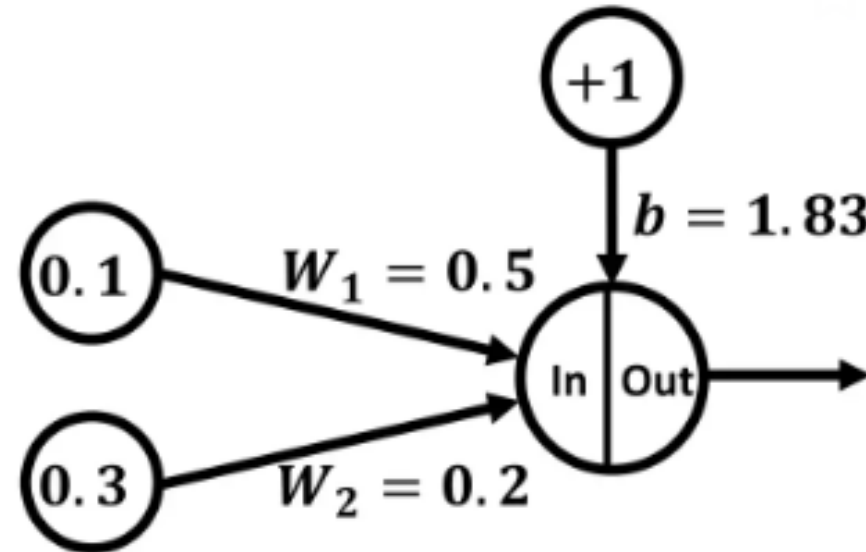
Ground truth Value

Initial Weights

W_1	W_2	b
0.5	0.5	1.83

Network Training

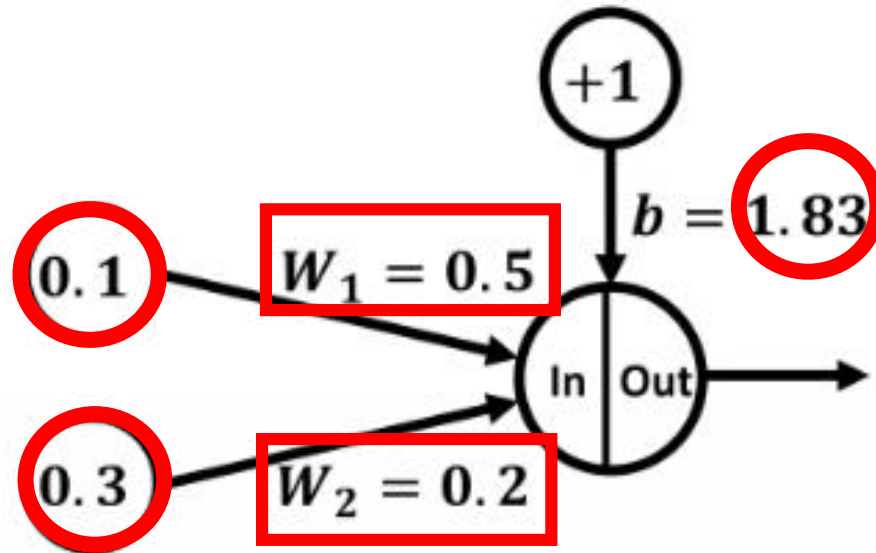
- Steps to train our network:
 1. Prepare activation function input (sum of products between inputs and weights).
 2. Activation function output.



Network Training: Sum of Products

$$s = X_1 * W_1 + X_2 * W_2 + b$$
$$s = 0.1 * 0.5 + 0.3 * 0.2 + 1.83$$
$$s = 1.94$$

- After calculating the sop between inputs and weights, next is to use this sop as the input to the activation function.



Network Training: Activation Function

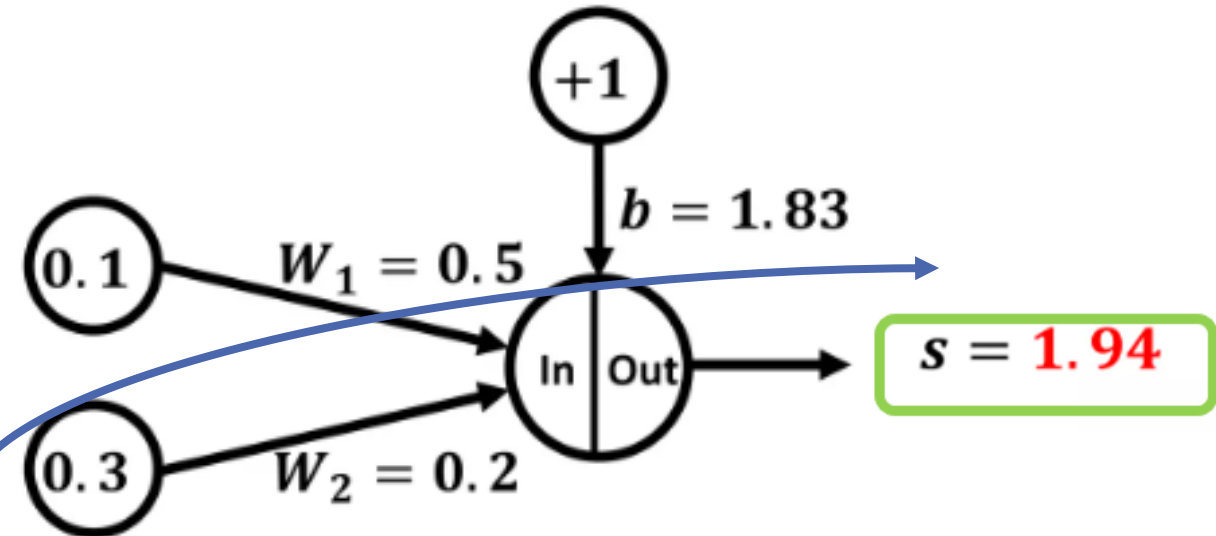
- In this example, the sigmoid activation function is used.

$$f(s) = \frac{1}{1 + e^{-s}}$$

- Based on the sop calculated previously, the output is as follows:

$$f(s) = \frac{1}{1 + e^{-1.94}} = \frac{1}{1 + 0.144} = \frac{1}{1.144}$$

$$f(s) = \mathbf{0.874}$$



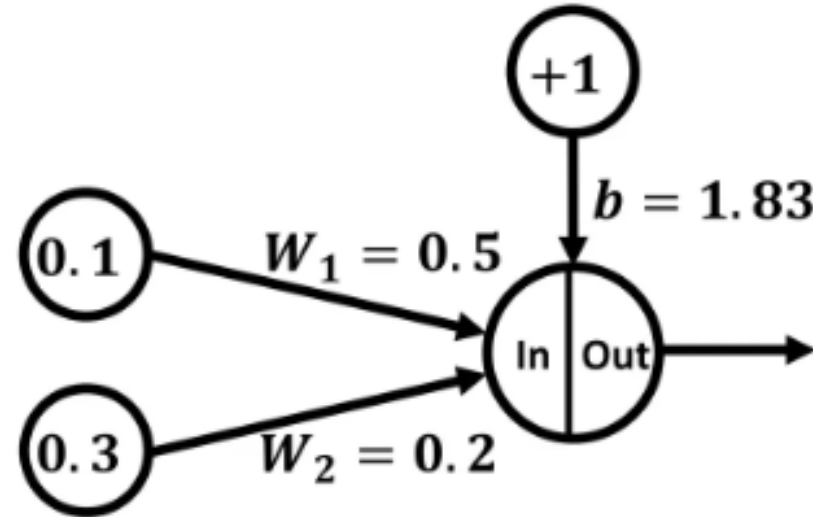
Network Training: Prediction Error

- After getting the predicted outputs, next is to measure the **prediction error** of the network.
- We can use the squared error function defined as follows:

$$E = \frac{1}{2} (\textit{desired} - \textit{predicted})^2$$

- Based on the predicted output, the prediction error is:

$$E = \frac{1}{2} (0.03 - 0.874)^2 = \frac{1}{2} (-0.844)^2 = \frac{1}{2} (0.713) = \mathbf{0.357}$$



This very away from Zero

How to Minimize Prediction Error?

- There is a prediction error and it should be minimized until reaching an acceptable error.

What should we do in order to minimize the error?

- There must be something to change in order to minimize the error. In our example, the only parameter to change is the weight.

How to update the weights?

- We can use the weights update equation:

$$W_{new} = W_{old} + \eta[d - Y]X$$

Weights Update Equation

- We can use the weights update equation:

$$W_{new} = W_{old} + \eta[d - Y]X$$

- W_{new} : new updated weights.

- W_{old} : current weights. [1.83, 0.5, 0.2]

Initial Weights		
W_1	W_2	b
0.5	0.5	1.83

- η : network learning rate 0.01

- d : desired output 0.03

- Y : predicted output 0.874

- X : current input at which the network made false prediction. [+1, 0.1, 0.3]

Training Data		
X_1	X_2	Output
0.1	0.3	0.03

Weights Update Equation

$$W_{new} = W_{old} + \eta[d - Y]X$$

$$= [1.83, 0.5, 0.2] + 0.01[0.03 - 0.874][+1, 0.1, 0.3]$$

$$= [1.83, 0.5, 0.2] + -0.0084[+1, 0.1, 0.3]$$

$$= [1.83, 0.5, 0.2] + [-0.0084, -0.00084, -0.0025]$$

$$= [1.822, 0.499, 0.198]$$

b

w1

w2



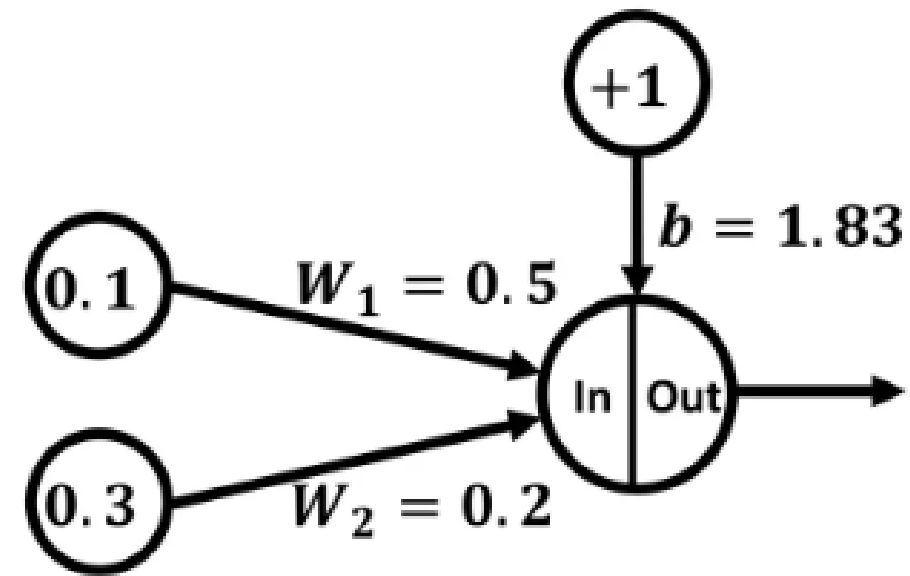
$$= [1.822, 0.499, 0.198]$$

Weights Update Equation

- The new weights are:

W_{1new}	W_{2new}	b_{new}
0.198	0.499	1.822

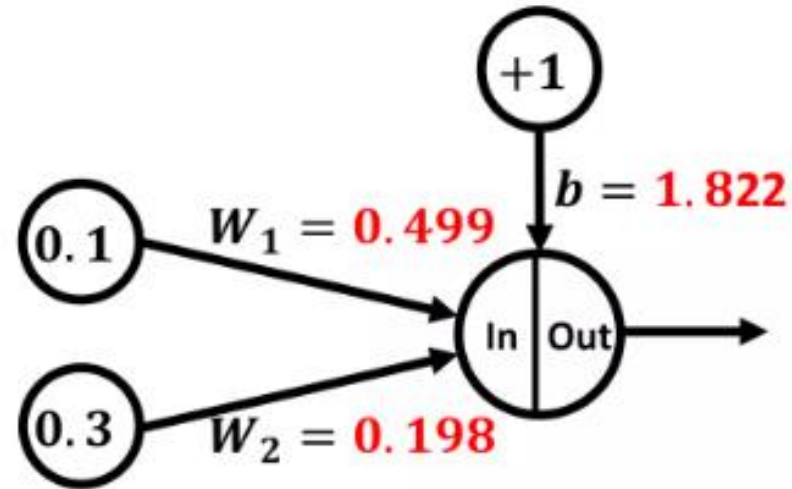
- Based on the new weights, the network will be re-trained.



Weights Update Equation

- The new weights are:

W_{1new}	W_{2new}	b_{new}
0.198	0.499	1.822



- Based on the new weights, the network will be re-trained.
- Continue these operations until prediction error reaches an acceptable value.
 1. Updating weights.
 2. Retraining network.
 3. Calculating prediction error.

Why Backpropagation Algorithm is Important?

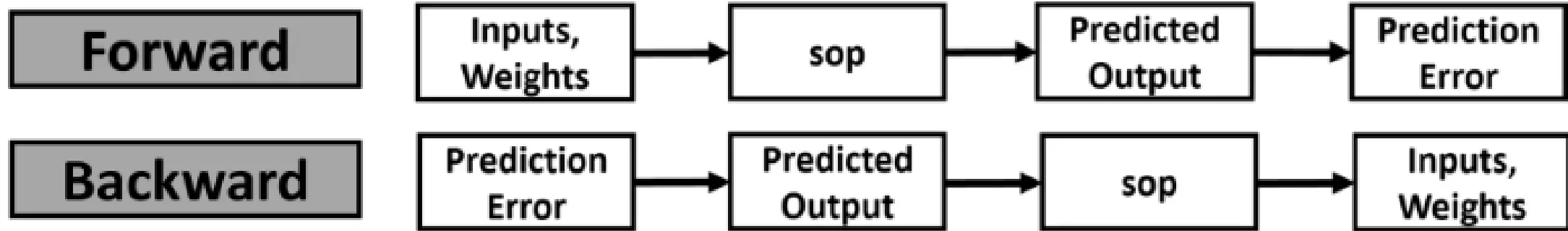
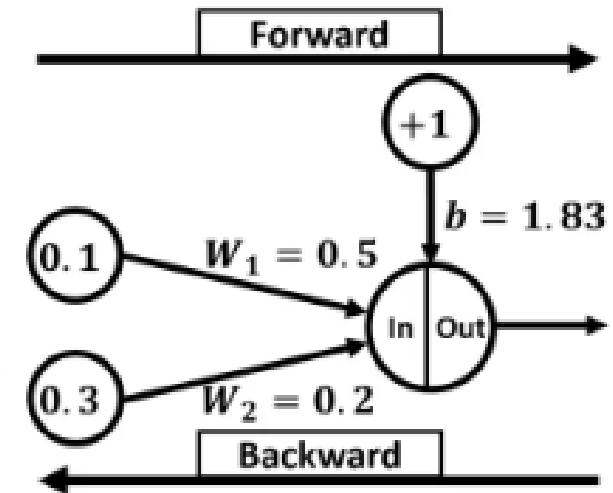


- The **backpropagation algorithm** is used to answer these questions and understand **effect** of each weight over the prediction error.

Backpropagation Algorithm Answers these Questions

Forward Vs. Backward Passes

- When training a neural network, there are two passes: forward and backward.



- The goal of the backward pass is to know how each weight affects the total error. In other words, how changing the weights changes the prediction error?

Backward Pass

- Let us work with a simpler example:

$$Y = X^2Z + H$$

- How to answer this question: **What is the effect on the output Y given a change in variable X?**
- This question is answered using derivatives. Derivative of Y wrt X ($\frac{\partial Y}{\partial X}$) will tell us the effect of changing the variable X over the output Y.

Calculating Derivatives

$$Y = X^2Z + H$$

- The derivative $\frac{\partial Y}{\partial X}$ can be calculated as follows:

$$\frac{\partial Y}{\partial X} = \frac{\partial}{\partial X} (X^2Z + H)$$

- Based on these two derivative rules:

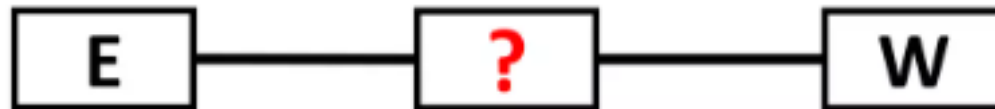
$$\text{Square} \quad \frac{\partial}{\partial X} (X^2) = 2X$$

$$\text{Constant} \quad \frac{\partial}{\partial X} (C) = 0$$

- The result will be:

$$\frac{\partial Y}{\partial X} = 2XZ + 0 = 2XZ$$

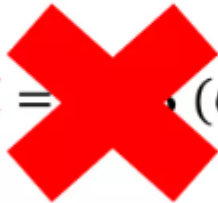
Prediction Error – Weight Derivative



$$E = \frac{1}{2} (\textit{desired} - \textit{predicted})^2$$

Prediction Error – Weight Derivative

$$E = \frac{1}{2}(\textit{desired} - \textit{predicted})^2$$

desired =  (Constant)

Prediction Error – Weight Derivative

$$E = \frac{1}{2} (\textit{desired} - \textit{predicted})^2$$


$$\textit{desired} = \text{Constant} \quad \textit{Predicted} = f(s) = \frac{1}{1 + e^{-s}}$$

Prediction Error – Weight Derivative

$$E = \frac{1}{2} (\textit{desired} - \textit{predicted})^2$$

$$\textit{desired} = 0.03 \text{ (Constant)} \quad \textit{Predicted} = f(s) = \frac{1}{1 + e^{-s}}$$

$$E = \frac{1}{2} \left(\textit{desired} - \frac{1}{1 + e^{-s}} \right)^2$$

Prediction Error – Weight Derivative

$$E = \frac{1}{2} (\textit{desired} - \textit{predicted})^2$$

$$\textit{desired} = 0.03 \text{ (Constant)} \quad \textit{Predicted} = f(s) = \frac{1}{1 + e^{-s}}$$

$$E = \frac{1}{2} \left(\textit{desired} - \frac{1}{1 + e^{-s}} \right)^2$$

Prediction Error – Weight Derivative

$$E = \frac{1}{2} (\textit{desired} - \textit{predicted})^2$$

$$\textit{desired} = 0.03 \text{ (Constant)} \quad \textit{Predicted} = f(s) = \frac{1}{1 + e^{-s}}$$

$$E = \frac{1}{2} \left(\textit{desired} - \frac{1}{1 + e^{-s}} \right)^2$$

$$\textcolor{red}{s} = X_1 * W_1 + X_2 * W_2 + b$$

Prediction Error – Weight Derivative

$$E = \frac{1}{2} (desired - predicted)^2$$

$$desired = 0.03 \text{ (Constant)} \quad Predicted = f(s) = \frac{1}{1 + e^{-s}}$$

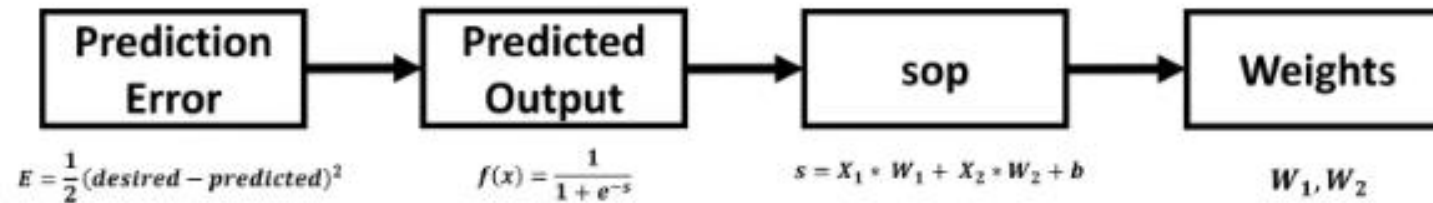
$$E = \frac{1}{2} \left(desired - \frac{1}{1 + e^{-s}} \right)^2$$

$$s = X_1 * W_1 + X_2 * W_2 + b$$

Complex Derivative

$$E = \frac{1}{2} \left(desired - \frac{1}{1 + e^{-(X_1 * W_1 + X_2 * W_2 + b)}} \right)^2$$

Multivariate Chain Rule

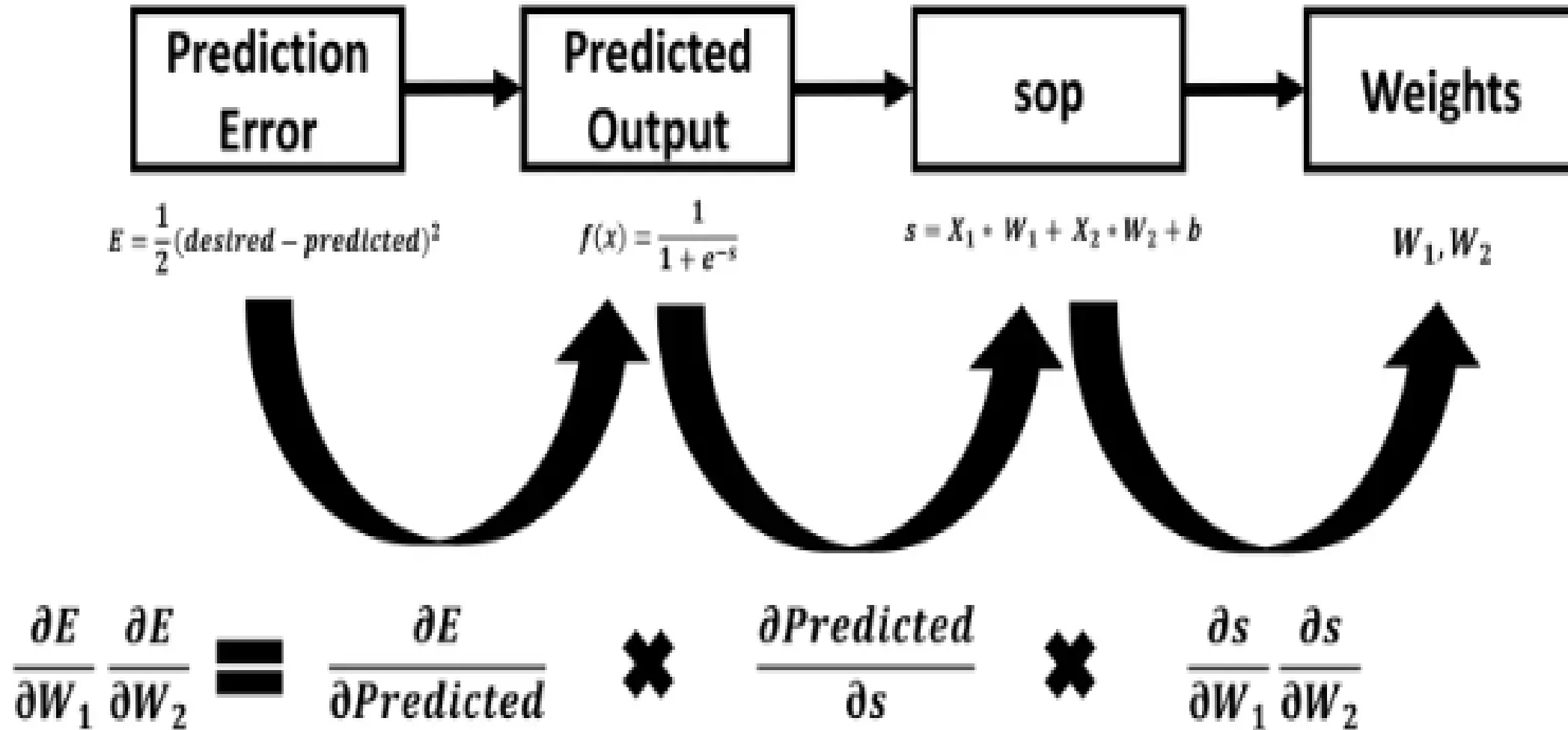


Chain Rule

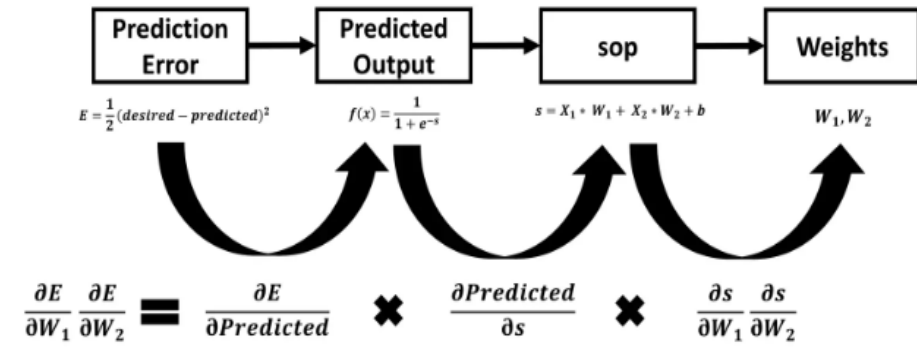
$$E = \frac{1}{2} \left(\text{desired} - \frac{1}{1 + e^{-(X_1 * W_1 + X_2 * W_2 + b)}} \right)^2$$

$$\frac{\partial E}{\partial W} = \frac{\partial}{\partial W} \left(\frac{1}{2} \left(\text{desired} - \frac{1}{1 + e^{-(X_1 * W_1 + X_2 * W_2 + b)}} \right)^2 \right)$$

Multivariate Chain Rule



Multivariate Chain Rule



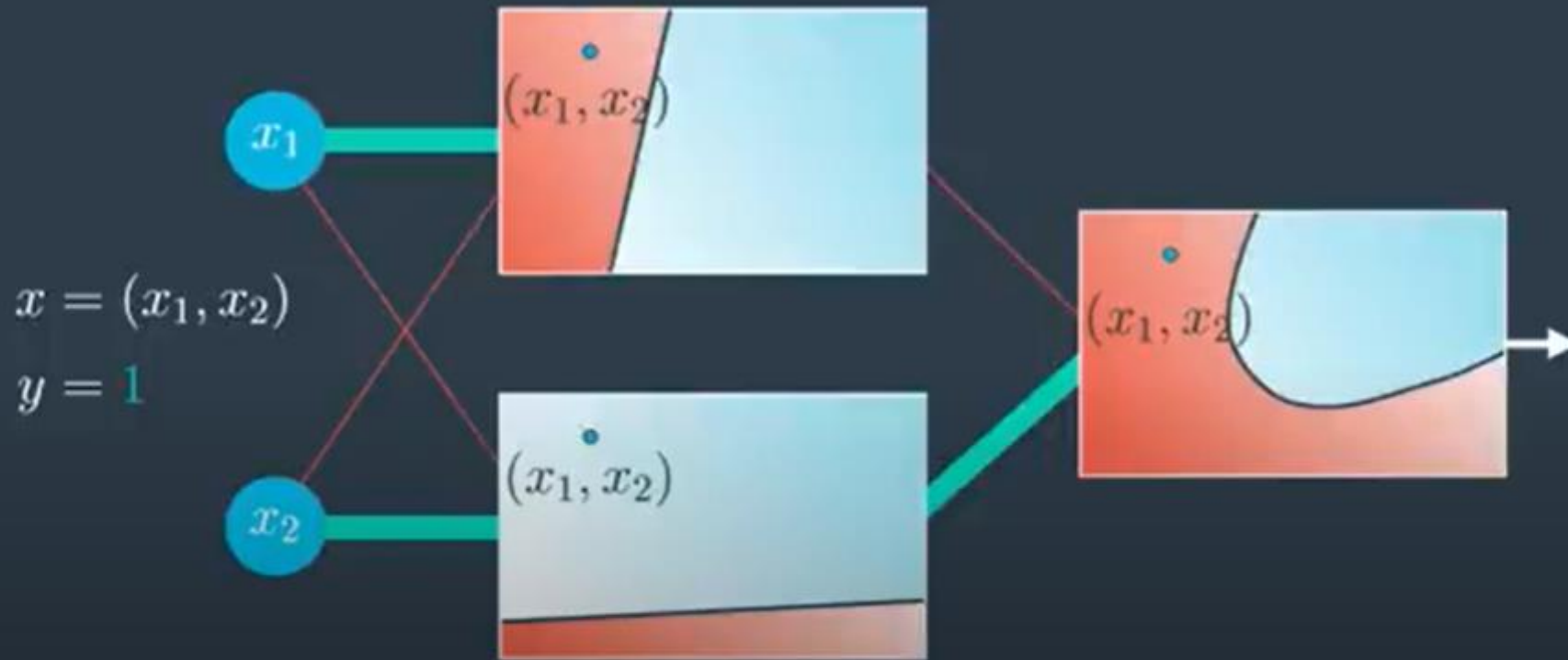
$$\frac{\partial E}{\partial W_1} = \frac{\partial E}{\partial \text{Predicted}} * \frac{\partial \text{Predicted}}{\partial s} * \frac{\partial s}{\partial W_1}$$

$$\frac{\partial E}{\partial W_2} = \frac{\partial E}{\partial \text{Predicted}} * \frac{\partial \text{Predicted}}{\partial s} * \frac{\partial s}{\partial W_2}$$

Let's calculate these individual partial derivatives.

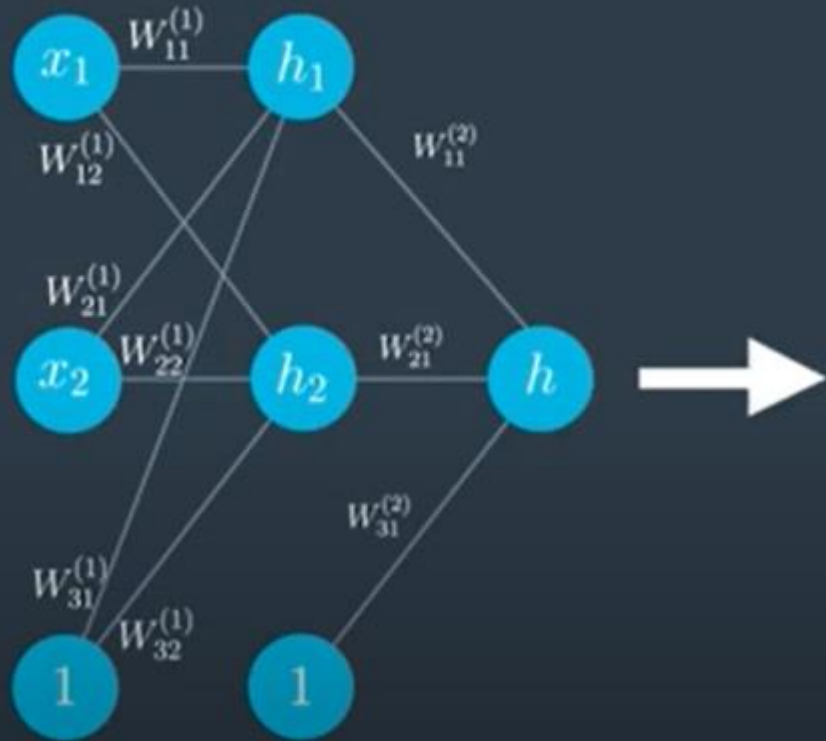
Deep Learning 1 : Artificial Neural Network

Backpropagation



Deep Learning 1 : Artificial Neural Network

Feedforward



$$E(W) = -\frac{1}{m} \sum_{i=1}^m y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)$$

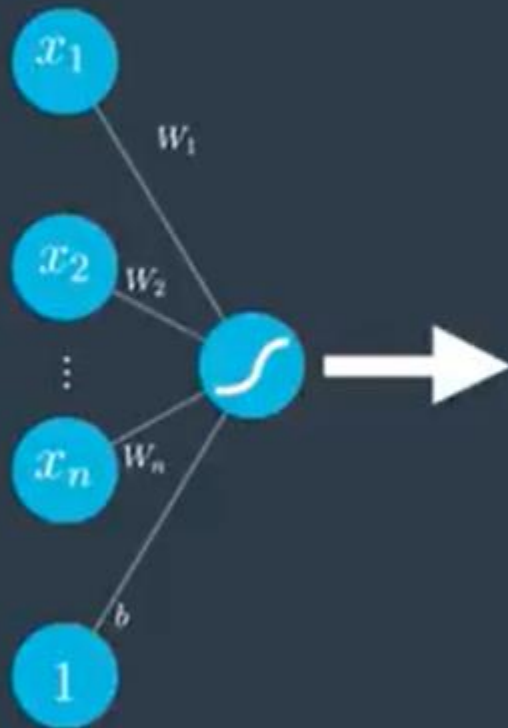
$$E(W) = E(W_{11}^{(1)}, W_{12}^{(1)}, \dots, W_{31}^{(2)})$$

$$\nabla E = \left(\frac{\partial E}{\partial W_{11}^{(1)}}, \dots, \frac{\partial E}{\partial W_{31}^{(2)}} \right)$$

$$\frac{\partial E}{\partial W_{11}^{(1)}} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial h_1} \frac{\partial h_1}{\partial W_{11}^{(1)}}$$

Deep Learning 1 : Artificial Neural Network

Perceptron



PREDICTION

Apply the network

$$\hat{y} = \sigma W^{(3)} \circ \sigma W^{(2)} \circ \sigma \circ W^{(1)}(x)$$

ERROR FUNCTION

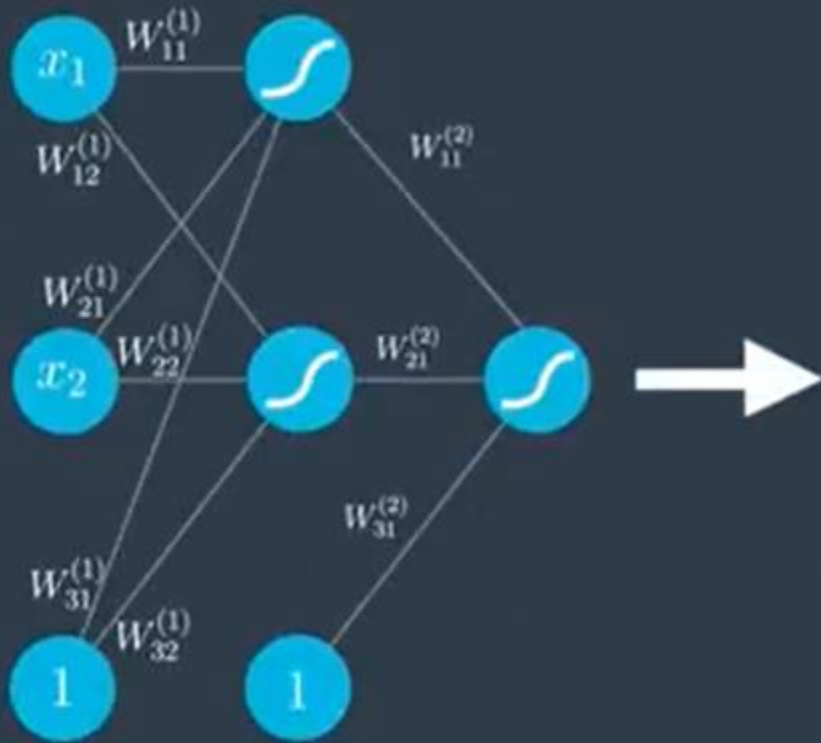
$$E(W) = -\frac{1}{m} \sum_{i=1}^m y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)$$

GRADIENT OF THE ERROR FUNCTION

$$\nabla E = \left(\frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n}, \frac{\partial E}{\partial b} \right)$$

Deep Learning 1 : Artificial Neural Network

Backpropagation



$$\hat{y} = \sigma W^{(2)} \circ \sigma \circ W^{(1)}(x)$$

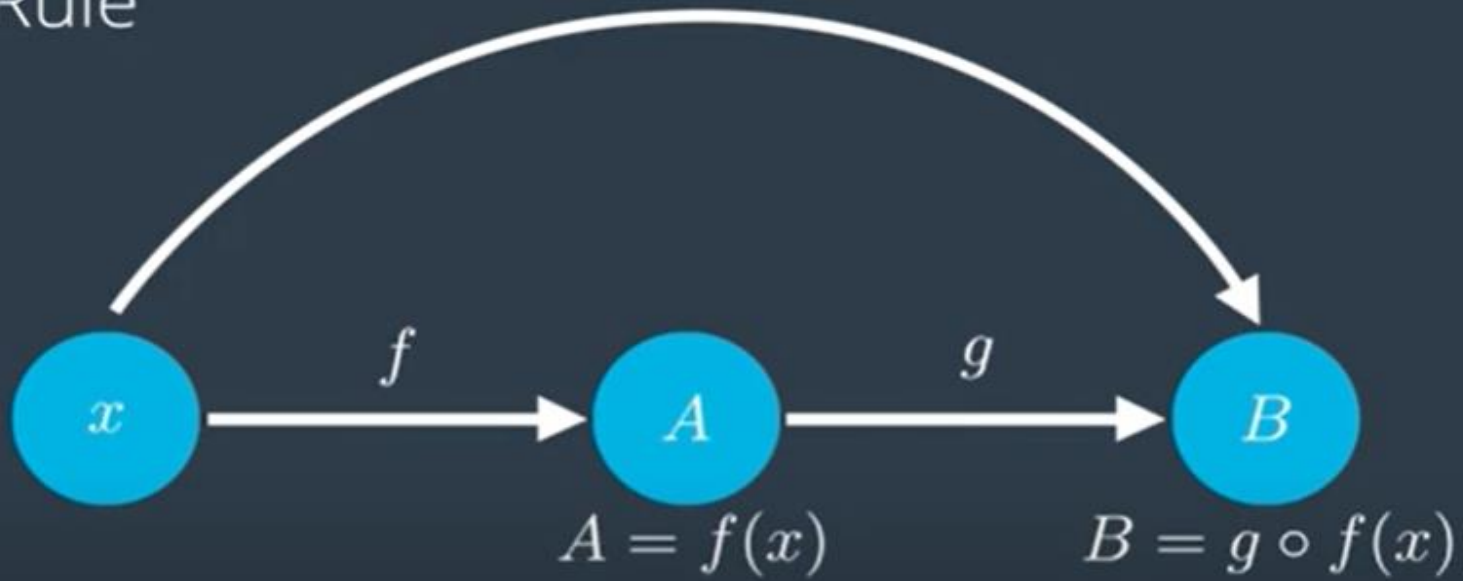
$$W^{(1)} = \begin{pmatrix} W_{11}^{(1)} & W_{12}^{(1)} \\ W_{21}^{(1)} & W_{22}^{(1)} \\ W_{31}^{(1)} & W_{32}^{(1)} \end{pmatrix} \quad W^{(2)} = \begin{pmatrix} W_{11}^{(2)} \\ W_{21}^{(2)} \\ W_{31}^{(2)} \end{pmatrix}$$

$$\nabla E = \begin{pmatrix} \frac{\partial E}{\partial W_{11}^{(1)}} & \frac{\partial E}{\partial W_{12}^{(1)}} & \frac{\partial E}{\partial W_{11}^{(2)}} \\ \frac{\partial E}{\partial W_{21}^{(1)}} & \frac{\partial E}{\partial W_{22}^{(1)}} & \frac{\partial E}{\partial W_{21}^{(2)}} \\ \frac{\partial E}{\partial W_{31}^{(1)}} & \frac{\partial E}{\partial W_{32}^{(1)}} & \frac{\partial E}{\partial W_{31}^{(2)}} \end{pmatrix}$$

$$W_{ij}'^{(k)} \leftarrow W_{ij}^{(k)} - \alpha \frac{\partial E}{\partial W_{ij}^{(k)}}$$

Deep Learning 1 : Artificial Neural Network

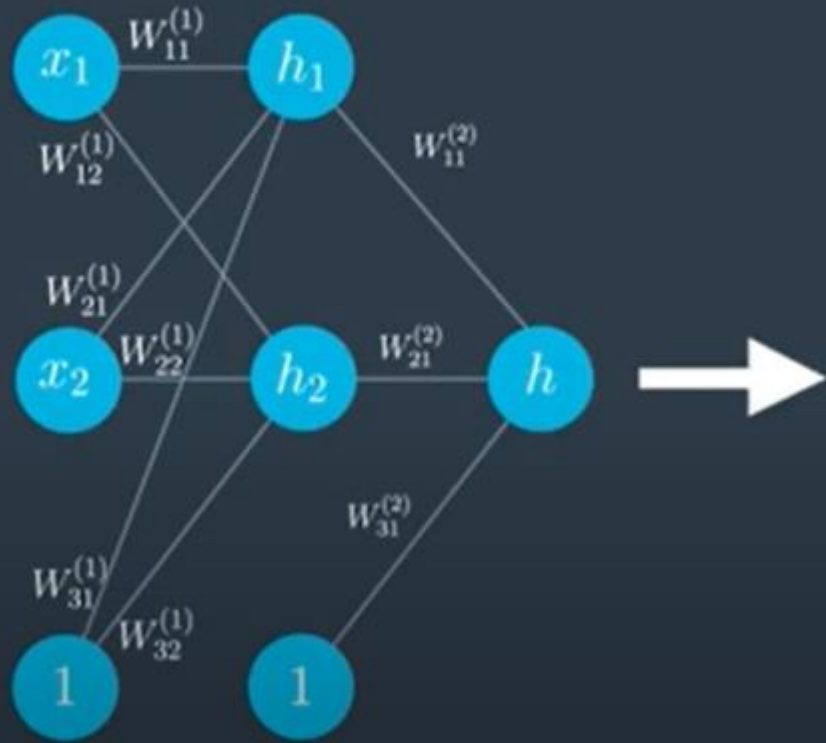
Chain Rule



$$\frac{\partial B}{\partial x} = \frac{\partial B}{\partial A} \frac{\partial A}{\partial x}$$

Deep Learning 1 : Artificial Neural Network

Feedforward



$$E(W) = -\frac{1}{m} \sum_{i=1}^m y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)$$

$$E(W) = E(W_{11}^{(1)}, W_{12}^{(1)}, \dots, W_{31}^{(2)})$$

$$\nabla E = \left(\frac{\partial E}{\partial W_{11}^{(1)}}, \dots, \frac{\partial E}{\partial W_{31}^{(2)}} \right)$$

$$\frac{\partial E}{\partial W_{11}^{(1)}} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial h_1} \frac{\partial h_1}{\partial W_{11}^{(1)}}$$

Error-Predicted $\left(\frac{\partial E}{\partial P_{\text{Predicted}}}\right)$ Partial Derivative

$$E = \frac{1}{2}(\text{desired} - \text{predicted})^2$$

$$\begin{aligned}\frac{\partial E}{\partial P_{\text{Predicted}}} &= \frac{\partial}{\partial P_{\text{Predicted}}} \left(\frac{1}{2}(\text{desired} - \text{predicted})^2 \right) \\ &= 2 * \frac{1}{2}(\text{desired} - \text{predicted})^{2-1} * (0 - 1) \\ &= (\text{desired} - \text{predicted}) * (-1) \\ &= \text{predicted} - \text{desired}\end{aligned}$$

Substitution

$$\frac{\partial E}{\partial P_{\text{Predicted}}} = \text{predicted} - \text{desired} = 0.874 - 0.03$$

$$\frac{\partial E}{\partial P_{\text{Predicted}}} = 0.844$$

Calculation of the derivative of the sigmoid function

Recall that the sigmoid function has a beautiful derivative, which we can see in the following calculation. This will make our backpropagation step much cleaner.

$$\begin{aligned}\sigma'(x) &= \frac{\partial}{\partial x} \frac{1}{1+e^{-x}} \\ &= \frac{e^{-x}}{(1+e^{-x})^2} \\ &= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} \\ &= \sigma(x)(1 - \sigma(x))\end{aligned}$$

Predicted-sop ($\frac{\partial Predicted}{\partial s}$) Partial Derivative

$$Predicted = \frac{1}{1 + e^{-s}}$$

$$\frac{\partial Predicted}{\partial s} = \frac{\partial}{\partial s} \left(\frac{1}{1 + e^{-s}} \right)$$

$$\frac{\partial Predicted}{\partial s} = \frac{1}{1 + e^{-s}} \left(1 - \frac{1}{1 + e^{-s}} \right)$$

Substitution

$$\frac{\partial Predicted}{\partial s} = \frac{1}{1 + e^{-s}} \left(1 - \frac{1}{1 + e^{-s}} \right) = \frac{1}{1 + e^{-1.94}} \left(1 - \frac{1}{1 + e^{-1.94}} \right)$$

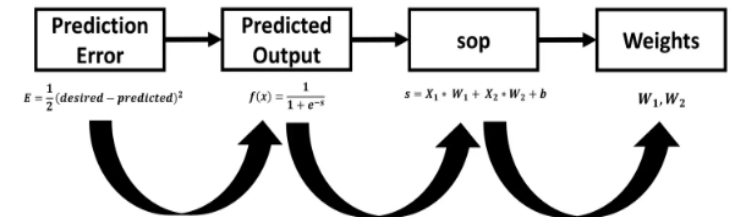
$$= \frac{1}{1 + 0.144} \left(1 - \frac{1}{1 + 0.144} \right)$$

$$= \frac{1}{1.144} \left(1 - \frac{1}{1.144} \right)$$

$$= \sigma(x)(1 - \sigma(x)) = 0.874(1 - 0.874)$$

$$= 0.874(0.126)$$

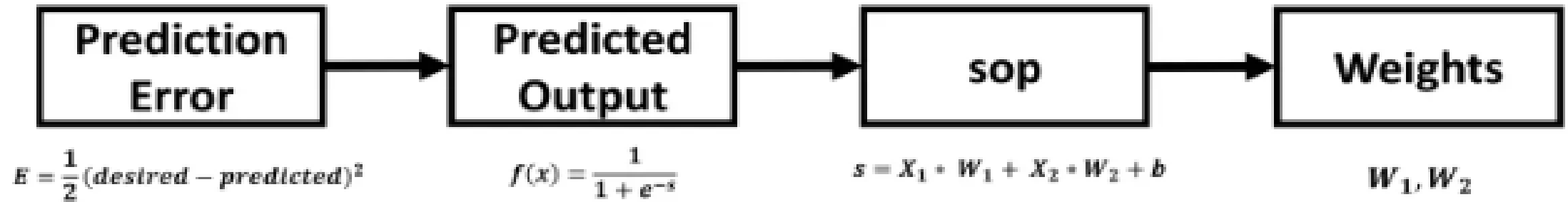
$$\frac{\partial Predicted}{\partial s} = 0.11$$



$$\frac{\partial E}{\partial W_1} \frac{\partial E}{\partial W_2} = \frac{\partial E}{\partial Predicted} \times \frac{\partial Predicted}{\partial s} \times \frac{\partial s}{\partial W_1} \frac{\partial s}{\partial W_2}$$

Deep Learning : Artificial Neural Network

Sop- W_1 ($\frac{\partial s}{\partial W_1}$) Partial Derivative

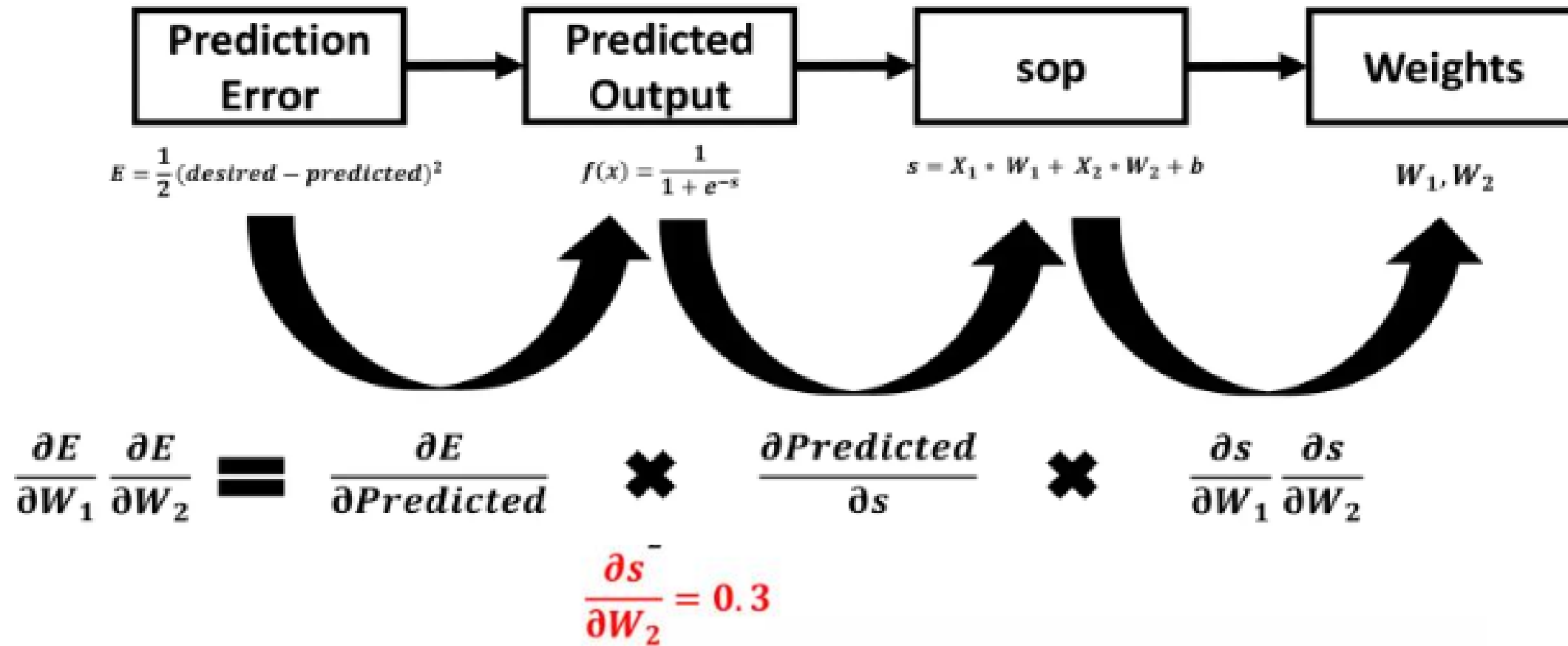


$$\frac{\partial E}{\partial W_1} \frac{\partial E}{\partial W_2} = \frac{\partial E}{\partial \text{Predicted}} \times \frac{\partial \text{Predicted}}{\partial s} \times \frac{\partial s}{\partial W_1} \frac{\partial s}{\partial W_2}$$

$$\frac{\partial s}{\partial W_1} = 0.1$$

Deep Learning : Artificial Neural Network

Sop- W_1 ($\frac{\partial s}{\partial W_2}$) Partial Derivative



Error- W_1 ($\frac{\partial E}{\partial W_1}$) Partial Derivative

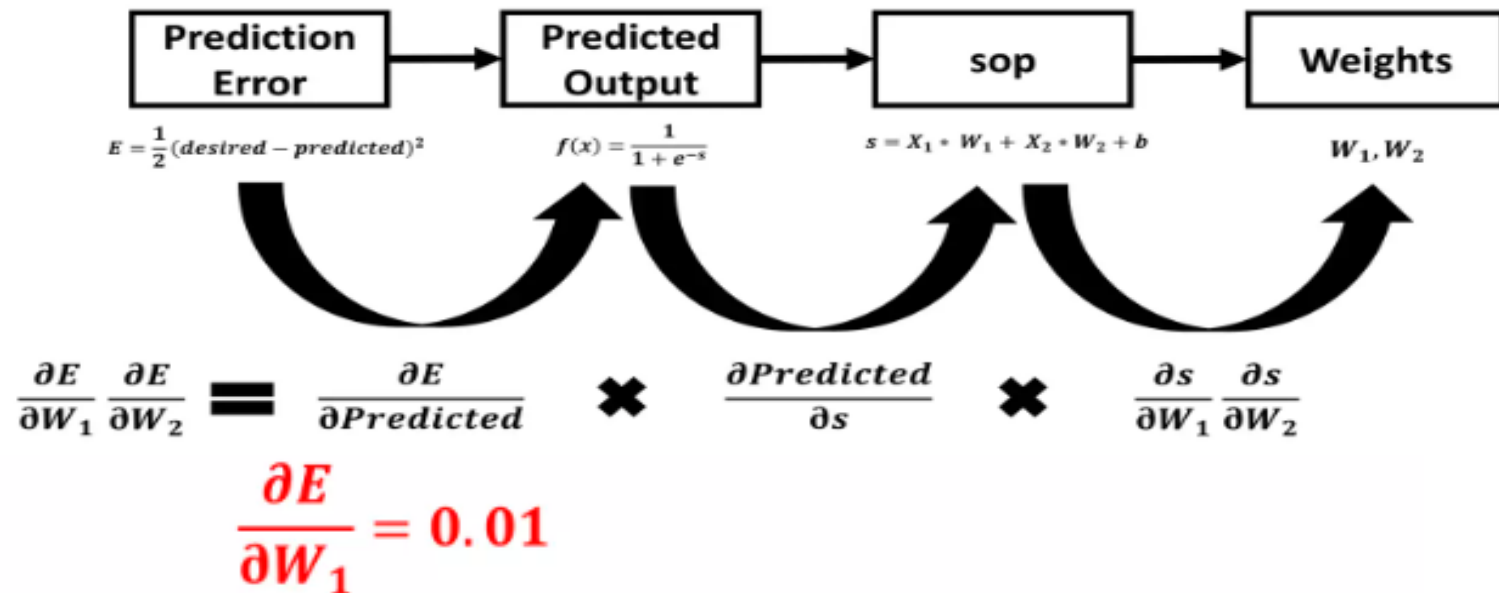
- After calculating each individual derivative, we can multiply all of them to get the desired relationship between the prediction error and each weight.

Calculated Derivatives

$$\frac{\partial E}{\partial \text{Predicted}} = 0.844$$

$$\frac{\partial \text{Predicted}}{\partial s} = 0.11$$

$$\frac{\partial s}{\partial W_1} = 0.1$$



Deep Learning : Artificial Neural Network

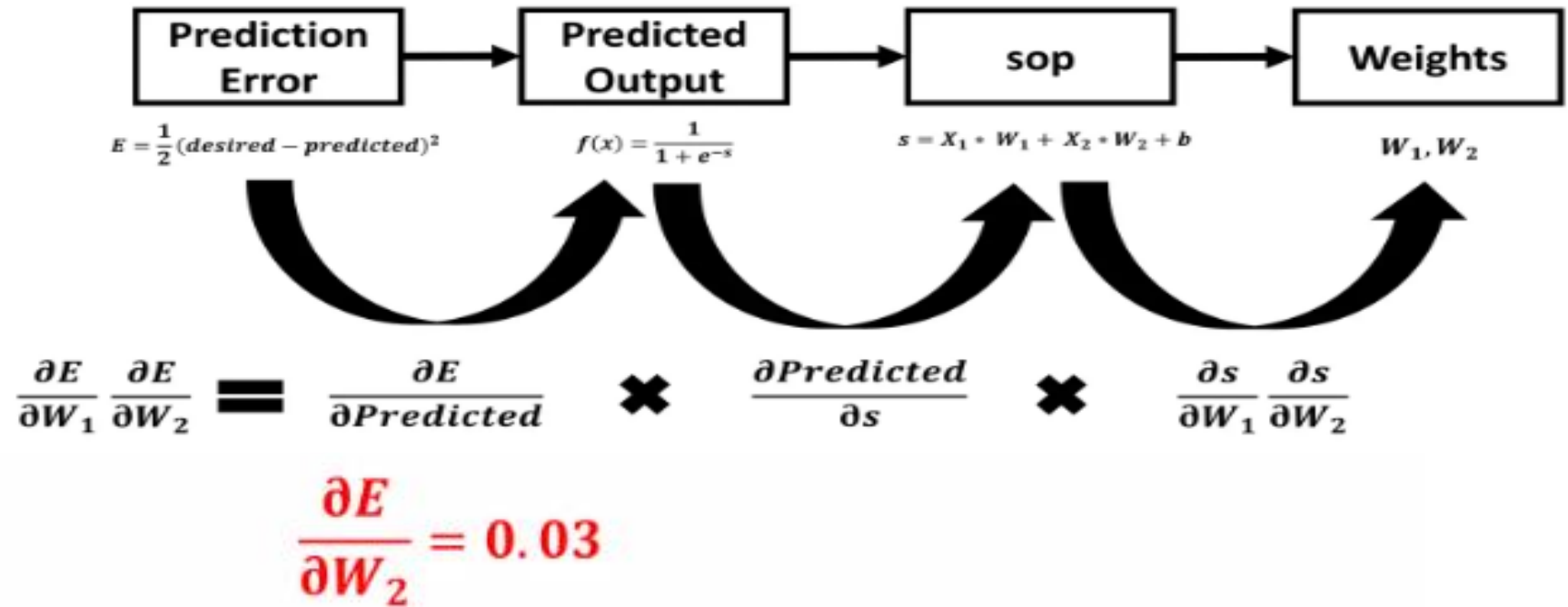
Error- W_2 ($\frac{\partial E}{\partial W_2}$) Partial Derivative

Calculated Derivatives

$$\frac{\partial E}{\partial \text{Predicted}} = 0.844$$

$$\frac{\partial \text{Predicted}}{\partial s} = 0.11$$

$$\frac{\partial s}{\partial W_2} = 0.3$$



Interpreting Derivatives

$$\frac{\partial E}{\partial W_1} = 0.01$$

$$\frac{\partial E}{\partial W_2} = 0.03$$

- There are two useful pieces of information from the derivatives calculated previously.

Derivative Sign

Positive

Increasing/decreasing weight
increases/decreases error.

Negative

Increasing/decreasing weight
decreases/increases error.

Derivative Magnitude

Positive
Sign

Increasing/decreasing weight by P
increases/decreases error by MAG*P.

Negative
Sign

Increasing/decreasing weight by P
decreases/increases error by MAG*P.

In our example, because both $\frac{\partial E}{\partial W_1}$ and $\frac{\partial E}{\partial W_2}$ are positive, then we would like to decrease the weights in order to decrease the prediction error.

Updating Weights

- Each weight will be updated based on its derivative according to this equation:

$$W_{inew} = W_{iold} - \eta * \frac{\partial E}{\partial W_i}$$

Updating W_1

$$\begin{aligned} W_{1new} &= W_1 - \eta * \frac{\partial E}{\partial W_1} \\ &= 0.5 - 0.01 * 0.01 \end{aligned}$$

$$W_{1new} = 0.49991$$

Updating W_2

$$\begin{aligned} W_{2new} &= W_2 - \eta * \frac{\partial E}{\partial W_2} \\ &= 0.2 - 0.01 * 0.028 \end{aligned}$$

$$W_{2new} = 0.1997$$

Continue updating weights according to derivatives and re-train the network until reaching an acceptable error.

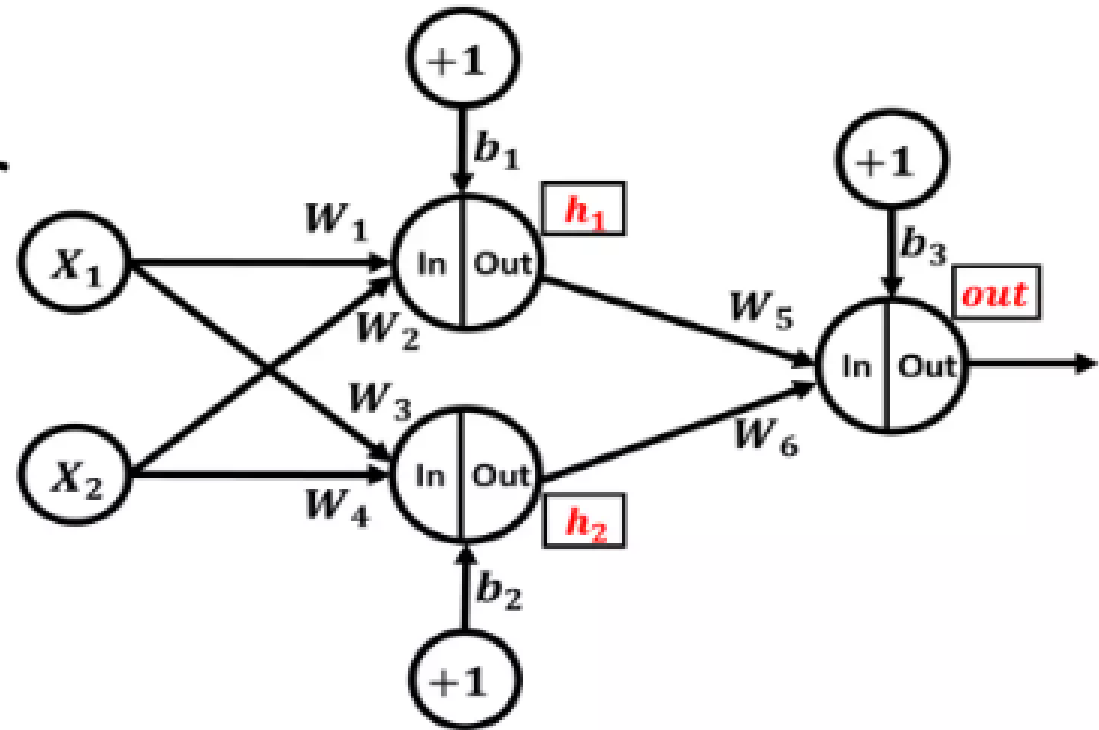
Second Example

Backpropagation for NN with Hidden Layer

ANN with Hidden Layer

Training Data

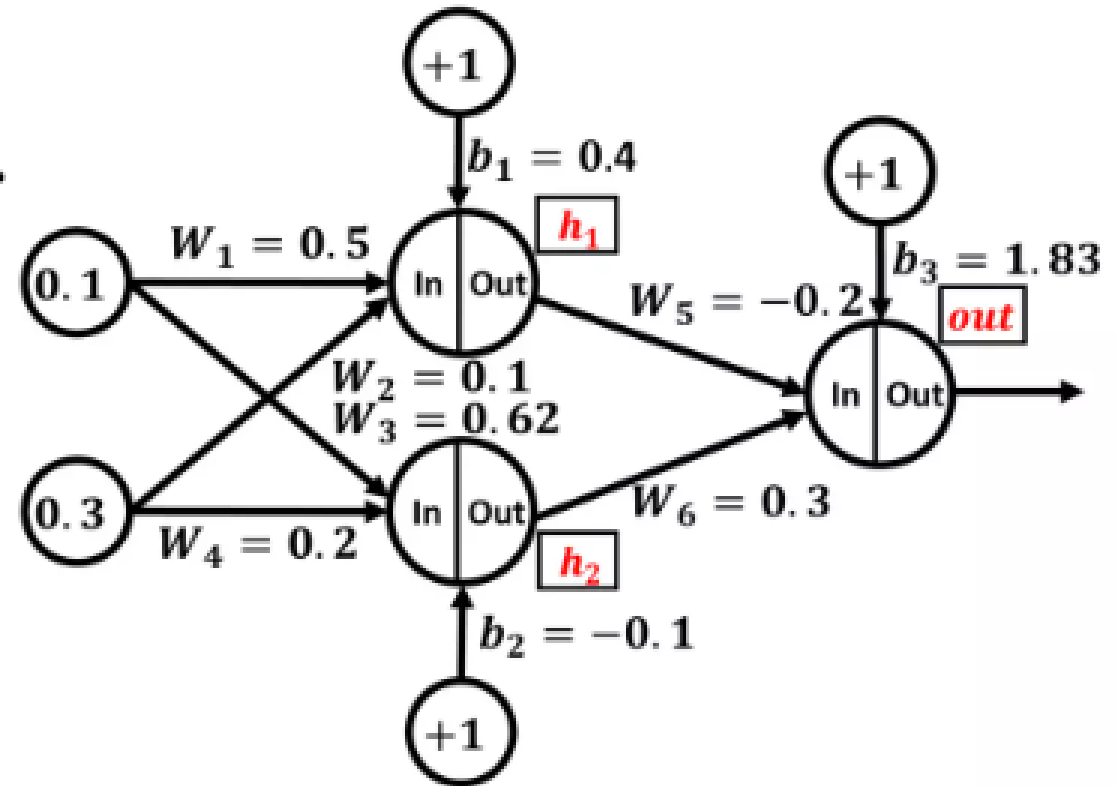
X_1	X_2	Output
0.1	0.3	0.03



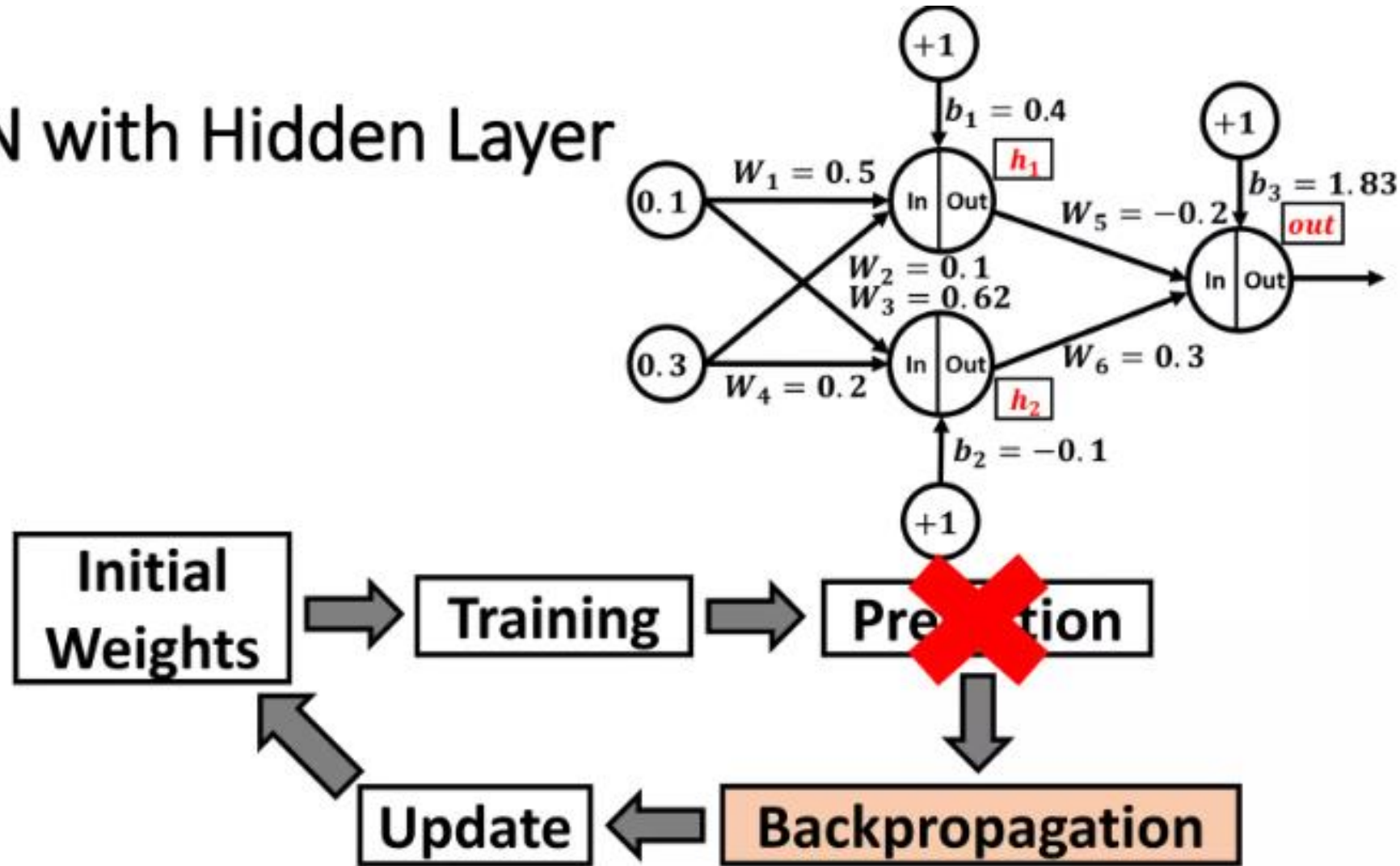
Initial Weights

W_1	W_2	W_3	W_4	W_5	W_6	b_1	b_2	b_3
0.5	0.1	0.62	0.2	-0.2	0.3	0.4	-0.1	1.83

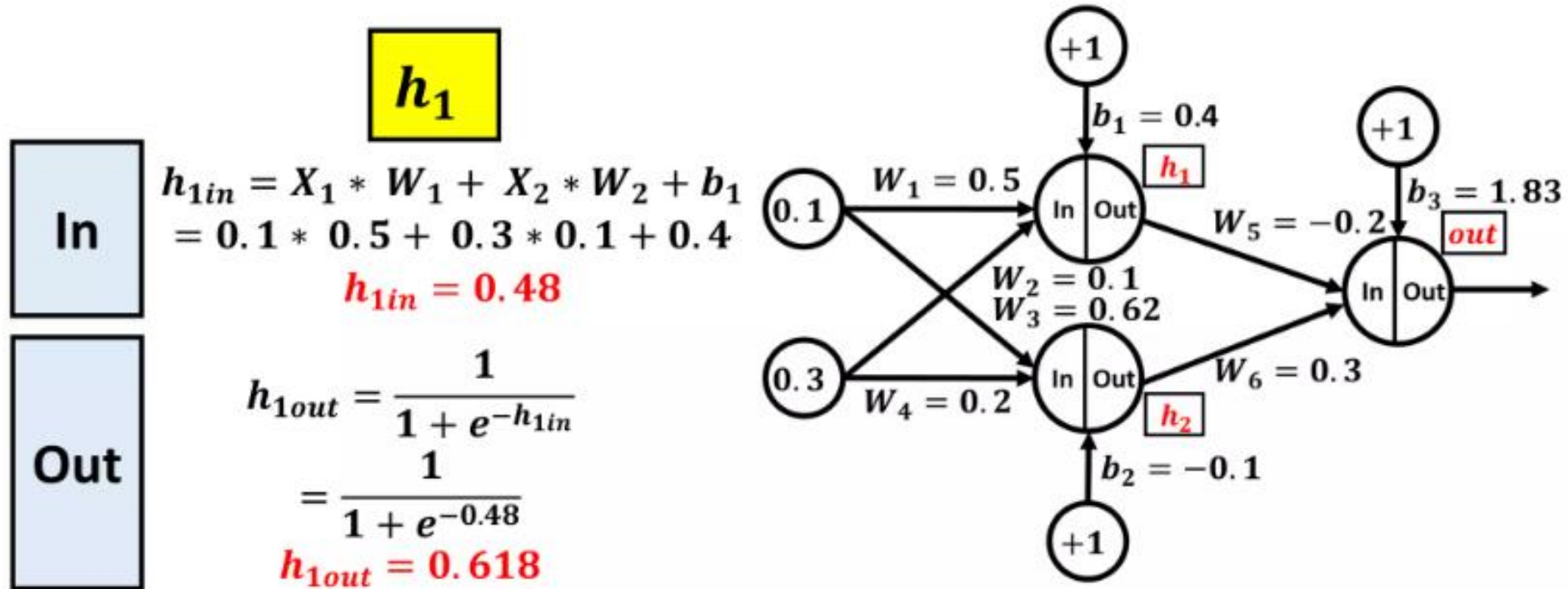
ANN with Hidden Layer



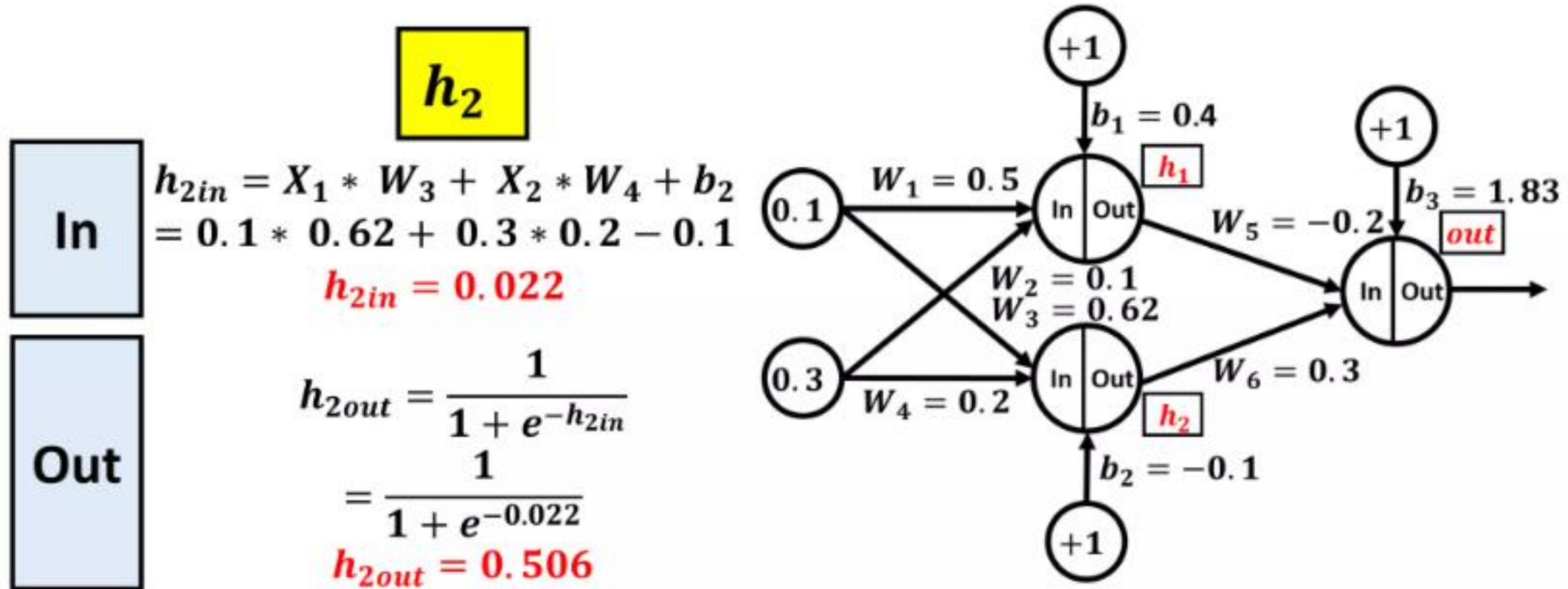
ANN with Hidden Layer



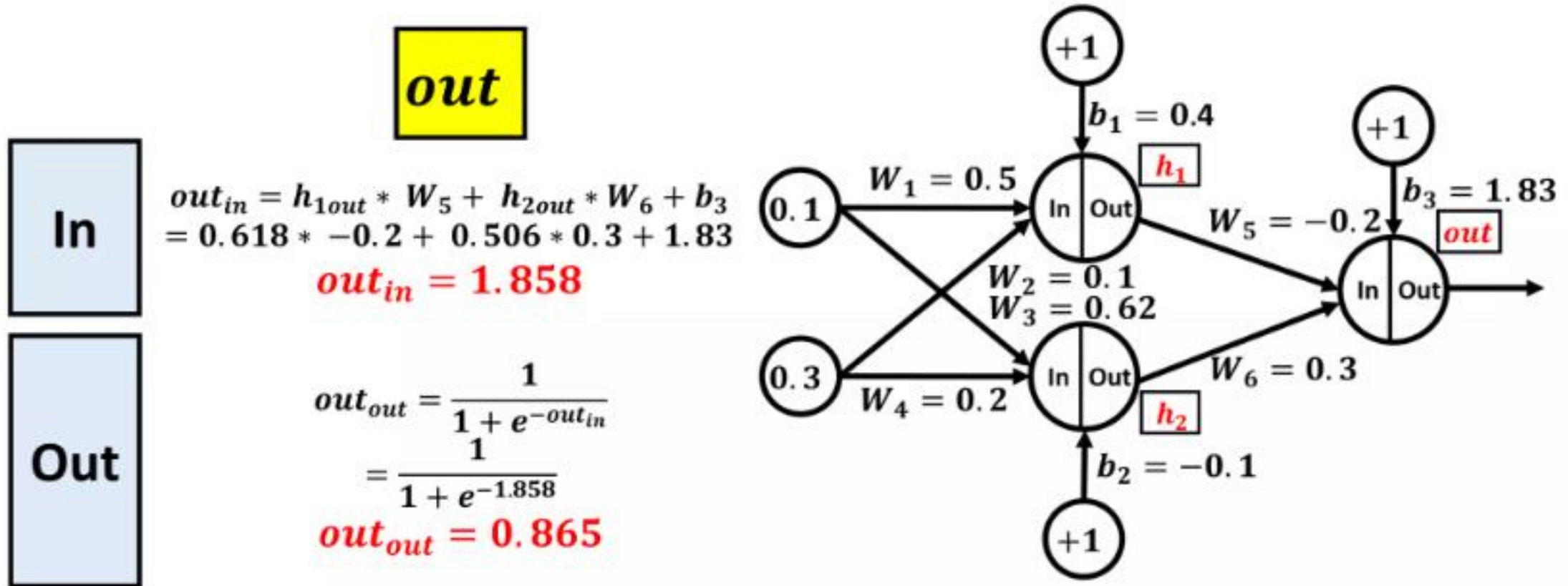
Forward Pass – Hidden Layer Neurons



Forward Pass – Hidden Layer Neurons



Forward Pass – Output Layer Neuron



Forward Pass – Prediction Error

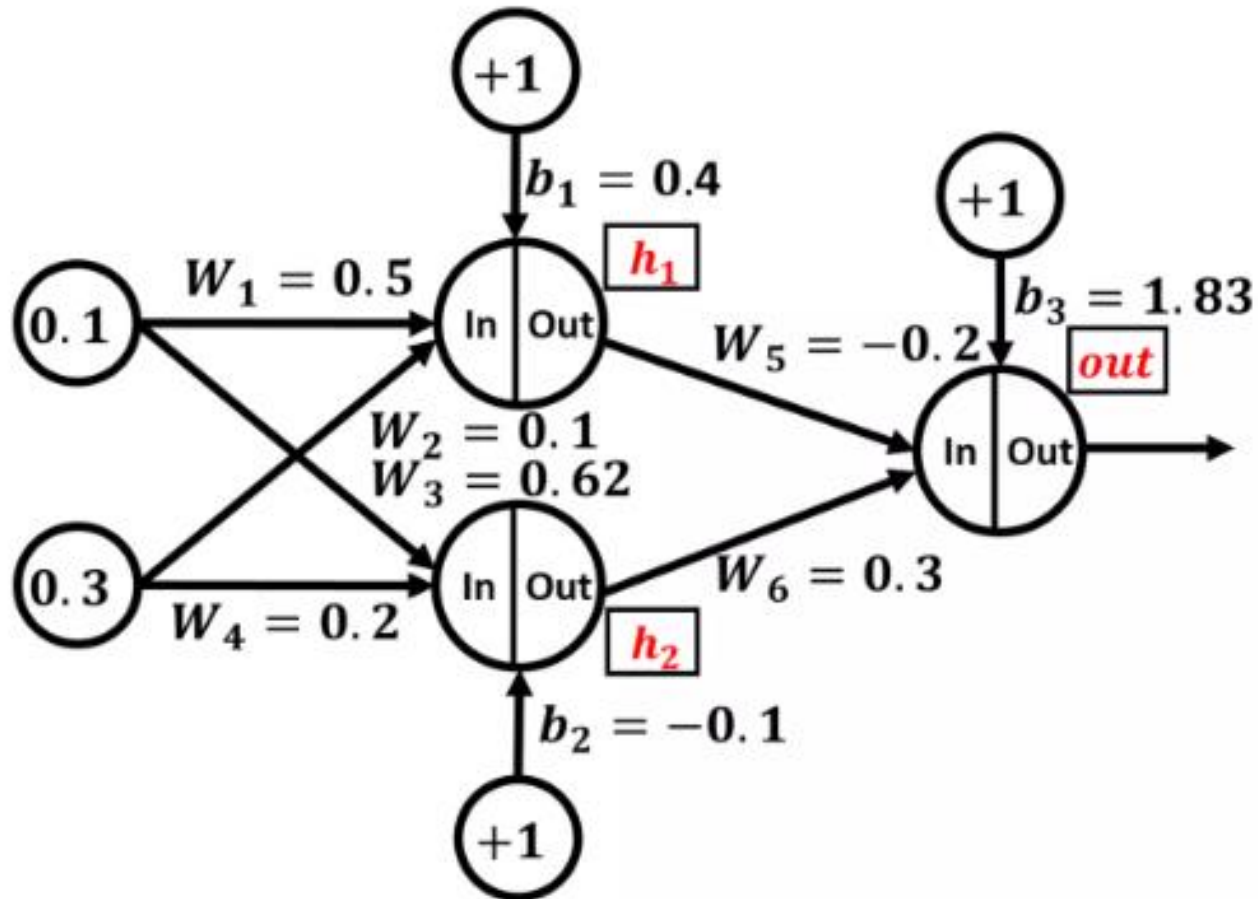
$$\textit{desired} = 0.03$$

$$\textit{Predicted} = \textit{out}_{\textit{out}} = 0.865$$

$$\begin{aligned} E &= \frac{1}{2} (\textit{desired} - \textit{out}_{\textit{out}})^2 \\ &= \frac{1}{2} (0.03 - 0.865)^2 \\ E &= 0.349 \end{aligned}$$

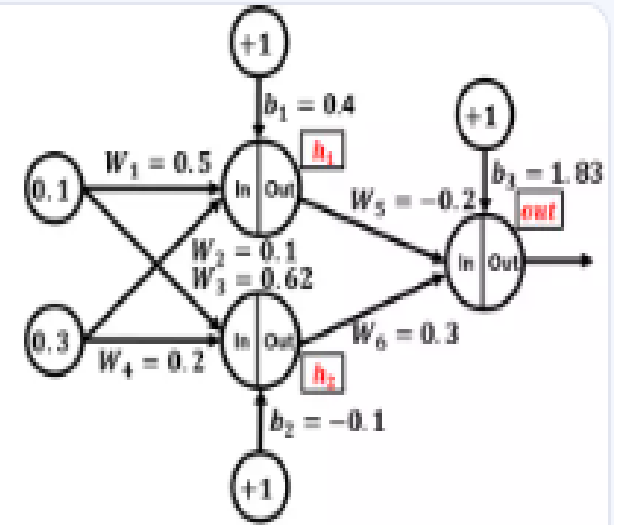
$$\frac{\partial E}{\partial W_1'}, \frac{\partial E}{\partial W_2'}, \frac{\partial E}{\partial W_3'}, \frac{\partial E}{\partial W_4'}, \frac{\partial E}{\partial W_5'}, \frac{\partial E}{\partial W_6'}$$

Partial Derivatives Calculation



$E - W_5 \left(\frac{\partial E}{\partial W_5} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_5} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial W_5}$$



Deep Learning : Artificial Neural Network

$E - W_5 \left(\frac{\partial E}{\partial W_5} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_5} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial W_5}$$

Partial Derivative

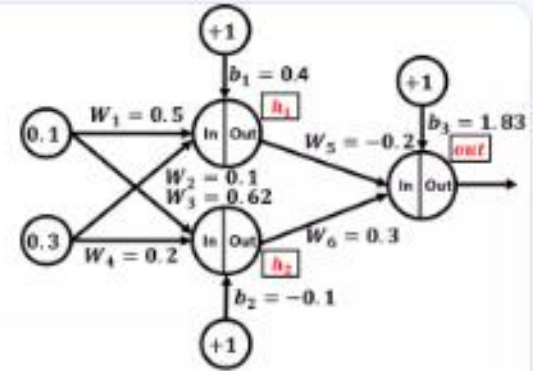
$$\begin{aligned} \frac{\partial E}{\partial out_{out}} &= \frac{\partial}{\partial out_{out}} \left(\frac{1}{2} (desired - out_{out})^2 \right) \\ &= 2 * \frac{1}{2} (desired - out_{out})^{2-1} * (0 - 1) \\ &= desired - out_{out} * (-1) \end{aligned}$$

$$\frac{\partial E}{\partial out_{out}} = out_{out} - desired$$

Substitution

$$\frac{\partial E}{\partial out_{out}} = out_{out} - desired = 0.865 - 0.03$$

$$\frac{\partial E}{\partial out_{out}} = 0.835$$



Deep Learning : Artificial Neural Network

$E - W_5 \left(\frac{\partial E}{\partial W_5} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_5} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial W_5}$$

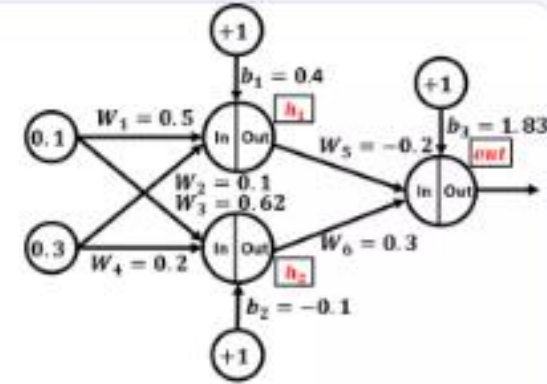
Partial Derivative

$$\frac{\partial out_{out}}{\partial out_{in}} = \frac{\partial}{\partial out_{in}} \left(\frac{1}{1 + e^{-out_{in}}} \right)$$

$$\frac{\partial out_{out}}{\partial out_{in}} = \left(\frac{1}{1 + e^{-out_{in}}} \right) \left(1 - \frac{1}{1 + e^{-out_{in}}} \right)$$

Substitution

$$\begin{aligned} \frac{\partial out_{out}}{\partial out_{in}} &= \left(\frac{1}{1 + e^{-1.858}} \right) \left(1 - \frac{1}{1 + e^{-1.858}} \right) \\ &= \left(\frac{1}{1.56} \right) \left(1 - \frac{1}{1.56} \right) \\ &= (0.641)(1 - 0.641) = (0.641)(0.359) \\ \frac{\partial out_{out}}{\partial out_{in}} &= 0.23 \end{aligned}$$



Deep Learning : Artificial Neural Network

$E - W_5 \left(\frac{\partial E}{\partial W_5} \right)$ Partial Derivative

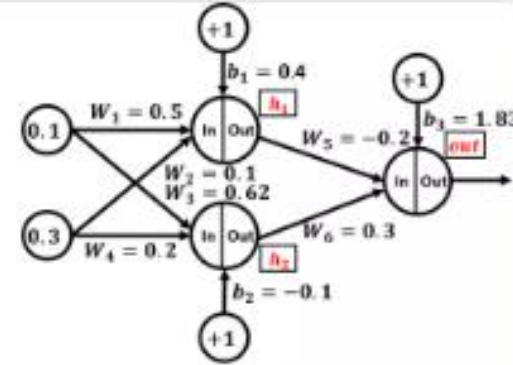
$$\frac{\partial E}{\partial W_5} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial W_5}$$

Partial Derivative

$$\begin{aligned} \frac{\partial out_{in}}{\partial W_5} &= \frac{\partial}{\partial W_5} (h_{1out} * W_5 + h_{2out} * W_6 + b_3) \\ &= 1 * h_{1out} * (W_5)^{1-1} + 0 + 0 \\ \frac{\partial out_{in}}{\partial W_5} &= h_{1out} \end{aligned}$$

Substitution

$$\begin{aligned} \frac{\partial out_{in}}{\partial W_5} &= h_{1out} \\ \frac{\partial out_{in}}{\partial W_5} &= 0.618 \end{aligned}$$



Deep Learning : Artificial Neural Network

$E - W_5 \left(\frac{\partial E}{\partial W_5} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_5} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial W_5}$$

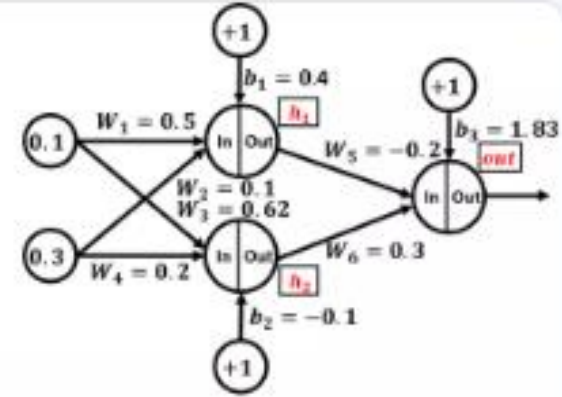
$$\frac{\partial E}{\partial out_{out}} = 0.835$$

$$\frac{\partial out_{out}}{\partial out_{in}} = 0.23$$

$$\frac{\partial out_{in}}{\partial W_5} = 0.618$$

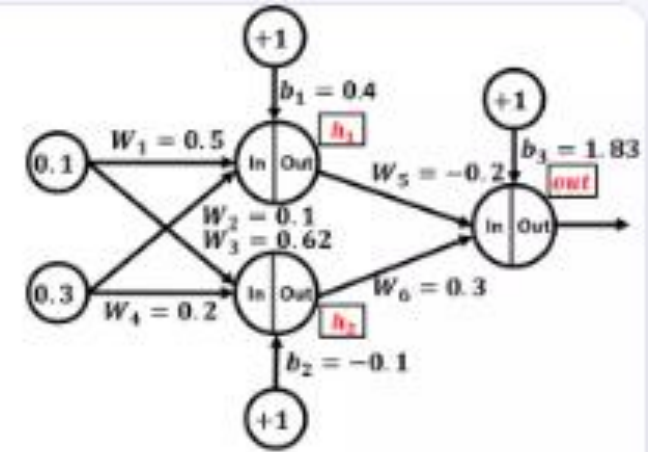
$$\frac{\partial E}{\partial W_5} = 0.835 * 0.23 * 0.618$$

$$\frac{\partial E}{\partial W_5} = 0.119$$



$E - W_6 \left(\frac{\partial E}{\partial W_6} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_6} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial W_6}$$

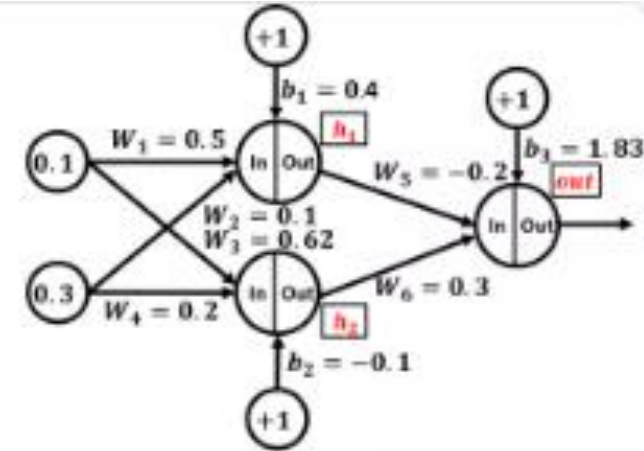


$E - W_6 \left(\frac{\partial E}{\partial W_6} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_6} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial W_6}$$

$$\frac{\partial E}{\partial out_{out}} = 0.835$$

$$\frac{\partial out_{out}}{\partial out_{in}} = 0.23$$



Deep Learning : Artificial Neural Network

$E - W_6 \left(\frac{\partial E}{\partial W_6} \right)$ Partial Derivative

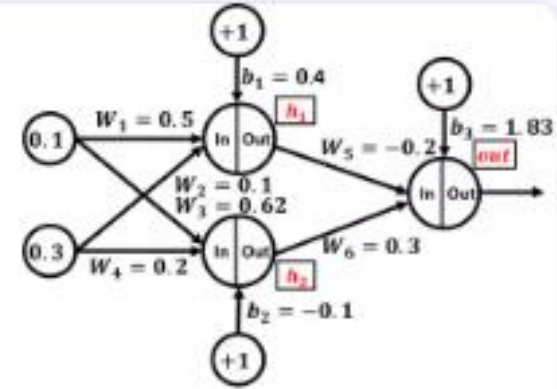
$$\frac{\partial E}{\partial W_5} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial W_6}$$

Partial Derivative

$$\begin{aligned} \frac{\partial out_{in}}{\partial W_6} &= \frac{\partial}{\partial W_6} (h_{1out} * W_5 + h_{2out} * W_6 + b_3) \\ &= 0 + 1 * h_{2out} * (W_6)^{1-1} + 0 \\ \frac{\partial out_{in}}{\partial W_6} &= h_{2out} \end{aligned}$$

Substitution

$$\begin{aligned} \frac{\partial out_{in}}{\partial W_6} &= h_{2out} \\ \frac{\partial out_{in}}{\partial W_6} &= 0.506 \end{aligned}$$



$E - W_6 \left(\frac{\partial E}{\partial W_6} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_6} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial W_6}$$

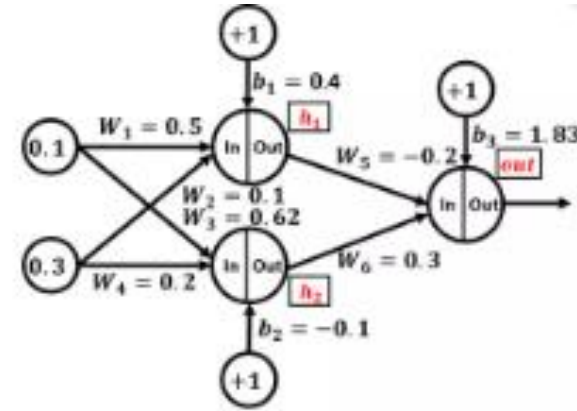
$$\frac{\partial E}{\partial out_{out}} = 0.835$$

$$\frac{\partial out_{out}}{\partial out_{in}} = 0.23$$

$$\frac{\partial out_{in}}{\partial W_6} = 0.506$$

$$\frac{\partial E}{\partial W_6} = 0.835 * 0.23 * 0.506$$

$$\frac{\partial E}{\partial W_6} = 0.097$$



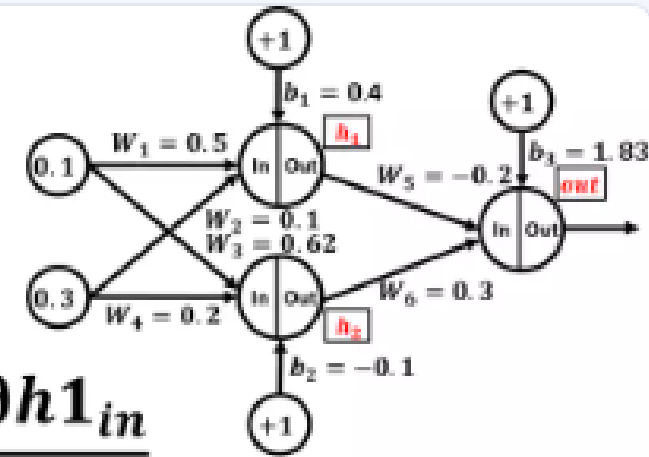
$E - W_1 \left(\frac{\partial E}{\partial W_1} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_1} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h1_{out}} * \frac{\partial h1_{out}}{\partial h1_{in}} * \frac{\partial h1_{in}}{\partial W_1}$$

$$\frac{\partial E}{\partial W_1} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h1_{out}} * \frac{\partial h1_{out}}{\partial h1_{in}} * \frac{\partial h1_{in}}{\partial W_1}$$

$$\frac{\partial E}{\partial out_{out}} = 0.835$$

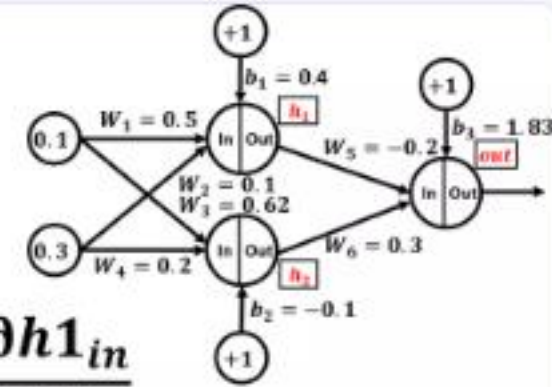
$$\frac{\partial out_{out}}{\partial out_{in}} = 0.23$$



Deep Learning : Artificial Neural Network

$E - W_1 \left(\frac{\partial E}{\partial W_1} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_1} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h1_{out}} * \frac{\partial h1_{out}}{\partial h1_{in}} * \frac{\partial h1_{in}}{\partial W_1}$$



Partial Derivative

$$\begin{aligned} \frac{\partial out_{in}}{\partial h1_{out}} &= \frac{\partial}{\partial h1_{out}} (h1_{out} * W_5 + h2_{out} * W_6 + b_3) \\ &= (h1_{out})^{1-1} * W_5 + 0 + 0 \\ \frac{\partial out_{in}}{\partial h1_{out}} &= W_5 \end{aligned}$$

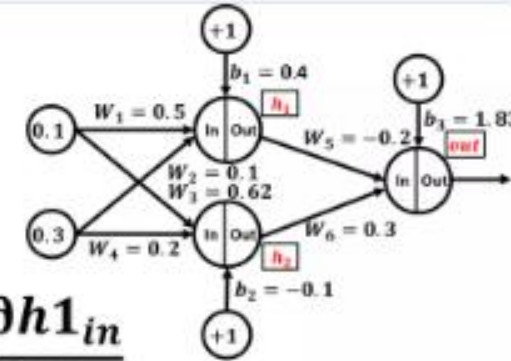
Substitution

$$\begin{aligned} \frac{\partial out_{in}}{\partial h1_{out}} &= W_5 \\ \frac{\partial out_{in}}{\partial h1_{out}} &= -0.2 \end{aligned}$$

Deep Learning : Artificial Neural Network

$E - W_1 \left(\frac{\partial E}{\partial W_1} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_1} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h1_{out}} * \frac{\partial h1_{out}}{\partial h1_{in}} * \frac{\partial h1_{in}}{\partial W_1}$$



Partial Derivative

$$\frac{\partial h1_{out}}{\partial h1_{in}} = \frac{\partial}{\partial h1_{in}} \left(\frac{1}{1 + e^{-h1_{in}}} \right)$$

$$\frac{\partial h1_{out}}{\partial h1_{in}} = \left(\frac{1}{1 + e^{-h1_{in}}} \right) \left(1 - \frac{1}{1 + e^{-h1_{in}}} \right)$$

Substitution

$$\frac{\partial h1_{out}}{\partial h1_{in}} = \left(\frac{1}{1 + e^{-h1_{in}}} \right) \left(1 - \frac{1}{1 + e^{-h1_{in}}} \right)$$

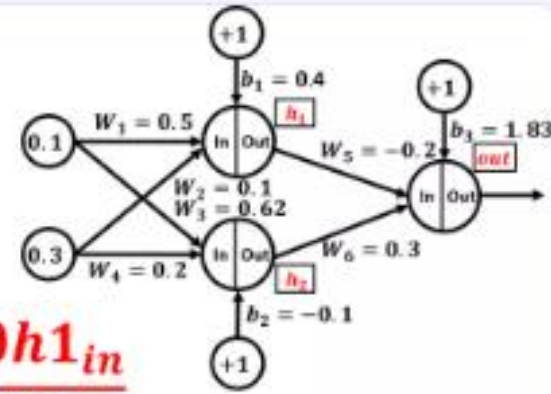
$$= \left(\frac{1}{1 + e^{-0.48}} \right) \left(1 - \frac{1}{1 + e^{-0.48}} \right)$$

$$\frac{\partial h2_{out}}{\partial h2_{in}} = 0.236$$

Deep Learning : Artificial Neural Network

$E - W_1 \left(\frac{\partial E}{\partial W_1} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_1} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h1_{out}} * \frac{\partial h1_{out}}{\partial h1_{in}} * \frac{\partial h1_{in}}{\partial W_1}$$



Partial Derivative

$$\begin{aligned} \frac{\partial h1_{in}}{\partial W_1} &= \frac{\partial}{\partial W_1} (X_1 * W_1 + X_2 * W_2 + b_1) \\ &= X_1 * (W_1)^{1-1} + 0 + 0 \\ \frac{\partial h1_{in}}{\partial W_1} &= X_1 \end{aligned}$$

Substitution

$$\begin{aligned} \frac{\partial h1_{in}}{\partial W_1} &= X_1 \\ \frac{\partial h1_{in}}{\partial W_1} &= 0.1 \end{aligned}$$

Deep Learning : Artificial Neural Network

$E - W_1 \left(\frac{\partial E}{\partial W_1} \right)$ Partial Derivative

$$\frac{\partial E}{\partial W_1} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h1_{out}} * \frac{\partial h1_{out}}{\partial h1_{in}} * \frac{\partial h1_{in}}{\partial W_1}$$

$$\frac{\partial E}{\partial out_{out}} = 0.835$$

$$\frac{\partial out_{out}}{\partial out_{in}} = 0.23$$

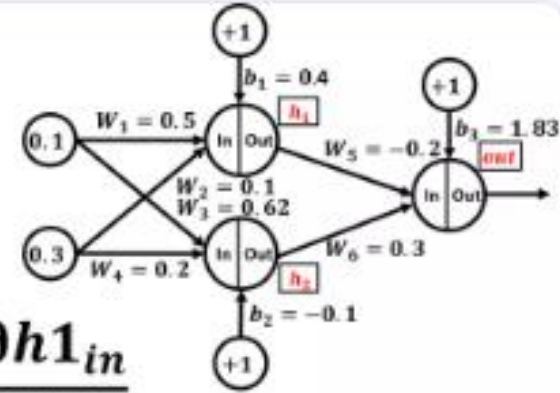
$$\frac{\partial out_{in}}{\partial h1_{out}} = -0.2$$

$$\frac{\partial h2_{out}}{\partial h2_{in}} = 0.236$$

$$\frac{\partial h1_{in}}{\partial W_1} = 0.1$$

$$\frac{\partial E}{\partial W_1} = 0.835 * 0.23 * -0.2 * 0.236 * 0.1$$

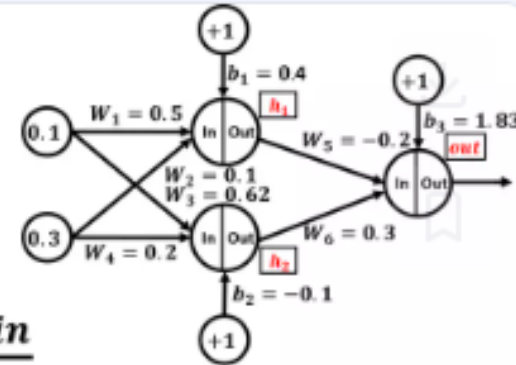
$$\frac{\partial E}{\partial W_1} = -0.001$$



Deep Learning : Artificial Neural Network

$E - W_2 \left(\frac{\partial E}{\partial W_2} \right)$ Partial Derivative:

$$\frac{\partial E}{\partial W_2} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h1_{out}} * \frac{\partial h1_{out}}{\partial h1_{in}} * \frac{\partial h1_{in}}{\partial W_2}$$



$$\frac{\partial E}{\partial W_2} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h1_{out}} * \frac{\partial h1_{out}}{\partial h1_{in}} * \frac{\partial h1_{in}}{\partial W_2}$$

$$\frac{\partial E}{\partial out_{out}} = 0.835 \quad \frac{\partial out_{out}}{\partial out_{in}} = 0.23 \quad \frac{\partial out_{in}}{\partial h1_{out}} = -0.2 \quad \frac{\partial h1_{out}}{\partial h1_{in}} = 0.236$$

Deep Learning : Artificial Neural Network

$E - W_2 \left(\frac{\partial E}{\partial W_2} \right)$ Partial Derivative:

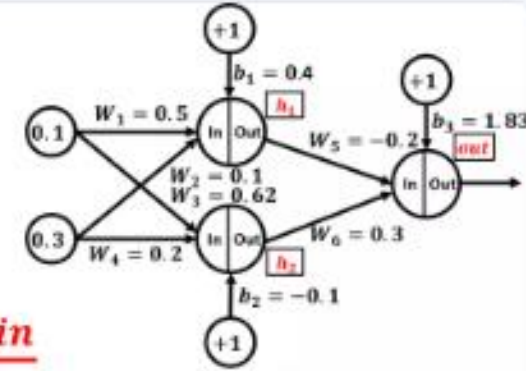
$$\frac{\partial E}{\partial W_2} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h1_{out}} * \frac{\partial h1_{out}}{\partial h1_{in}} * \frac{\partial h1_{in}}{\partial W_2}$$

Partial Derivative

$$\begin{aligned} \frac{\partial h1_{in}}{\partial W_2} &= \frac{\partial}{\partial W_2} (X_1 * W_1 + X_2 * W_2 + b_1) \\ &= 0 + X_2 * (W_2)^{1-1} + 0 \\ \frac{\partial h1_{in}}{\partial W_2} &= X_2 \end{aligned}$$

Substitution

$$\begin{aligned} \frac{\partial h1_{in}}{\partial W_2} &= X_2 \\ \frac{\partial h1_{in}}{\partial W_2} &= 0.3 \end{aligned}$$



Deep Learning : Artificial Neural Network

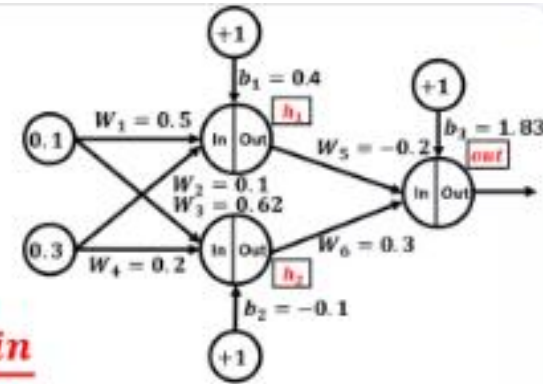
$E - W_2 \left(\frac{\partial E}{\partial W_2} \right)$ Partial Derivative:

$$\frac{\partial E}{\partial W_2} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h1_{out}} * \frac{\partial h1_{out}}{\partial h1_{in}} * \frac{\partial h1_{in}}{\partial W_2}$$

$$\frac{\partial E}{\partial out_{out}} = 0.835 \quad \frac{\partial out_{out}}{\partial out_{in}} = 0.23 \quad \frac{\partial out_{in}}{\partial h1_{out}} = -0.2 \quad \frac{\partial h1_{out}}{\partial h2_{in}} = 0.236 \quad \frac{\partial h1_{in}}{\partial W_2} = 0.3$$

$$\frac{\partial E}{\partial W_2} = 0.835 * 0.23 * -0.2 * 0.236 * 0.3$$

$$\frac{\partial E}{\partial W_2} = -.003$$



Deep Learning : Artificial Neural Network

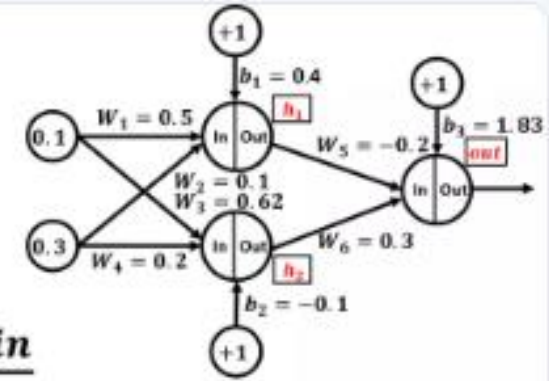
$E - W_3 \left(\frac{\partial E}{\partial W_3} \right)$ Partial Derivative:

$$\frac{\partial E}{\partial W_3} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h2_{out}} * \frac{\partial h2_{out}}{\partial h2_{in}} * \frac{\partial h2_{in}}{\partial W_3}$$

$$\frac{\partial E}{\partial W_3} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h2_{out}} * \frac{\partial h2_{out}}{\partial h2_{in}} * \frac{\partial h2_{in}}{\partial W_3}$$

$$\frac{\partial E}{\partial out_{out}} = 0.835$$

$$\frac{\partial out_{out}}{\partial out_{in}} = 0.23$$



Deep Learning : Artificial Neural Network

$E - W_3 \left(\frac{\partial E}{\partial W_3} \right)$ Partial Derivative:

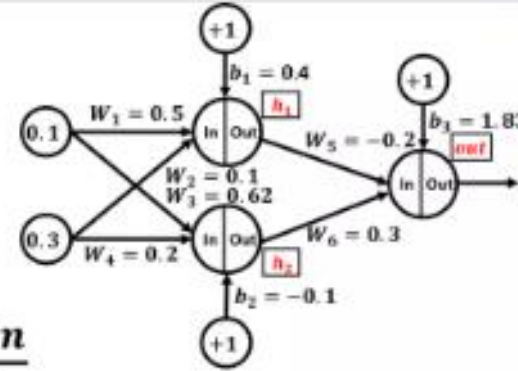
$$\frac{\partial E}{\partial W_3} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h2_{out}} * \frac{\partial h2_{out}}{\partial h2_{in}} * \frac{\partial h2_{in}}{\partial W_3}$$

Partial Derivative

$$\begin{aligned} \frac{\partial out_{in}}{\partial h2_{out}} &= \frac{\partial}{\partial h2_{out}} (h1_{out} * W_5 + h2_{out} * W_6 + b_3) \\ &= 0 + (h2_{out})^{1-1} * W_6 + 0 \\ \frac{\partial out_{in}}{\partial h2_{out}} &= W_6 \end{aligned}$$

Substitution

$$\begin{aligned} \frac{\partial out_{in}}{\partial h2_{out}} &= W_6 \\ \frac{\partial out_{in}}{\partial h2_{out}} &= 0.3 \end{aligned}$$



Deep Learning : Artificial Neural Network

$E - W_3 \left(\frac{\partial E}{\partial W_3} \right)$ Partial Derivative:

$$\frac{\partial E}{\partial W_3} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h2_{out}} * \frac{\partial h2_{out}}{\partial h2_{in}} * \frac{\partial h2_{in}}{\partial W_3}$$

Partial Derivative

$$\frac{\partial h2_{out}}{\partial h2_{in}} = \frac{\partial}{\partial h2_{in}} \left(\frac{1}{1 + e^{-h2_{in}}} \right)$$

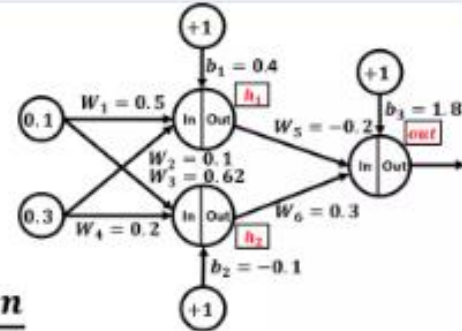
$$\frac{\partial h2_{out}}{\partial h2_{in}} = \left(\frac{1}{1 + e^{-h2_{in}}} \right) \left(1 - \frac{1}{1 + e^{-h2_{in}}} \right)$$

Substitution

$$\frac{\partial h2_{out}}{\partial h2_{in}} = \left(\frac{1}{1 + e^{-h2_{in}}} \right) \left(1 - \frac{1}{1 + e^{-h2_{in}}} \right)$$

$$= \left(\frac{1}{1 + e^{-0.022}} \right) \left(1 - \frac{1}{1 + e^{-0.022}} \right)$$

$$\frac{\partial h2_{out}}{\partial h2_{in}} = 0.25$$



Deep Learning : Artificial Neural Network

$E - W_3 \left(\frac{\partial E}{\partial W_3} \right)$ Partial Derivative:

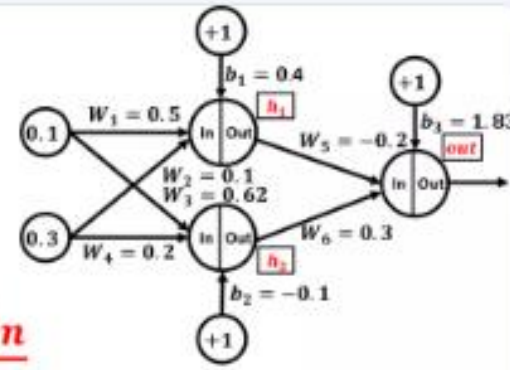
$$\frac{\partial E}{\partial W_3} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h2_{out}} * \frac{\partial h2_{out}}{\partial h2_{in}} * \frac{\partial h2_{in}}{\partial W_3}$$

Partial Derivative

$$\begin{aligned} \frac{\partial h2_{in}}{\partial W_3} &= \frac{\partial}{\partial W_3} (X_1 * W_3 + X_2 * W_4 + b_2) \\ &= X_1 * W_3 + X_2 * W_4 + b_2 \\ &= (X_1)^{1-1} * W_3 + 0 + 0 \\ \frac{\partial h2_{in}}{\partial W_3} &= W_3 \end{aligned}$$

Substitution

$$\begin{aligned} \frac{\partial h2_{in}}{\partial W_3} &= W_3 \\ \frac{\partial h2_{in}}{\partial W_3} &= 0.62 \end{aligned}$$



Deep Learning : Artificial Neural Network

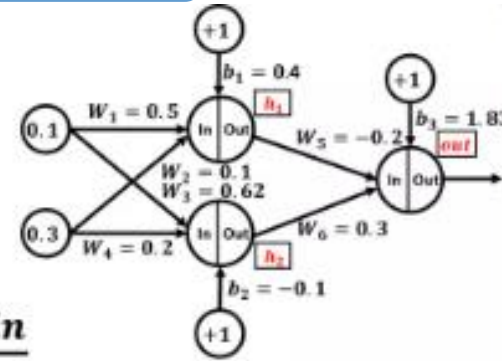
$E - W_3 \left(\frac{\partial E}{\partial W_3} \right)$ Partial Derivative:

$$\frac{\partial E}{\partial W_3} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h2_{out}} * \frac{\partial h2_{out}}{\partial h2_{in}} * \frac{\partial h2_{in}}{\partial W_3}$$

$$\frac{\partial E}{\partial out_{out}} = 0.835 \quad \frac{\partial out_{out}}{\partial out_{in}} = 0.23 \quad \frac{\partial out_{in}}{\partial h2_{out}} = 0.3 \quad \frac{\partial h2_{out}}{\partial h2_{in}} = 0.25 \quad \frac{\partial h2_{in}}{\partial W_3} = 0.62$$

$$\frac{\partial E}{\partial W_3} = 0.835 * 0.23 * 0.3 * 0.25 * 0.62$$

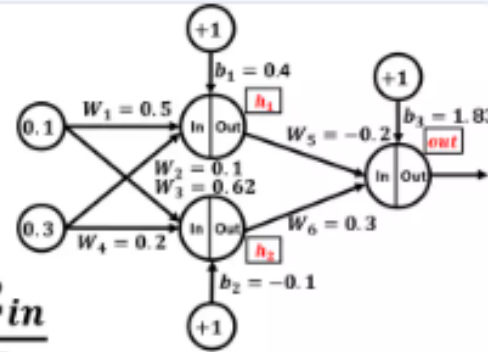
$$\frac{\partial E}{\partial W_3} = 0.009$$



Deep Learning : Artificial Neural Network

$E - W_4 \left(\frac{\partial E}{\partial W_4} \right)$ Partial Derivative:

$$\frac{\partial E}{\partial W_4} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h2_{out}} * \frac{\partial h2_{out}}{\partial h2_{in}} * \frac{\partial h2_{in}}{\partial W_4}$$



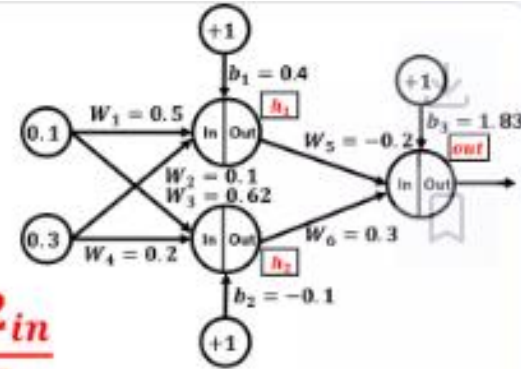
$$\frac{\partial E}{\partial W_4} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h2_{out}} * \frac{\partial h2_{out}}{\partial h2_{in}} * \frac{\partial h2_{in}}{\partial W_4}$$

$$\frac{\partial E}{\partial out_{out}} = 0.835 \quad \frac{\partial out_{out}}{\partial out_{in}} = 0.23 \quad \frac{\partial out_{in}}{\partial h2_{out}} = 0.3 \quad \frac{\partial h2_{out}}{\partial h2_{in}} = 0.25$$

Deep Learning : Artificial Neural Network

$E - W_4 \left(\frac{\partial E}{\partial W_4} \right)$ Partial Derivative:

$$\frac{\partial E}{\partial W_4} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h2_{out}} * \frac{\partial h2_{out}}{\partial h2_{in}} * \frac{\partial h2_{in}}{\partial W_4}$$



Partial Derivative

$$\begin{aligned} \frac{\partial h2_{in}}{\partial W_4} &= \frac{\partial}{\partial W_4} (X_1 * W_3 + X_2 * W_4 + b_2) \\ &= X_1 * W_3 + X_2 * W_4 + b_2 \\ &= 0 + (X_2)^{1-1} * W_4 + 0 \\ \frac{\partial h2_{in}}{\partial W_4} &= W_4 \end{aligned}$$

Substitution

$$\begin{aligned} \frac{\partial h2_{in}}{\partial W_4} &= W_4 \\ \frac{\partial h2_{in}}{\partial W_4} &= 0.2 \end{aligned}$$

Deep Learning : Artificial Neural Network

$E - W_4 \left(\frac{\partial E}{\partial W_4} \right)$ Partial Derivative:

$$\frac{\partial E}{\partial W_4} = \frac{\partial E}{\partial out_{out}} * \frac{\partial out_{out}}{\partial out_{in}} * \frac{\partial out_{in}}{\partial h2_{out}} * \frac{\partial h2_{out}}{\partial h2_{in}} * \frac{\partial h2_{in}}{\partial W_4}$$

$$\frac{\partial E}{\partial out_{out}} = 0.835$$

$$\frac{\partial out_{out}}{\partial out_{in}} = 0.23$$

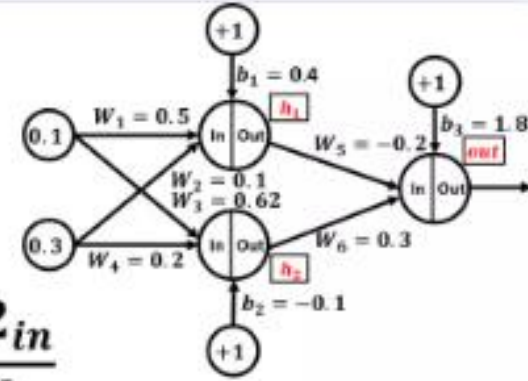
$$\frac{\partial out_{in}}{\partial h2_{out}} = 0.3$$

$$\frac{\partial h2_{out}}{\partial h2_{in}} = 0.25$$

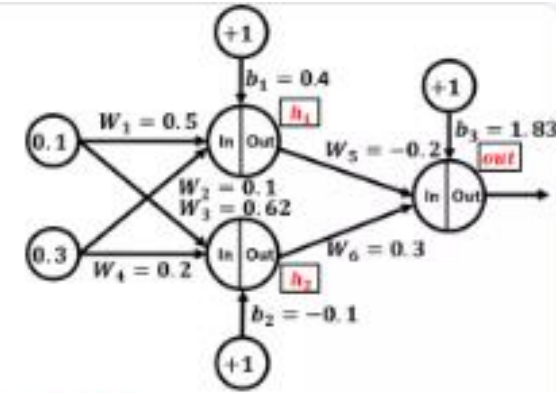
$$\frac{\partial h2_{in}}{\partial W_4} = 0.2$$

$$\frac{\partial E}{\partial W_4} = 0.835 * 0.23 * 0.3 * 0.25 * 0.2$$

$$\frac{\partial E}{\partial W_4} = 0.003$$



All Error-Weights Partial Derivatives



$$\frac{\partial E}{\partial W_1} = -0.001$$

$$\frac{\partial E}{\partial W_2} = -.003$$

$$\frac{\partial E}{\partial W_3} = 0.009$$

$$\frac{\partial E}{\partial W_4} = 0.003$$

$$\frac{\partial E}{\partial W_5} = 0.119$$

$$\frac{\partial E}{\partial W_6} = 0.097$$

Updated Weights

$$W_{1new} = W_1 - \eta * \frac{\partial E}{\partial W_1} = 0.5 - 0.01 * -0.001 = \mathbf{0.50001}$$

$$W_{2new} = W_2 - \eta * \frac{\partial E}{\partial W_2} = 0.1 - 0.01 * -0.003 = \mathbf{0.10003}$$

$$W_{3new} = W_3 - \eta * \frac{\partial E}{\partial W_3} = 0.62 - 0.01 * 0.009 = \mathbf{0.61991}$$

$$W_{4new} = W_4 - \eta * \frac{\partial E}{\partial W_4} = 0.2 - 0.01 * 0.003 = \mathbf{0.1997}$$

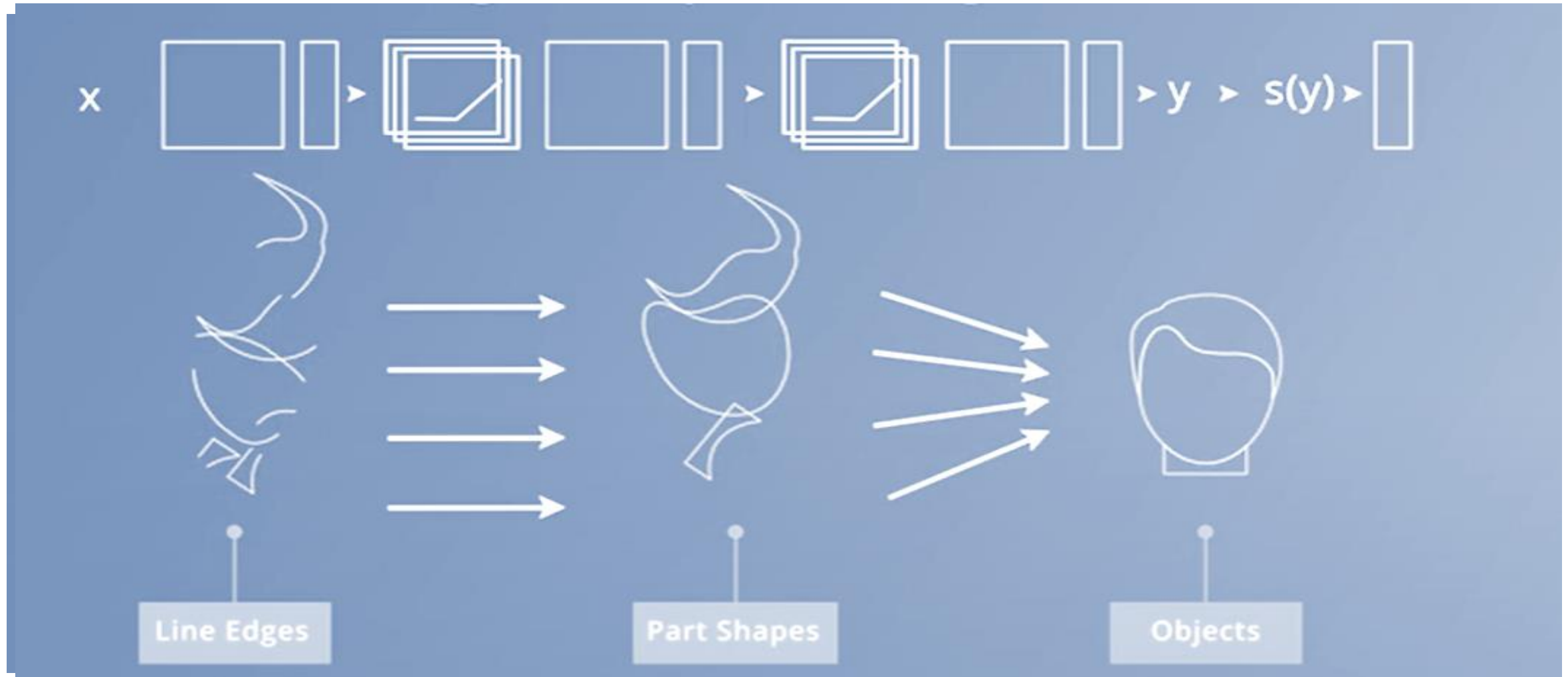
$$W_{5new} = W_5 - \eta * \frac{\partial E}{\partial W_5} = -0.2 - 0.01 * 0.618 = \mathbf{-0.20618}$$

$$W_{6new} = W_6 - \eta * \frac{\partial E}{\partial W_6} = 0.3 - 0.01 * 0.097 = \mathbf{0.29903}$$

Continue updating weights according to derivatives and re-train the network until reaching an acceptable error.

Deep Learning 1 : Artificial Neural Network

Training a Deep Learning Network



Activation Function

How to choose right activation functions?

Sometimes leaky ReLU doesn't help in waking up dead neurons so parametric is used which can give more stable training if the parameter was learnt well.

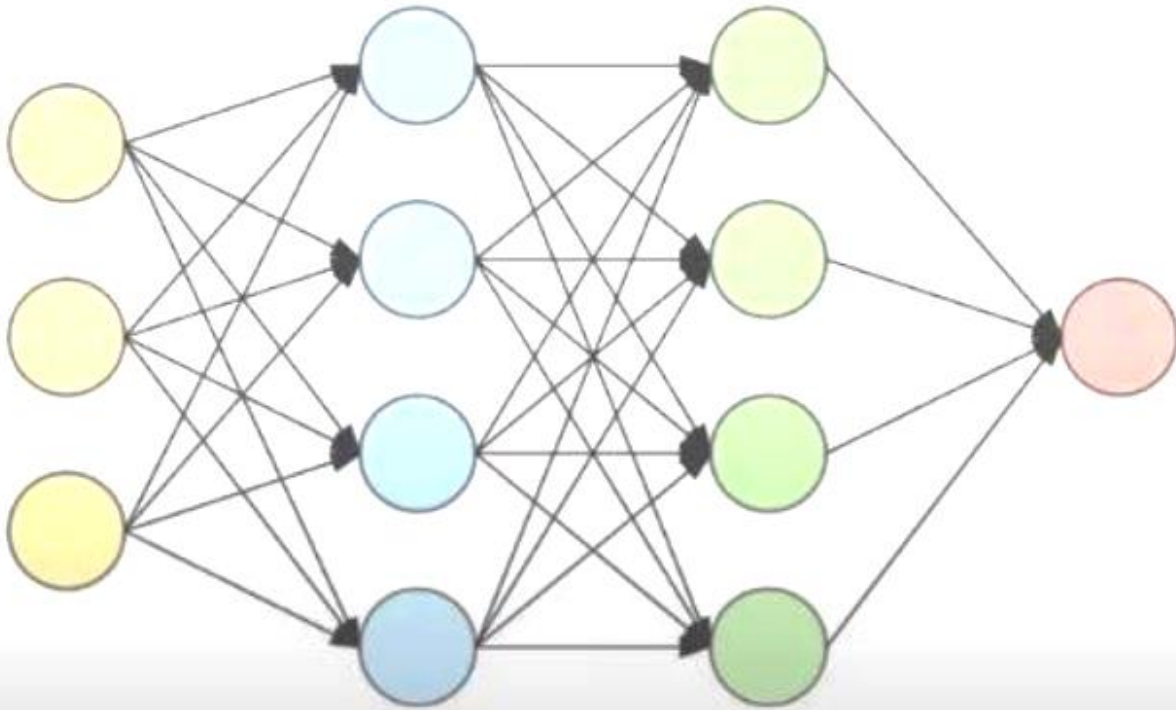
Rule of thumb:

You can begin with using ReLU function and then move over to other activation functions in case ReLU doesn't provide with optimum results.

Important Note:

Always keep in mind that in classification, activation functions should only be used in the hidden layers. For output layers, we use Softmax -sigmoid with cross-entropy loss for classification. However in regression, use linear or nothing in the output layer

Activation Functions in Neural Networks



Why we need Activation Functions ?

$$Z1 = W1 * A0 + B1$$

$$A1 = f (Z1)$$

$$Z2 = W2 * A1 + B2$$

$$A2 = f (Z2)$$

Why we need Activation Functions ?

$$Z1 = W1 * A0 + B1$$

~~$$A1 = f(Z1)$$~~

$$Z2 = W2 * A1 + B2$$

~~$$A2 = f(Z2)$$~~

$$A2 = W2 * (W1 * A0 + B1) + B2$$

$$A2 = W2 * W1 * A0 + W2 * B1 + B2$$

$$A2 = W' * A0 + B'$$

Why we need Activation Functions ?

$$Z1 = W1 * A0 + B1$$

~~$$A1 = f(Z1)$$~~

$$Z2 = W2 * A1 + B2$$

~~$$A2 = f(Z2)$$~~

$$Z3 = W3 * A2 + B3$$

~~$$A3 = f(Z3)$$~~

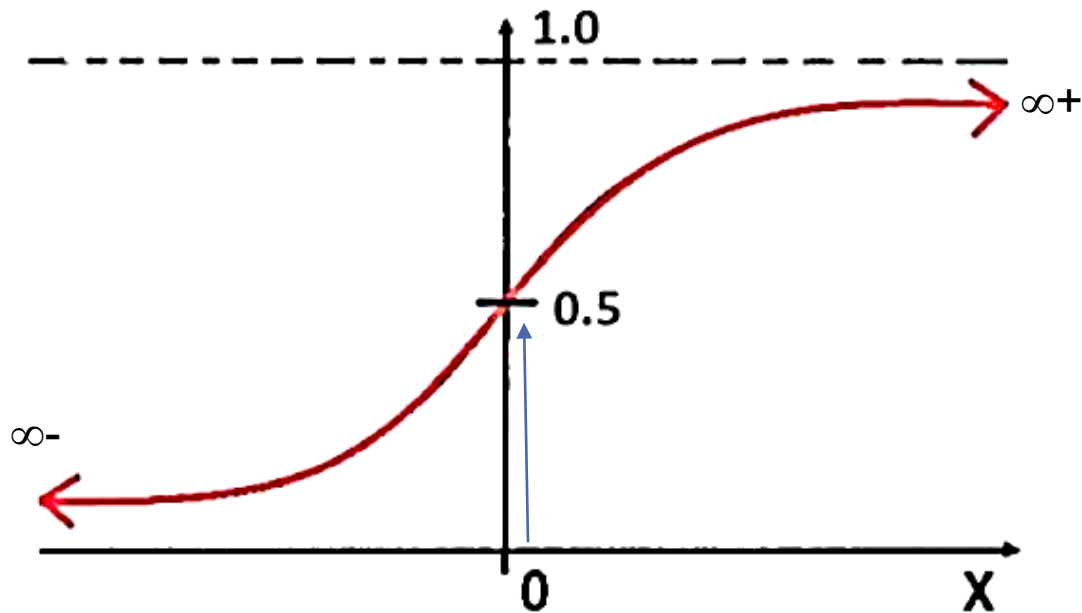
$$A2 = W2 * (W1 * A0 + B1) + B2$$

$$A2 = \underbrace{W2 * W1}_{W'} * A0 + \underbrace{W2 * B1 + B2}_{B'}$$

$$A2 = W' * A0 + B'$$

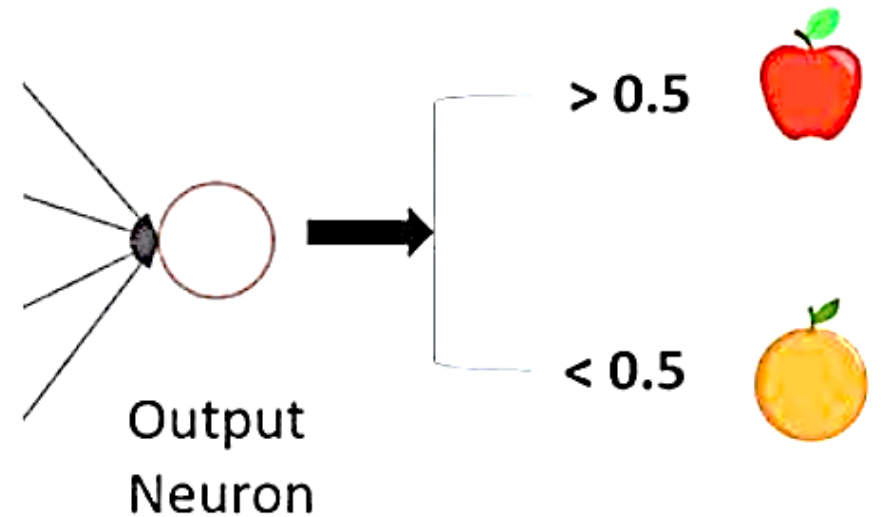
$$A3 = W'' * A0 + B''$$

Sigmoid Function

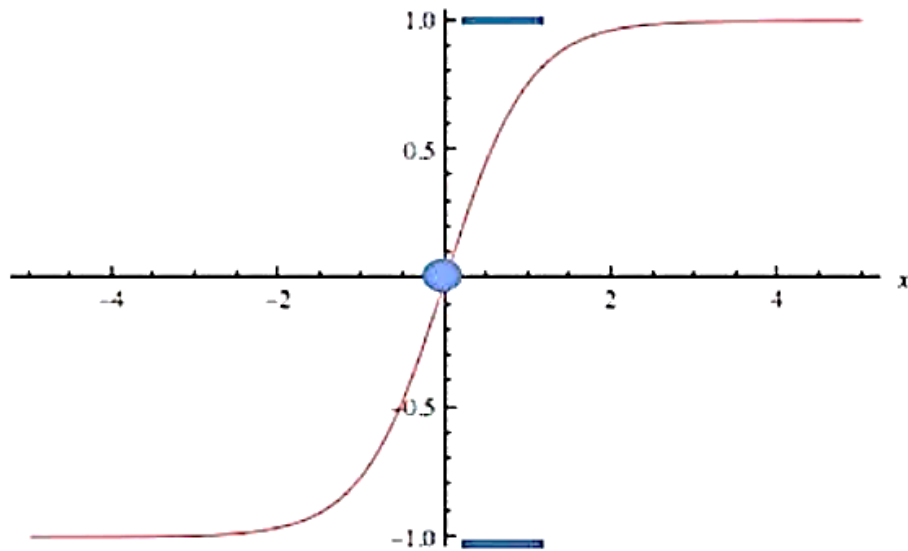


Sigmoid

$$f(x) = \frac{1}{1 + e^{-x}}$$



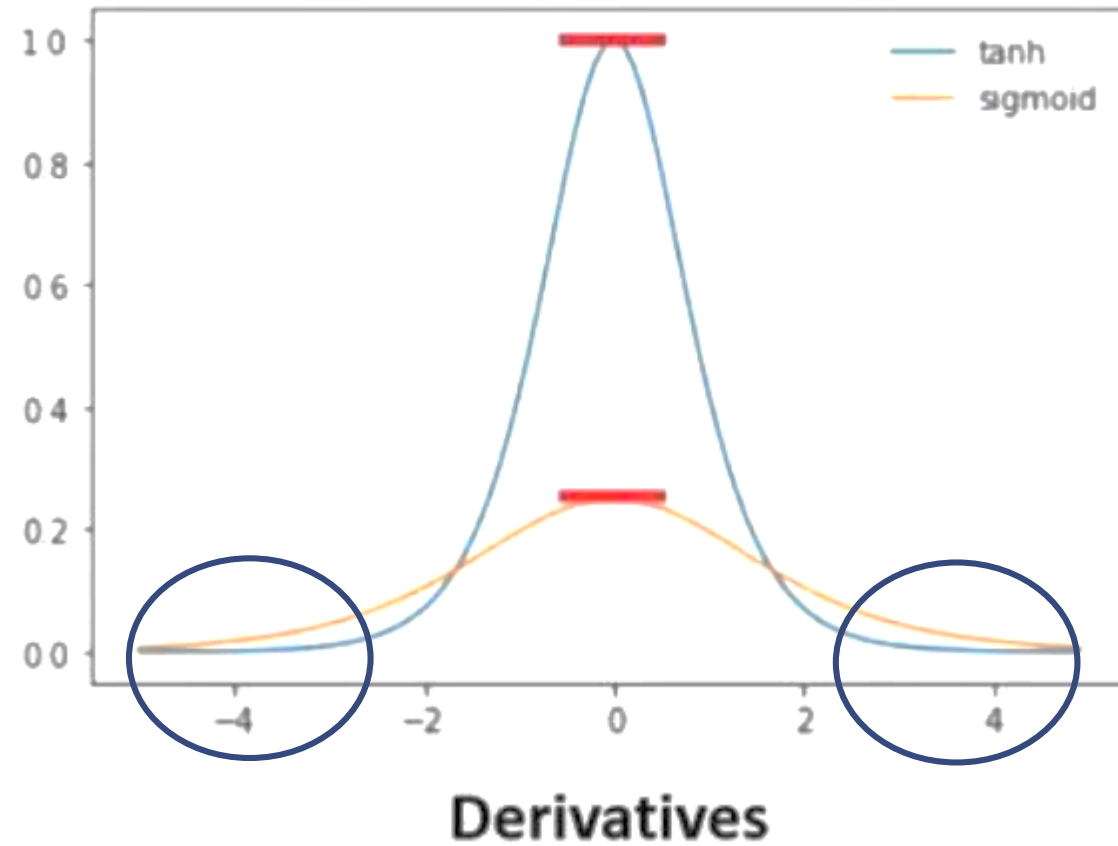
Tanh Function



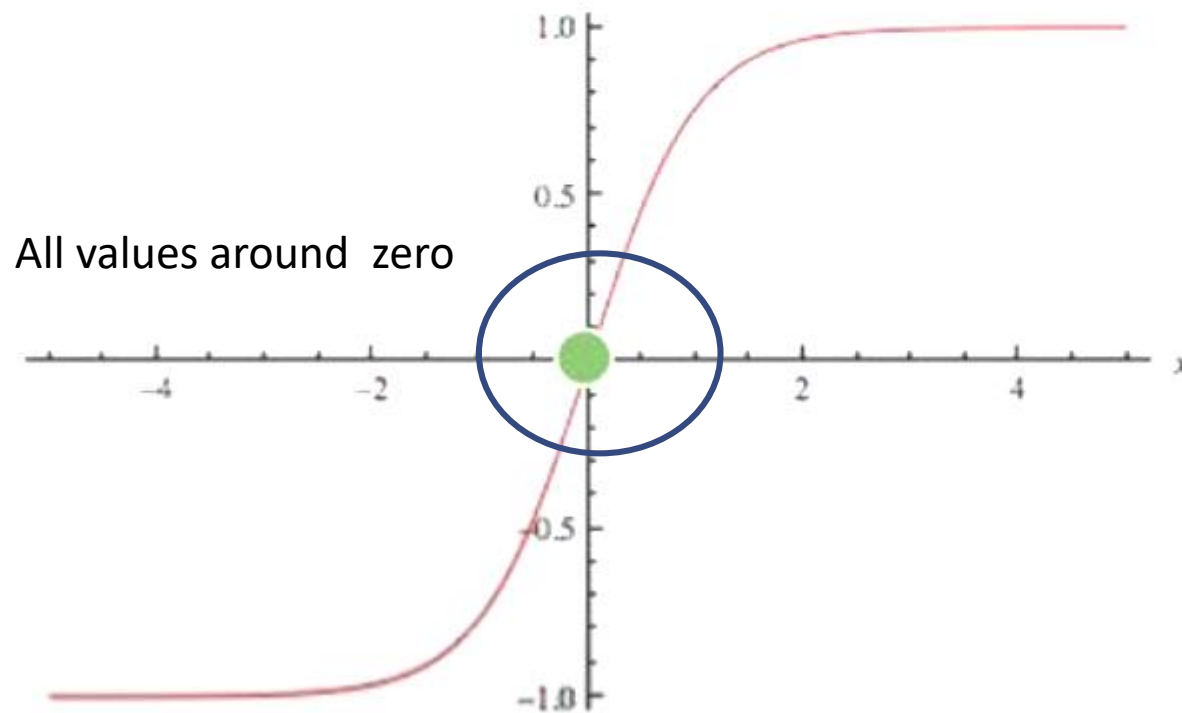
$$W = W - \text{alpha} * \frac{\partial \text{cost}}{\partial W}$$

$$\tanh = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

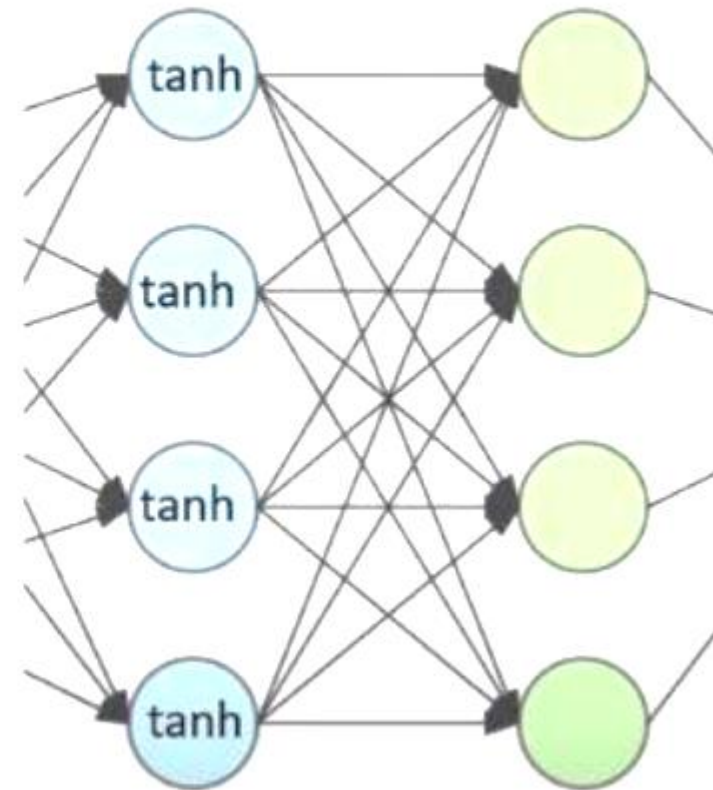
Deep Learning : Artificial Neural Network



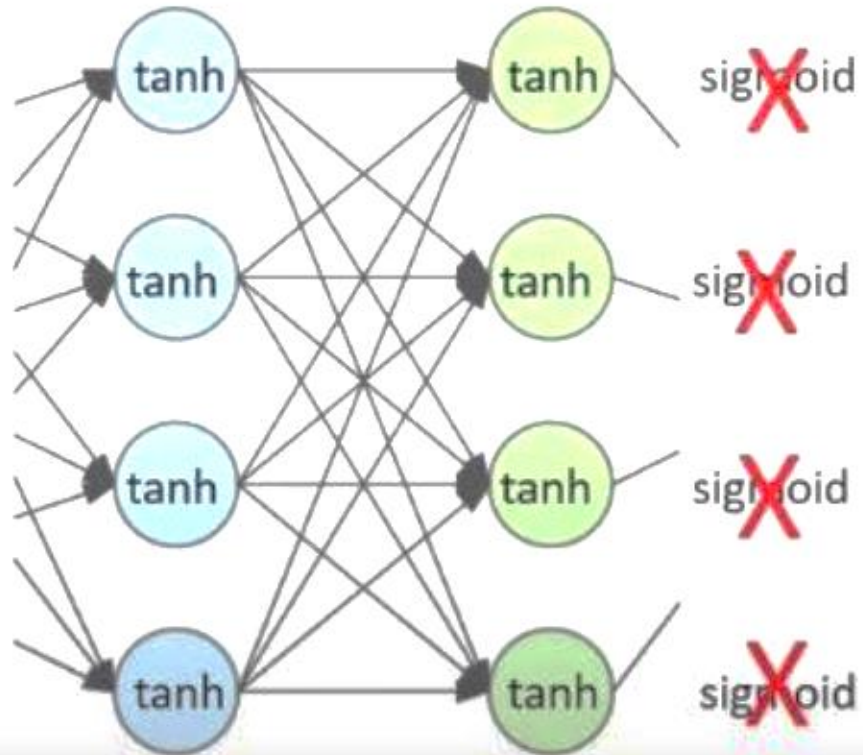
Tanh Function



Look like Normalization

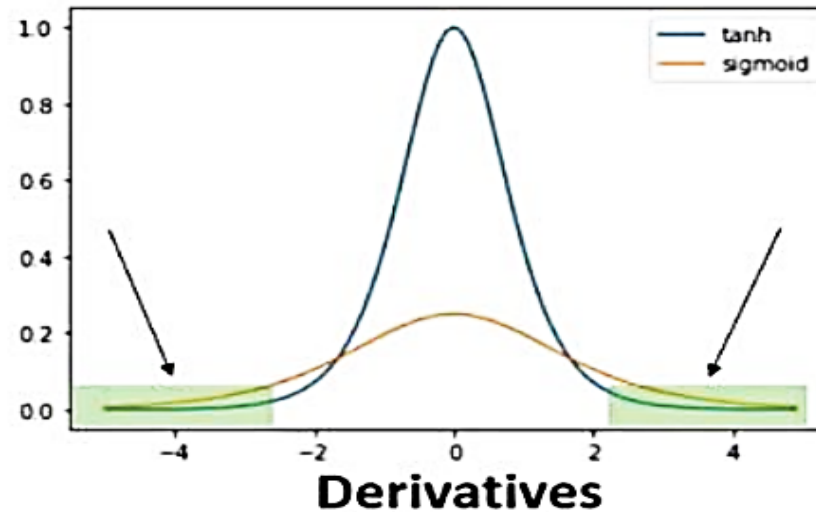
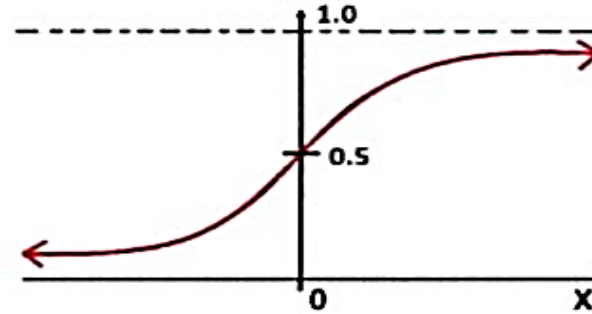
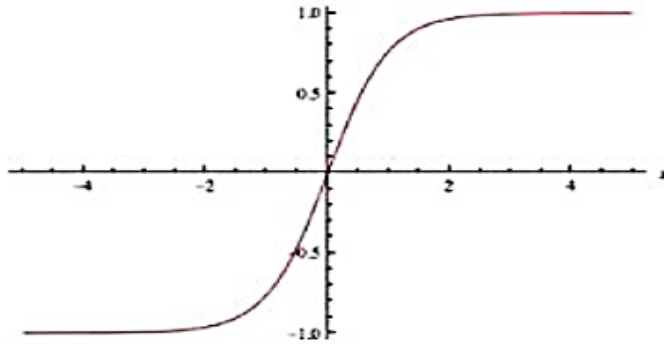


Tanh Function



This why we not Use Sigmoid function

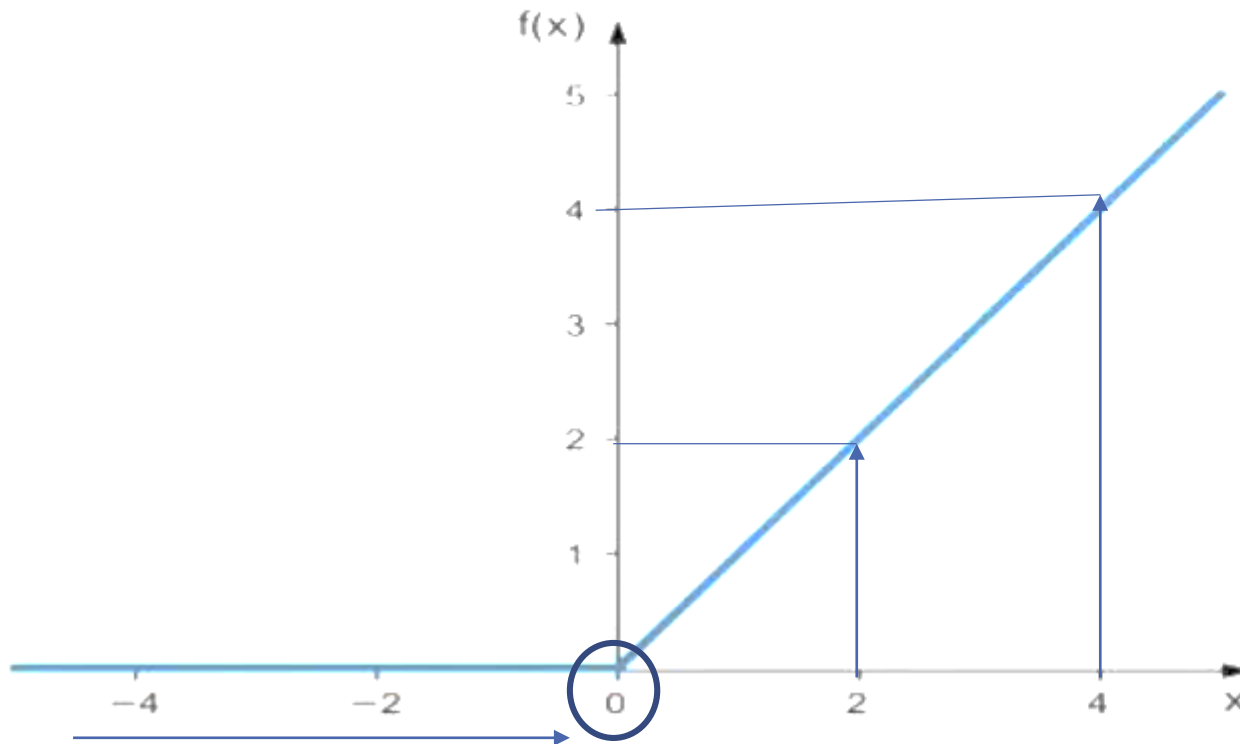
Deep Learning : Artificial Neural Network



Smaller value of derivative leads to very slow learning

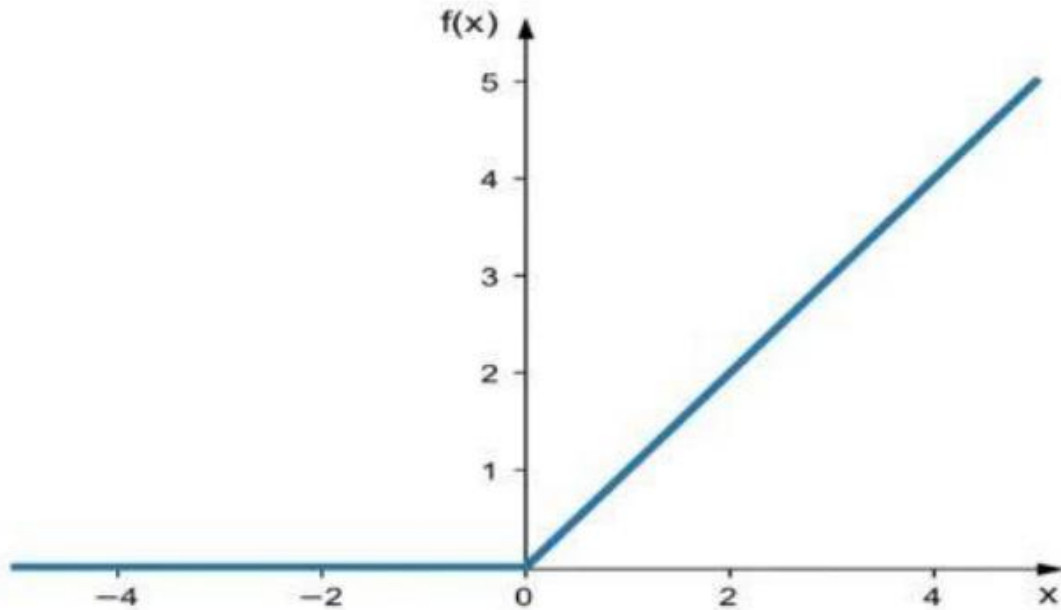
Vanishing Gradient Problem

ReLU (Rectified Linear Unit)



$$f(x) = \max(0, x)$$

ReLU (Rectified Linear Unit)

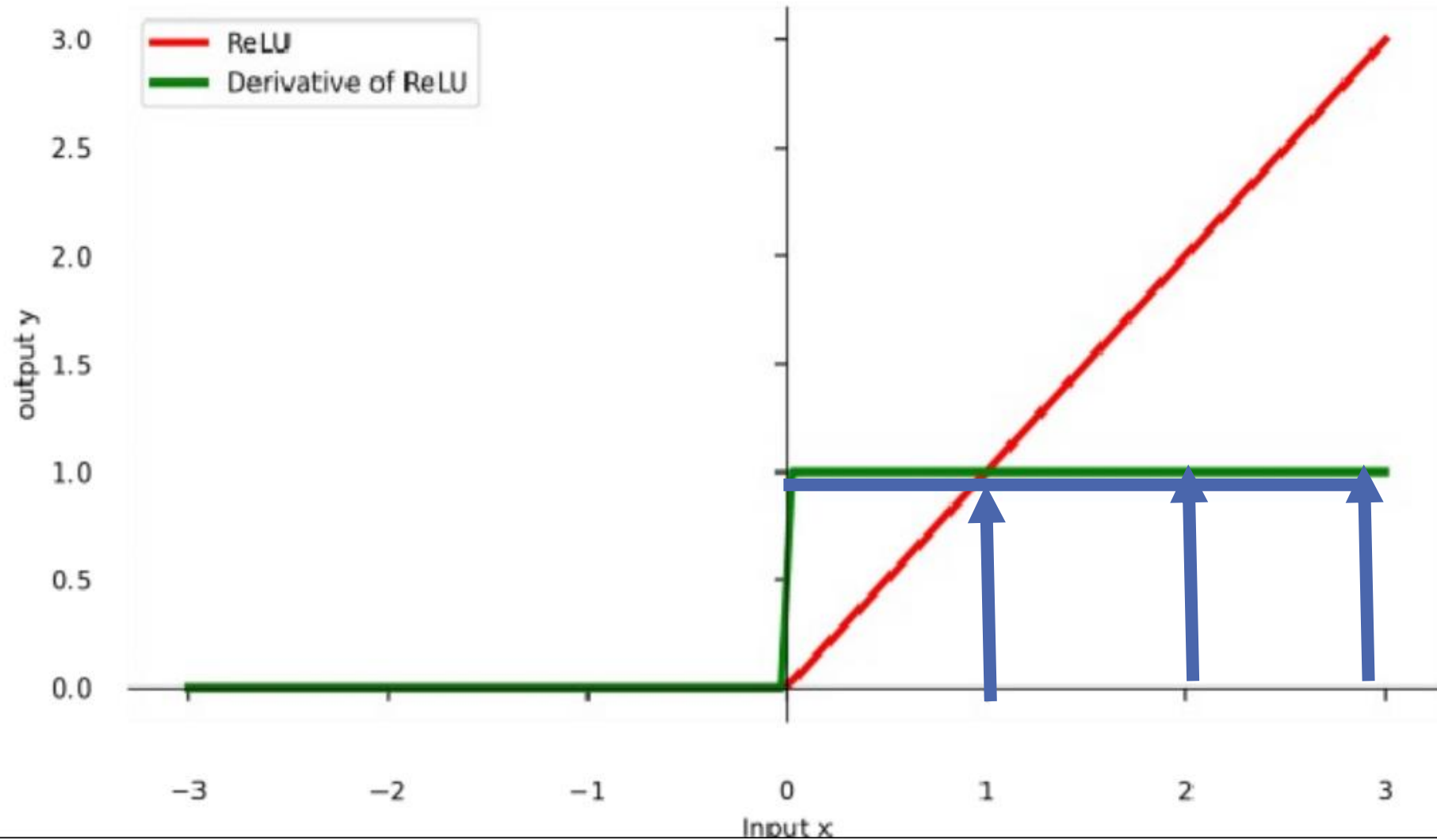


$$f(x) = \max(0, x)$$

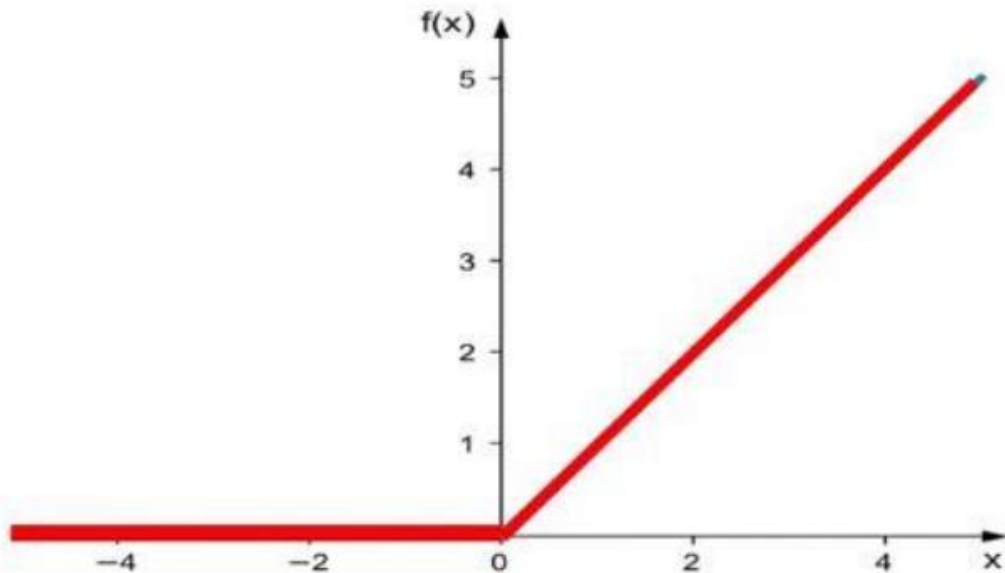
Overcome the
Vanishing Gradient
Problem

$$\frac{\partial f(x)}{\partial x} = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

Deep Learning : Artificial Neural Network



ReLU (Rectified Linear Unit)



Piece-wise Linear

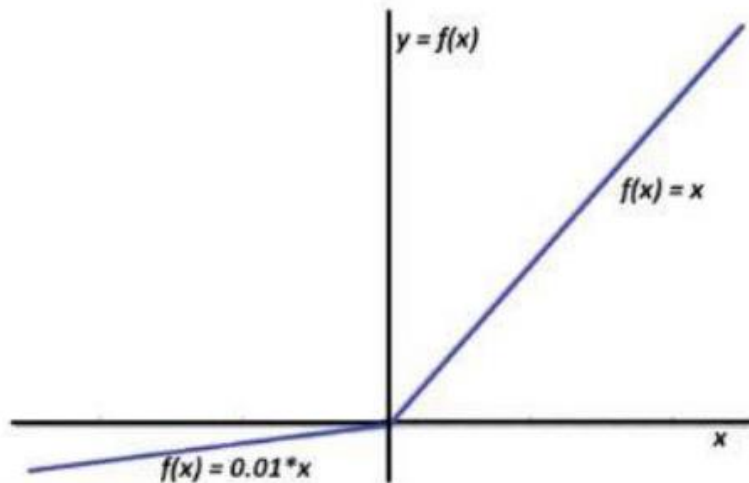
Advantages of both linear and non-linear property

$$f(x) = \max(0, x)$$

Overcome the
Vanishing Gradient
Problem

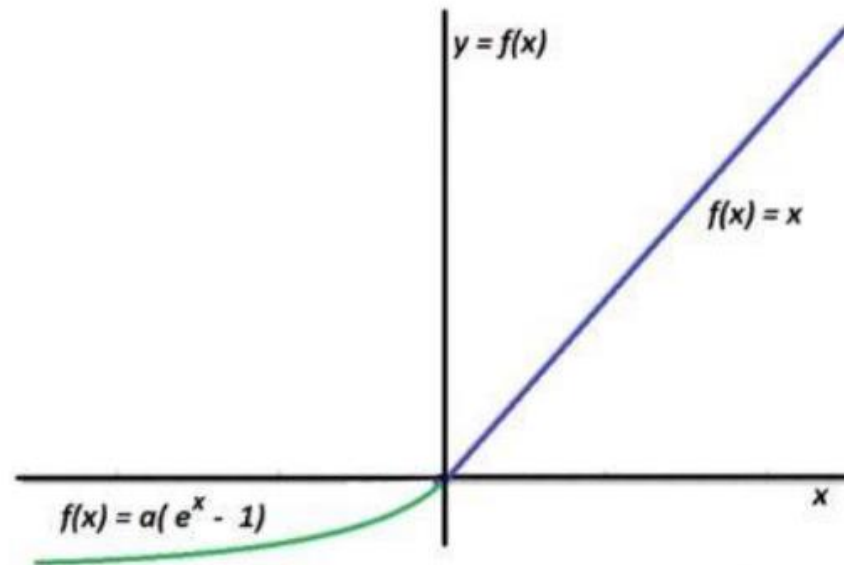
$$\frac{\partial f(x)}{\partial x} = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

Variations of ReLU



Leaky ReLU

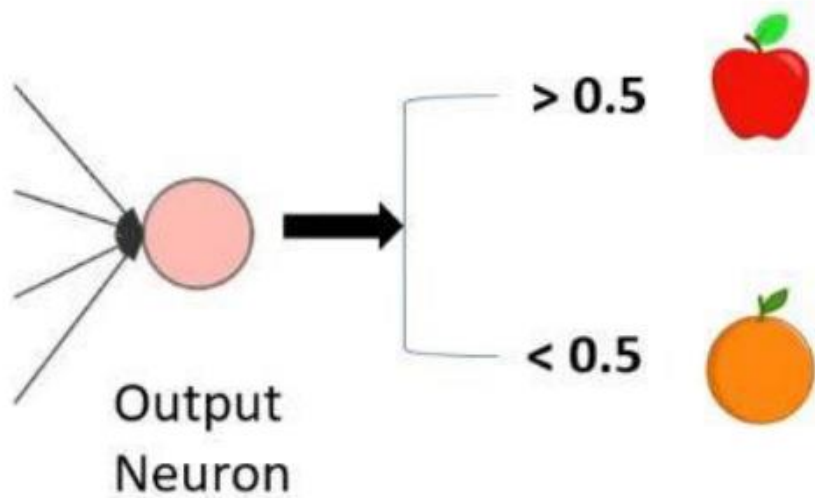
$$f(x) = \max(0.01 * x, x)$$



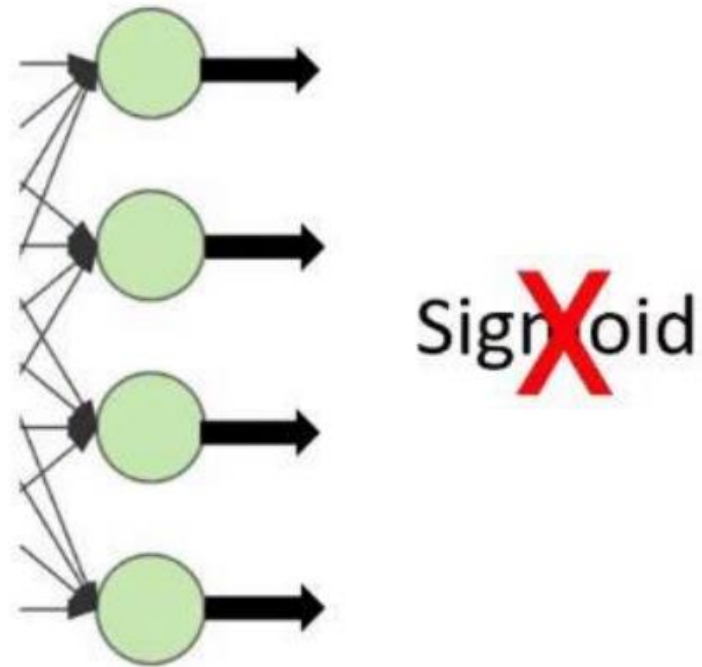
ELU (Exponential Linear Unit)

$$f(x) = \max(\alpha * (e^x - 1), x)$$

Deep Learning : Artificial Neural Network

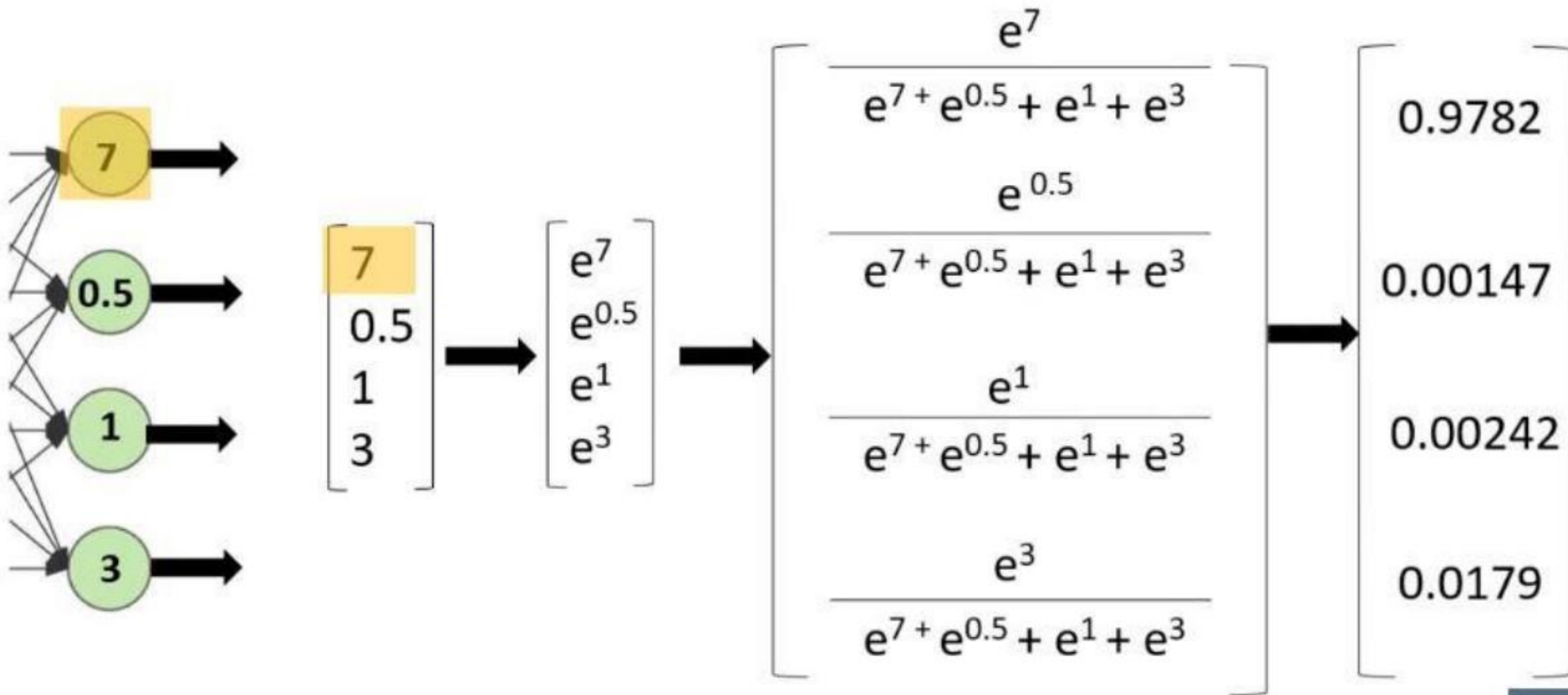


Binary Classification



Multi-Class Classification
??

Softmax Function

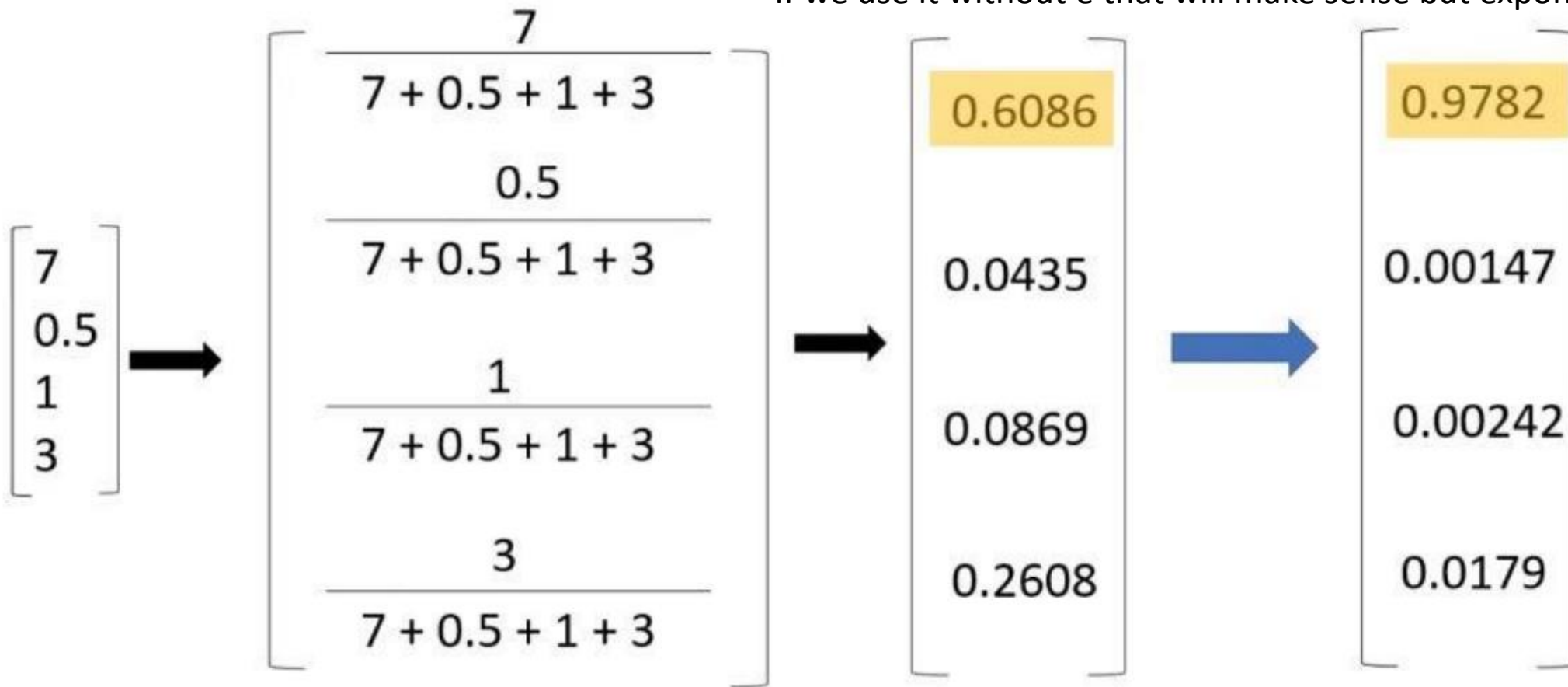


$$0.9782 + 0.00147 + 0.00242 + 0.0179 = 1$$



Softmax Function

If we use it without e that will make sense but exponential better than



Softmax Function

$$\text{Softmax } f_i(x) = \frac{e^{a_i}}{\sum_k e^{a_k}}$$

Linear Regression



House Price Prediction

*Price of the house can be
anywhere from 0 to \$1 million*



*No Activation
Function in
Output Neuron*

Vanishing Gradient Exploding Gradient

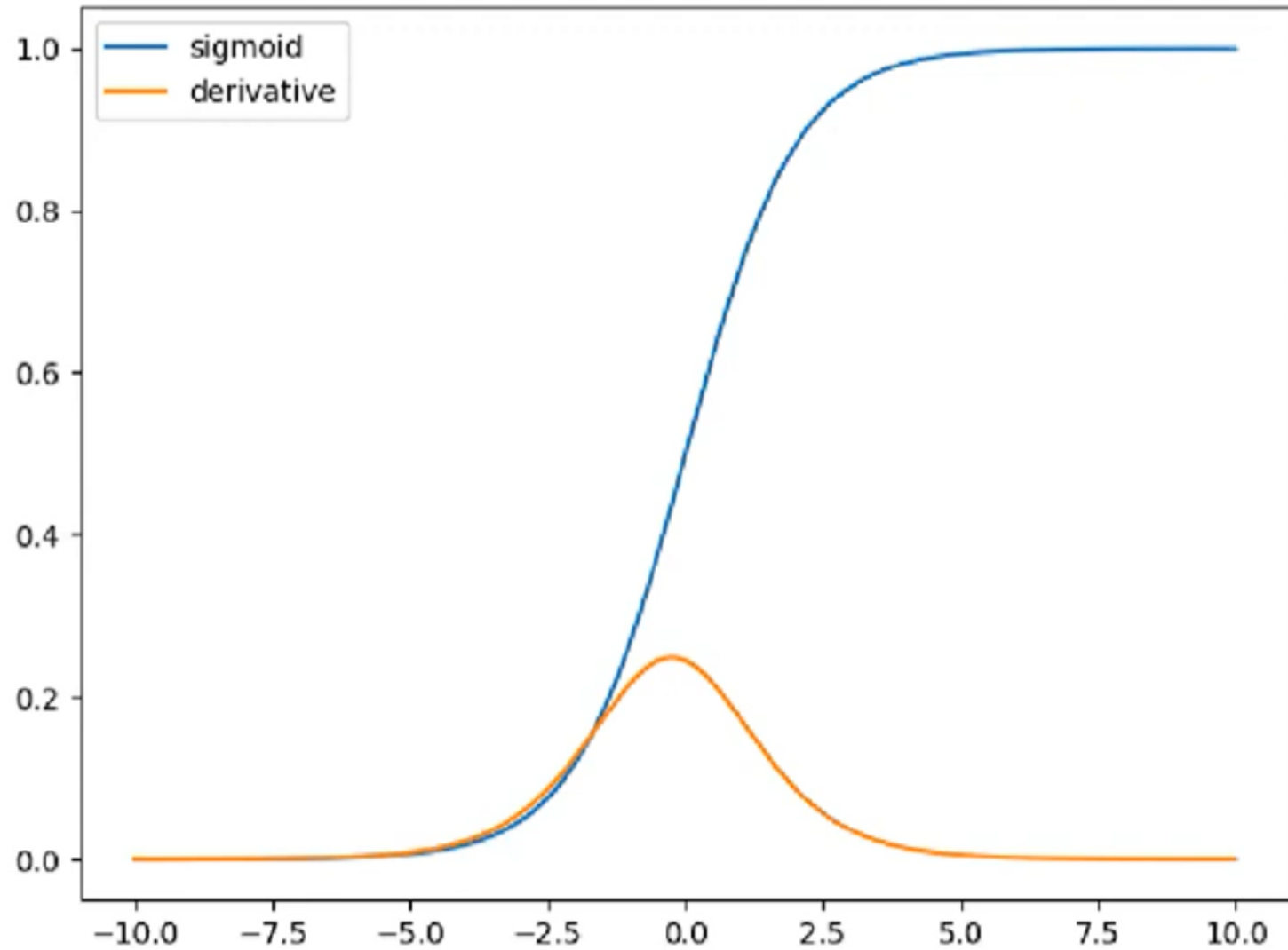
Vanishing Gradient in neural network

```
Epoch 1/10
60000/60000 [=====] -
Epoch 2/10
60000/60000 [=====] -
Epoch 3/10
60000/60000 [=====] -
Epoch 4/10
60000/60000 [=====] -
Epoch 5/10
60000/60000 [=====] -
Epoch 6/10
60000/60000 [=====] -
Epoch 7/10
60000/60000 [=====] -
Epoch 8/10
60000/60000 [=====] -
Epoch 9/10
60000/60000 [=====] -
Epoch 10/10
60000/60000 [=====] -
```

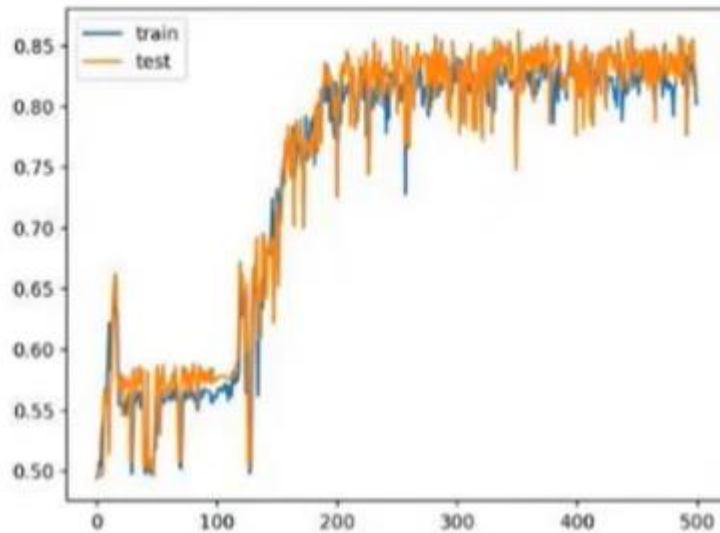

Deep Learning : Artificial Neural Network

```
Epoch 1/10
60000/60000 [-----] - 5s 82us/step - loss: 3.9591 accuracy: 0.1005
Epoch 2/10
60000/60000 [-----] - 4s 65us/step - loss: 2.3061 accuracy: 0.1043
Epoch 3/10
60000/60000 [-----] - 4s 67us/step - loss: 2.3066 accuracy: 0.1069
Epoch 4/10
60000/60000 [-----] - 5s 75us/step - loss: 2.3076 accuracy: 0.1050
Epoch 5/10
60000/60000 [-----] - 4s 72us/step - loss: 2.3079 accuracy: 0.1060
Epoch 6/10
60000/60000 [-----] - 4s 68us/step - loss: 2.3092 accuracy: 0.1035
Epoch 7/10
60000/60000 [-----] - 5s 77us/step - loss: 2.3085 accuracy: 0.1032
Epoch 8/10
60000/60000 [-----] - 4s 72us/step - loss: 2.3084 accuracy: 0.1060
Epoch 9/10
60000/60000 [-----] - 4s 62us/step - loss: 2.3085 accuracy: 0.1041
Epoch 10/10
60000/60000 [-----] - 4s 66us/step - loss: 2.3085 accuracy: 0.1069
```

Deep Learning : Artificial Neural Network



Exploding Gradient in Neural Network



Gradient Clipping

$$G = \begin{bmatrix} 1.02, -0.47, -0.92, \\ 0.30, 1.23, 0.27, \\ 0.93, 0.46, -1.95 \end{bmatrix}, \quad T = \pm 1$$

$$\text{Clip by value (G)} = \begin{bmatrix} 1.00, -0.47, -0.92, \\ 0.30, 1.00, 0.27, \\ 0.93, 0.46, -1.00 \end{bmatrix}$$

Overfitting

Regularization

L2 REGULARIZATION

$$\mathcal{L}' = \mathcal{L} + \beta \underbrace{\frac{1}{2} \|\omega\|_2^2}_{\frac{1}{2} (\omega_1^2 + \omega_2^2 + \dots + \omega_n^2)}$$

$$\left(\frac{1}{2}x^2\right)^1 = x \quad \left(\frac{1}{2}\omega_i\right)^1 = \omega_i$$

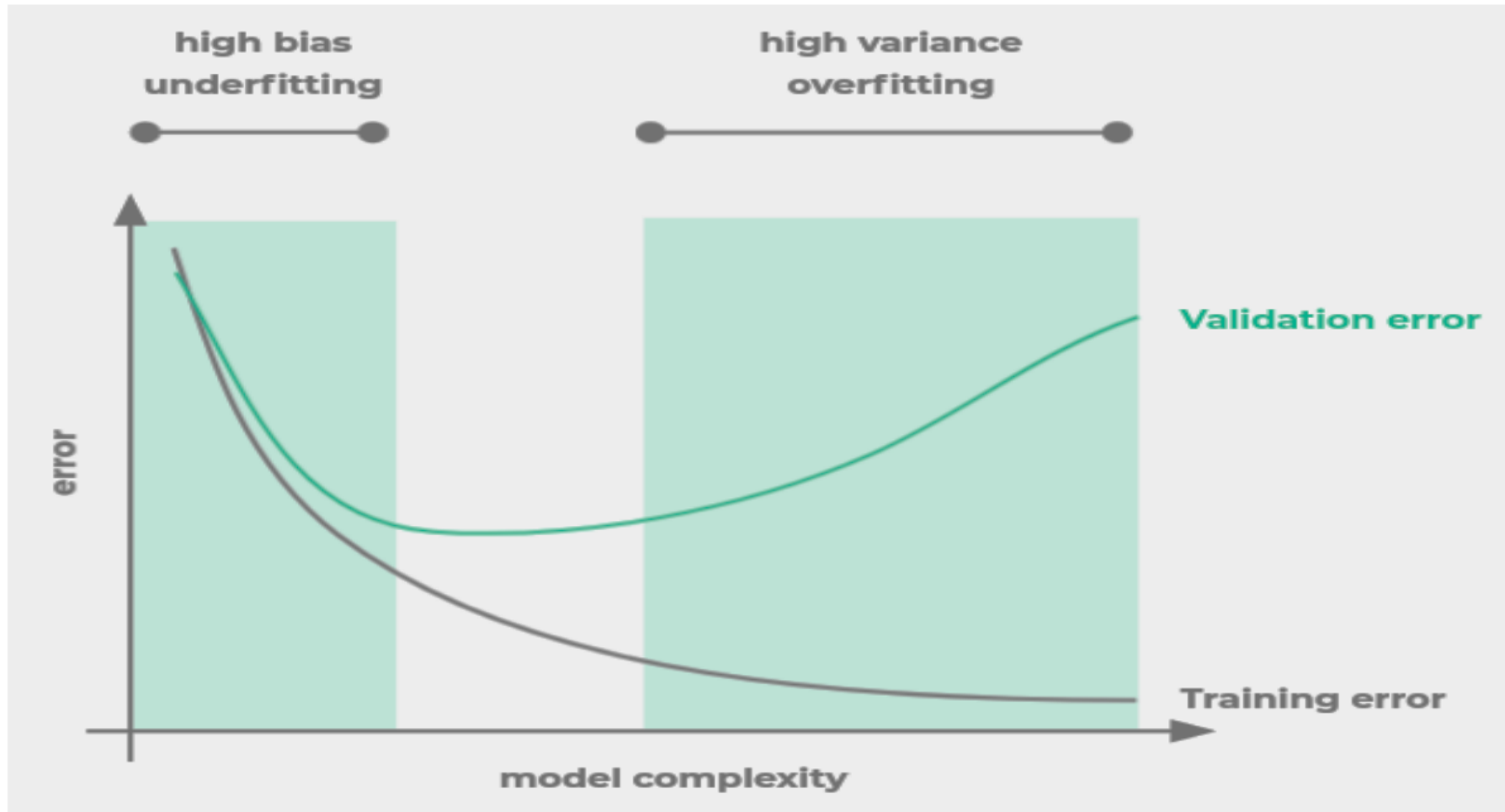
DERIVATIVE?

☐ $\frac{1}{3} \|\omega\|_2^3$

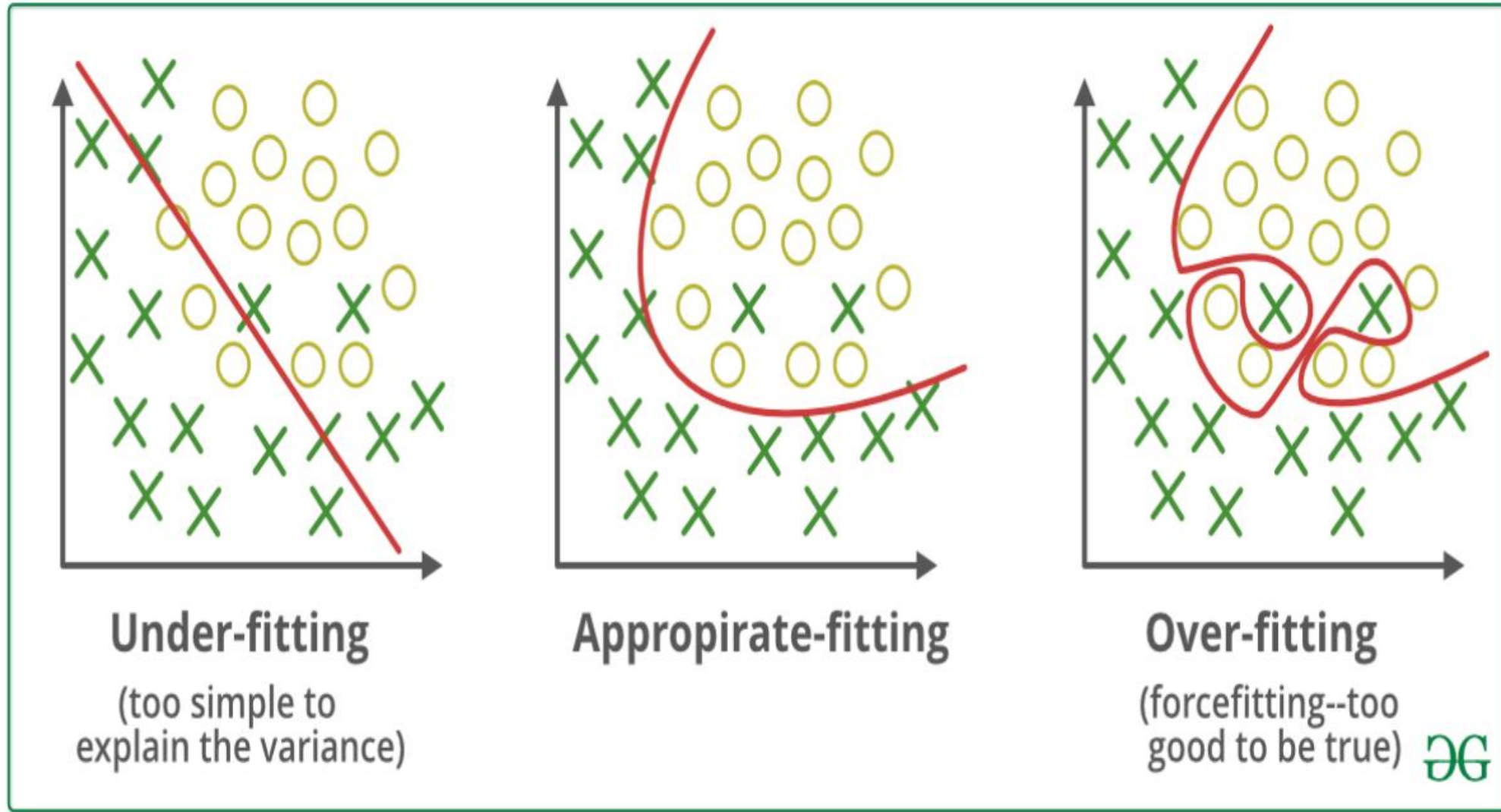
☐ $\omega^T \omega$

☒ ω

Dropout



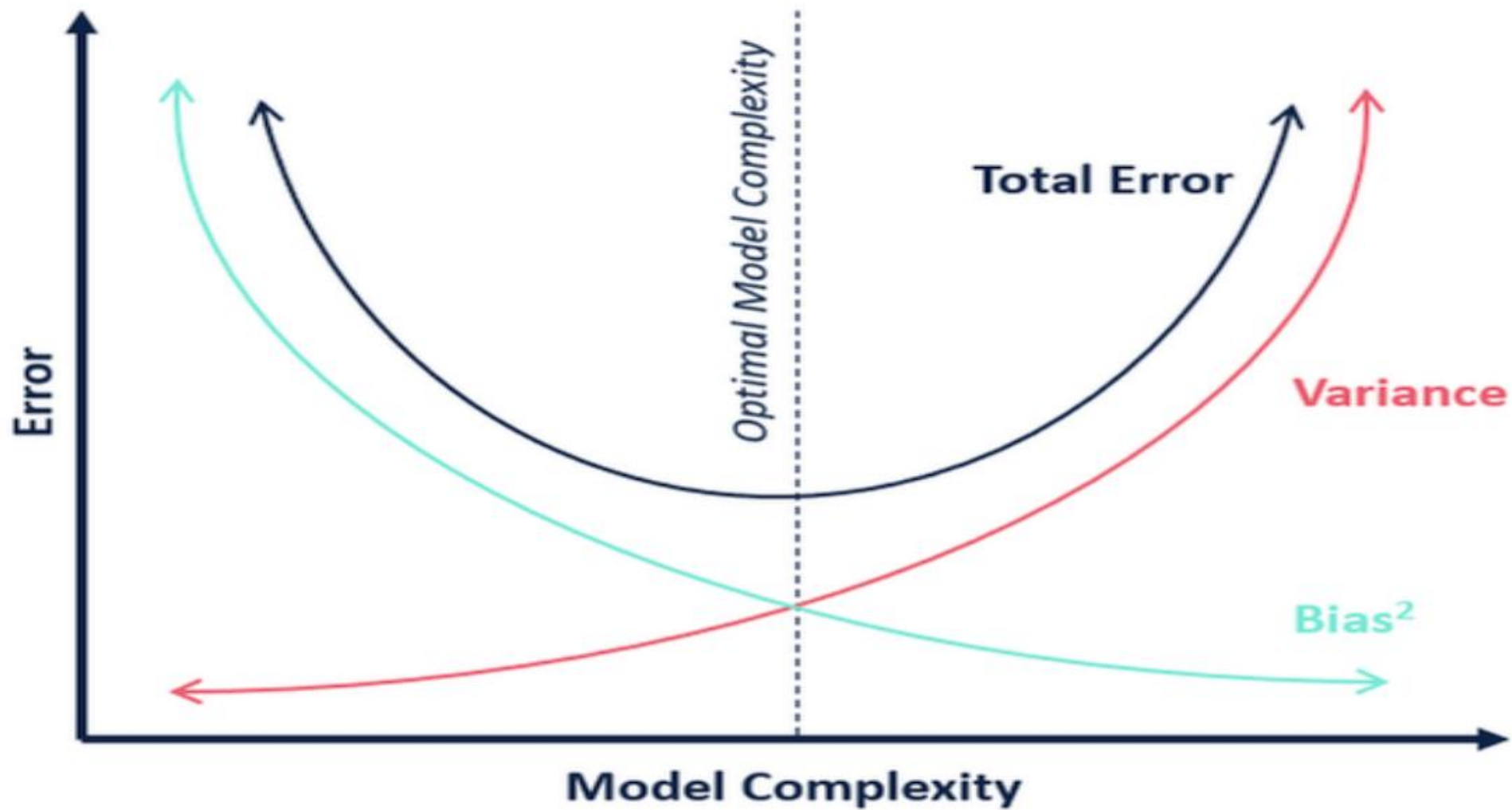
Deep Learning 1 : Artificial Neural Network



Deep Learning 1 : Artificial Neural Network

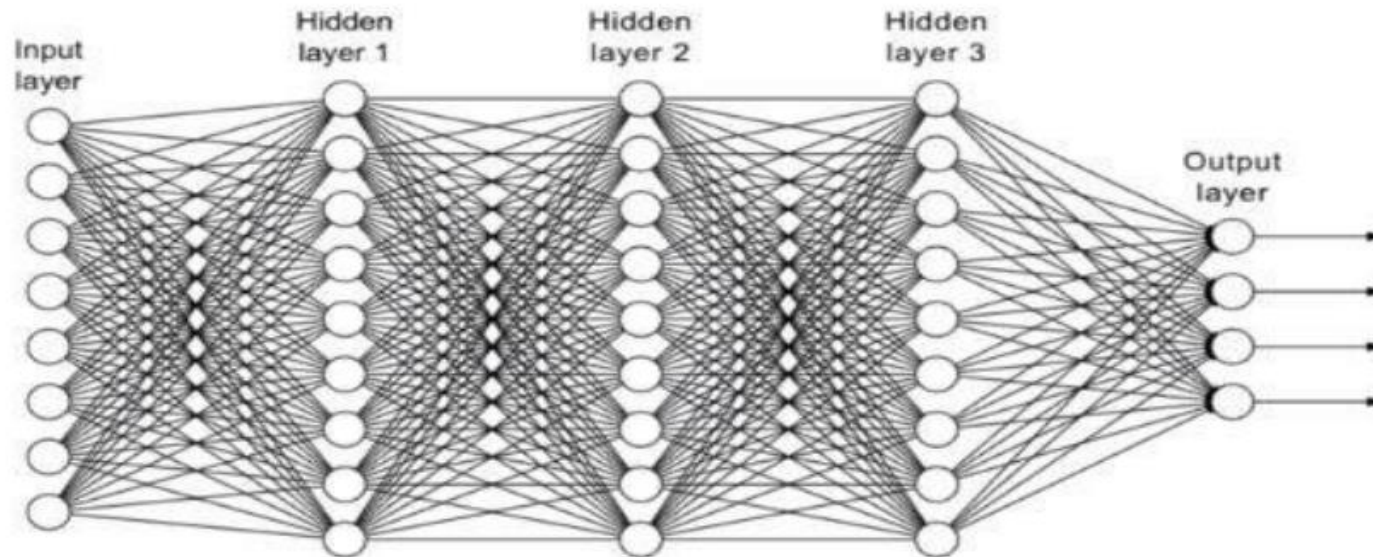
- low bias refers to a model that is able to capture the underlying patterns and relationships in the data accurately. It means that the model is flexible enough to fit the training data well.
- On the other hand, high variance refers to a model that is overly complex and sensitive to the training data. It means that the model may fit the training data too closely, resulting in poor generalization to new, unseen data.
- In simpler terms, low bias implies that the model is not making many assumptions about the data, allowing it to learn complex patterns. High variance, however, suggests that the model is overfitting the training data, resulting in poor performance on new data.
- Finding the right balance between low bias and high variance is crucial in machine learning, as both extremes can lead to poor model performance.

Deep Learning 1 : Artificial Neural Network

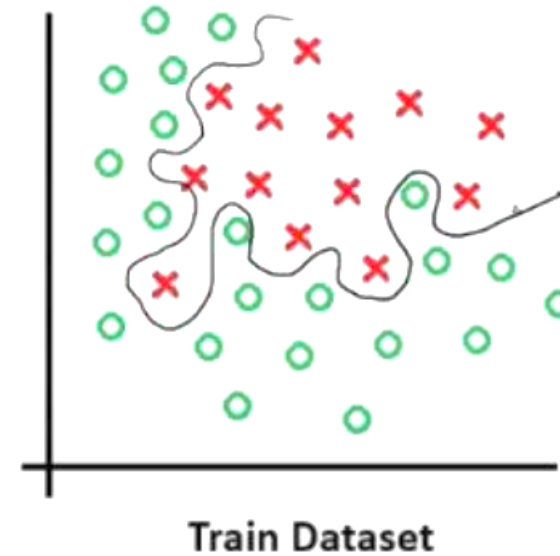
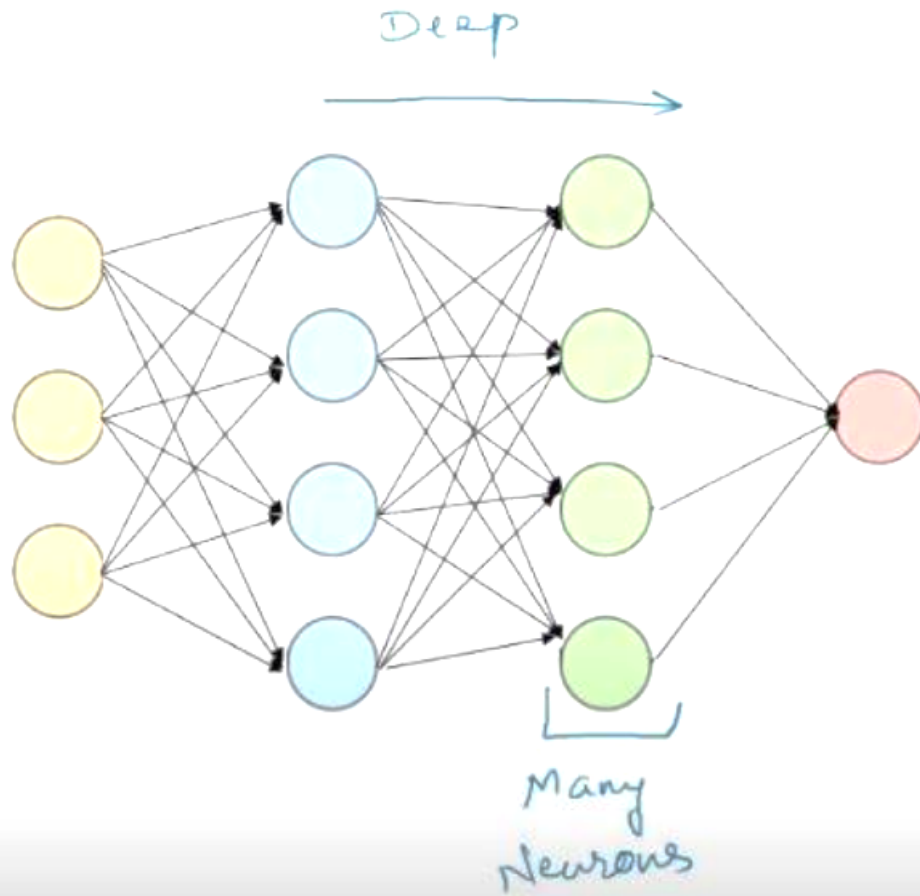


How to reduce Overfitting?

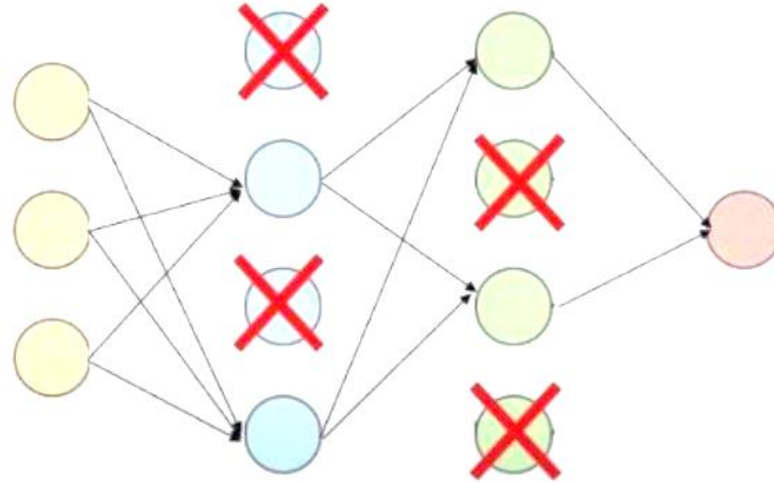
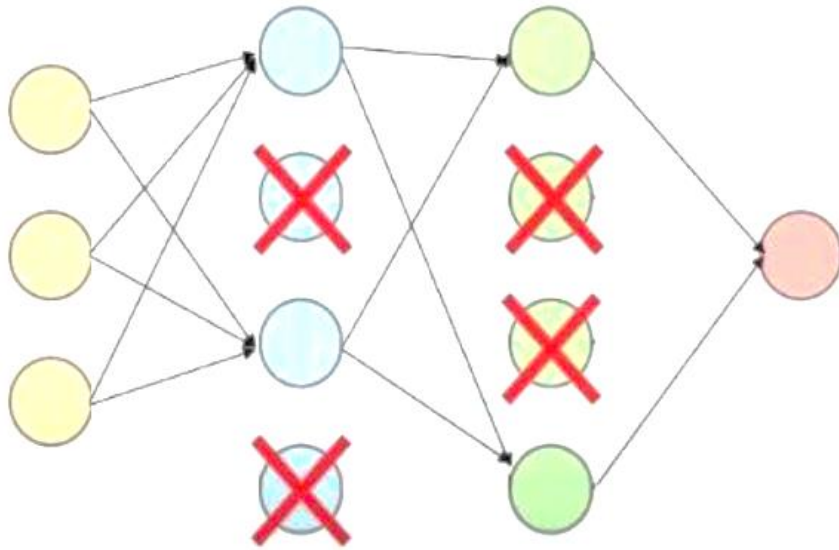
- Reduce complexity of Neural Network
- Regularization
- Dropout
- Reduce the number of epochs - the training duration



Deep Learning 1 : Artificial Neural Network



Deep Learning 1 : Artificial Neural Network



Deep Learning 1 : Artificial Neural Network

new = A

```
D = np.random.rand(A.shape[0], A.shape[1])
```

```
[[0.13691879 0.12486968 0.76703112 0.80144702]
 [0.61693956 0.71061175 0.39941722 0.57400298]
 [0.01633737 0.94949367 0.93394508 0.27578972]
 [0.46148637 0.09595069 0.96184166 0.66788549]
 [0.15366178 0.32287791 0.47652757 0.26758814]]
```

keep_rate = 0.8

```
D = D < keep_rate
```

```
[[ True  True  True False]
 [ True  True  True  True]
 [ True False False  True]
 [ True  True False  True]
 [ True  True  True  True]]
```

```
A = A * D
```

```
[[0.23869521 0.96682018 0.11877874 0.57609517]
 [0.94703416 0.51971021 0.11884226 0.26263551]
 [0.02637477 0.60834206 0.97092361 0.32589815]
 [0.34884988 0.36680243 0.95451758 0.83512696]
 [0.43897514 0.54082403 0.90827132 0.82675546]] *
[[ True  True  True False]
 [ True  True  True  True]
 [ True False False  True]
 [ True  True False  True]
 [ True  True  True  True]] =
[[0.23869521 0.96682018 0.11877874 0.
 [0.94703416 0.51971021 0.11884226 0.26263551]
 [0.02637477 0.
 [0.34884988 0.36680243 0.
 [0.43897514 0.54082403 0.90827132 0.82675546]]
```

Forward Propagation

$Z1 = W1 * A0 + B1$

$A1 = f (Z1)$

$D1 = \text{np.random.rand}(A1.\text{shape}[0], A1.\text{shape}[1]) < \text{keep_rate}$

$A1 = A1 * D1$

$Z2 = W2 * A1 + B2$

$A2 = f (Z2)$

$D2 = \text{np.random.rand}(A2.\text{shape}[0], A2.\text{shape}[1]) < \text{keep_rate}$

$A2 = A2 * D2$

$Z3 = W3 * A2 + B3$

$A3 = f (Z3)$

Forward Propagation

$$Z1 = W1 * A0 + B1$$

$$A1 = f (Z1)$$

$$\left\{ \begin{array}{l} D1 = \text{np.random.rand}(A1.\text{shape}[0], A1.\text{shape}[1]) < \text{keep_rate} \\ A1 = A1 * D1 / \text{keep_rate} \end{array} \right.$$

$$Z2 = W2 * A1 + B2$$

$$A2 = f (Z2)$$

$$\left\{ \begin{array}{l} D2 = \text{np.random.rand}(A2.\text{shape}[0], A2.\text{shape}[1]) < \text{keep_rate} \\ A2 = A2 * D2 / \text{keep_rate} \end{array} \right.$$

$$Z3 = W3 * A2 + B3$$

$$A3 = f (Z3)$$

Deep Learning 1 : Artificial Neural Network

Without dropout: the output of neurons is [0.2,0.3,0.5,0.7]

The total output would be $0.2+0.3+0.5+0.7=1.7$. With dropout (50% dropout rate): half the neurons are randomly deactivated. If neurons 1 and 3 are dropped out, the output becomes [0, 0.3, 0, 0.7] and the total output is $0.3+0.7=1.0$, which is much lower.

To correct for this, we multiply the remaining active neurons by. 2 (since $\frac{1}{1-0.5} = 2$)

Now the output becomes [0, 0.6, 0, 1.4] and the total output is $0.6+1.4=2.0$, closer to the original sum. This scaling ensures the model can learn as though all neurons are active, despite the dropout.

Deep Learning 1 : Artificial Neural Network

Let's say we have 4 neurons in a layer, and their outputs are [1,2,3,4]

- Without Dropout:** All neurons are active, and the total output is:

- Total output=1+2+3+4=10

- With Dropout (50% dropout rate):** Half the neurons are randomly deactivated. Suppose the first and third neurons are dropped. The outputs of the remaining neurons are [0, 2, 0, 4], and the total output is:

- Total output=0 + 2 + 0 + 4 = 6 This output is much smaller than 10, which would throw off the training process.

To correct for this, we scale the output of the remaining neurons by $\frac{1}{1-0.5} = 2$.

Now the outputs are [0, 4, 0, 8], and the total output becomes:

Total output= 0 + 4 + 0 + 8 = 12

This new total is much closer to the original 10, keeping the layer's behavior consistent despite dropout.

How to reduce Underfitting?

- Make the Neural Network more complex
- Add more training data so the network can learn more about the specific task
- Preprocessing of the data - Remove noise as an example
- Increase number of epochs - Increase training duration
- Fine-tune parameters for the network and training process

THANK YOU!